

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Khoa Công nghệ Thông tin

ĐỒ ÁN 2: MỘT SỐ CHỦ ĐỀ NỀN TẢNG
TRONG HỌC MÁY

Chương 10: BÀI TOÁN PHÂN HẠNG
(RANKING)

CSC14005 – Machine Learning

Nhóm: Group_11

Thành viên: Nguyễn Mạnh Thắng – 23120084
Trần Kim Ngọc – 23120062
Nguyễn Thành Nguyên – 23120063
Trần Đình Luân – 23120059
Nguyễn Thiện Nhân – 23120064

Năm học: 2025–2026

Mục lục

1	Giới thiệu chương	3
2	Phần 1: Báo cáo lý thuyết	3
2.1	Kiến thức chuẩn bị	3
2.1.1	Không gian giả thuyết và hàm mất mát	3
2.1.2	Rademacher Complexity	4
2.1.3	Support Vector Machine (SVM)	4
2.2	Bài toán Ranking	4
2.2.1	Phát biểu hình thức	4
2.2.2	Ví dụ minh hoạ	5
2.2.3	Thiết lập học thống kê	5
2.3	Generalization Bounds	5
2.3.1	Quan sát cốt lõi: Ranking như Classification trên cặp	6
2.3.2	Rademacher Complexity của lớp hàm ranking	6
2.3.3	Định lý chặn tổng quát hoá chính	6
2.3.4	Ý nghĩa của chặn	9
2.4	Ranking with SVMs	10
2.4.1	Nhắc lại SVM phân loại	10
2.4.2	Bài toán tối ưu hoá SVM Ranking	10
2.4.3	Bài toán đối ngẫu (Dual Problem)	11
2.4.4	Dự đoán và margin	12
2.4.5	So sánh với SVM phân loại	12
2.5	Mối liên hệ giữa các nội dung	13
2.6	RankBoost	14
2.6.1	Định nghĩa các đại lượng và Ký hiệu học thuật	14
2.6.2	Pseudocode RankBoost	15
2.6.3	Trực giác hoạt động của thuật toán	16
2.6.4	Đẳng thức toán học nền tảng	17
2.6.5	Cận trên sai số thực nghiệm (Empirical Error Bound)	17
2.6.6	Ví dụ tính toán từng bước minh hoạ RankBoost	21
2.6.7	Liên hệ với Coordinate Descent	23
2.6.8	Margin bound cho ensemble methods trong ranking	25
2.6.9	Khảo sát mối liên hệ học thuật với các chương khác trong giáo trình	27
2.7	Bipartite Ranking	29
2.7.1	Thiết lập Bài toán (Setting)	29
2.7.2	Generalization error và empirical error	29
2.7.3	Vấn đề hiệu năng tính toán: Thách thức mn cặp	30
2.7.4	BipartiteRankBoost: Độ phức tạp tuyến tính $\mathcal{O}(m + n)$	31
2.7.5	Độ đo AUC và đường cong ROC học thuật	34
2.7.6	Trực quan hoá Bipartite Ranking qua phân bố và ROC	35
2.7.7	Mối liên hệ học thuật AdaBoost $\leftrightarrow$ RankBoost	38
2.7.8	Khảo sát mối liên hệ học thuật với các chương khác trong giáo trình	40
2.8	Cách Tiếp Cận Dựa Trên Ưu Tiên (Preference-Based Setting)	42
2.8.1	Động Lực và So Sánh Với Score-Based Setting	42
2.8.2	Hai Giai Đoạn Của Preference-Based Setting	42

2.8.3	Mô Hình Xác Suất Cho Bài Toán Giai Đoạn Thứ Hai	43
2.8.4	Hàm Mất Mát Và Regret	43
2.8.5	Thuật Toán Xác Định: Sort-by-Degree	45
2.8.6	Chặn Dưới Cho Thuật Toán Xác Định (Theorem 10.5)	46
2.8.7	Thuật Toán Ngẫu Nhiên: QuickSort Dựa Trên Hàm Ưu Tiên	47
2.8.8	Mở Rộng Sang Các Hàm Mất Mát Có Trọng Số	49
2.9	Các Tiêu Chí Xếp Hạng Khác (Other Ranking Criteria)	50
2.9.1	Precision, Recall và Average Precision	50
2.9.2	DCG và NDCG	51
2.9.3	Mối Liên Hệ Giữa Các Tiêu Chí và Với AUC	53
2.9.4	Mối Liên Hệ Giữa Preference-Based Setting Và Các Phần Khác	53
3	Phần 2: Thực nghiệm minh họa	54
3.1	Thiết lập Dữ liệu Giả lập (Synthetic Data)	54
3.2	Thực nghiệm 1: Sự hội tụ và Chặn sai số lý thuyết của RankBoost	55
3.3	Thực nghiệm 2: Phân hạng Nhị phân (Bipartite) và Chỉ số ROC/AUC	56
3.4	Thực nghiệm 3: So sánh RankBoost vs. Ranking SVM	58
3.5	Các đoạn mã nguồn quan trọng	58
4	Phần 3: Nghiên cứu tiên tiến liên quan	59
4.1	Phân tích bài báo: Active Bipartite Ranking	59
4.1.1	Vấn đề	59
4.1.2	Đóng góp chính	59
4.1.3	Thiết lập và ký hiệu	60
4.1.4	Chiến lược Lấy mẫu Chủ động (Active-Rank)	60
4.1.5	Thực nghiệm kiểm chứng	61
4.1.6	Đánh giá phê bình và câu hỏi mở	61
4.1.7	Tóm tắt	62
4.1.8	Sinh viên Demo thực nghiệm	62
4.2	Phân tích bài báo: Which Tricks are Important for Learning to Rank?	64
4.2.1	Vấn đề	64
4.2.2	Đóng góp chính	64
4.2.3	Thiết lập và Hàm chất lượng xếp hạng	65
4.2.4	Ba trường phái thuật toán LTR	66
4.2.5	Thực nghiệm kiểm chứng	67
4.2.6	Đánh giá phê bình và câu hỏi mở	68
4.2.7	Tóm tắt	69
5	Kết luận	70

1. Giới thiệu chương

Bài toán **xếp hạng** (ranking) là một trong những bài toán trung tâm của học máy hiện đại, xuất hiện rộng rãi trong nhiều ứng dụng thực tế. Khi người dùng nhập một truy vấn vào công cụ tìm kiếm như Google, hệ thống không chỉ cần tìm ra các trang web liên quan mà còn phải *sắp xếp* chúng theo mức độ liên quan giảm dần. Tương tự, các hệ thống gợi ý như Netflix hay Spotify cần xếp hạng hàng nghìn bộ phim hay bài hát để đưa ra danh sách phù hợp nhất với từng người dùng.

Điểm khác biệt cốt lõi giữa ranking và các bài toán học máy truyền thống là: thay vì dự đoán một nhãn hay một giá trị tuyệt đối cho từng điểm dữ liệu, mục tiêu của ranking là học một **thứ tự tương đối** giữa các điểm. Nói cách khác, điều quan trọng không phải là điểm dữ liệu x được gán nhãn gì, mà là liệu x có đứng *trước* hay *sau* x' trong thứ tự xếp hạng.

Chương 10 của [4] trình bày một cách hệ thống các nền tảng lý thuyết cho bài toán ranking, bao gồm:

- **Định nghĩa hình thức bài toán ranking** (Mục 2.2): Phát biểu bài toán dưới dạng học hàm scoring từ các cặp dữ liệu có thứ tự ưu tiên.
- **Chặn tổng quát hoá** (Mục 2.3): Dẫn xuất các chặn trên cho ranking loss tổng quát dựa trên Rademacher complexity.
- **Ranking với SVM** (Mục 2.4): Mở rộng bài toán tối ưu hoá SVM sang không gian các cặp dữ liệu.
- **Thuật toán RankBoost** (Mục 2.6): Áp dụng tư tưởng boosting để kết hợp nhiều hàm ranking yếu thành một hàm ranking mạnh.
- **Bipartite ranking và AUC** (Mục 2.7): Xem xét trường hợp đặc biệt khi tập dữ liệu chia thành hai nhóm dương và âm, liên hệ trực tiếp với diện tích dưới đường cong ROC (AUC).
- **Preference-based setting** (Mục 2.8): Tổng quát hoá sang trường hợp học từ dữ liệu sở thích (preference feedback).

Về mặt lý thuyết, chương này được xây dựng trên nền tảng của hai chương trước trong sách: **Rademacher complexity** (Chương 3) được dùng trực tiếp để chứng minh các chặn tổng quát hoá, và **Support Vector Machines** (Chương 5) được mở rộng tự nhiên sang không gian cặp dữ liệu. Ngoài ra, RankBoost (Mục 2.6) kế thừa tư tưởng từ **AdaBoost** (Chương 7).

2. Phần 1: Báo cáo lý thuyết

2.1. Kiến thức chuẩn bị

2.1.1. Không gian giả thuyết và hàm mất mát

Định nghĩa 2.1. Cho tập mẫu $\mathcal{X}$, một **hàm scoring** là ánh xạ $f : \mathcal{X} \rightarrow \mathbb{R}$ dùng để gán điểm số cho từng điểm dữ liệu.

2.1.2. Rademacher Complexity

Nhắc lại khái niệm Rademacher complexity sẽ được dùng trong chứng minh generalization bound ở Phần 2.3.

Định nghĩa 2.2. Cho tập mẫu $S = \{x_1, \dots, x_m\}$ lấy từ phân phối $\mathcal{D}$, **Rademacher complexity thực nghiệm** của lớp hàm $\mathcal{H}$ là:

$$\hat{\mathfrak{R}}_S(\mathcal{H}) = \mathbb{E}_{\sigma} \left[\sup_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma_i h(x_i) \right]$$

trong đó σ_i là các biến ngẫu nhiên Rademacher độc lập.

2.1.3. Support Vector Machine (SVM)

SVM (Chương 5 của [4]) là thuật toán học có giám sát tìm siêu phẳng phân tách hai lớp với **margin lớn nhất**. Với tập huấn luyện $\{(z_i, y_i)\}_{i=1}^n$, $y_i \in \{-1, +1\}$, bài toán tối ưu hoá SVM soft-margin là:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \phi(z_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i, \end{aligned}$$

trong đó $\phi : \mathcal{X} \rightarrow \mathcal{F}$ là ánh xạ đặc trưng vào không gian Hilbert $\mathcal{F}$, $C > 0$ là tham số điều chỉnh. Ở Mục 2.4, ta sẽ thấy bài toán này được mở rộng tự nhiên sang không gian cặp (x, x') để giải bài toán ranking.

2.2. Bài toán Ranking

2.2.1. Phát biểu hình thức

Trong bài toán ranking, ta được cho một tập mẫu huấn luyện gồm các **cặp có thứ tự ưu tiên**. Cụ thể, mỗi mẫu huấn luyện là một cặp $(x, x') \in \mathcal{X} \times \mathcal{X}$ kèm theo thông tin rằng x được ưu tiên hơn x' , ký hiệu là $x \succ x'$.

Định nghĩa 2.3 – Hàm scoring. Một **hàm scoring** (hay hàm ranking) là ánh xạ $f : \mathcal{X} \rightarrow \mathbb{R}$ sao cho với mọi cặp (x, x') có $x \succ x'$, ta mong muốn:

$$f(x) > f(x').$$

Nói cách khác, điểm dữ liệu được ưu tiên hơn sẽ nhận điểm số (score) cao hơn.

Định nghĩa 2.4 – Ranking loss. Cho hàm scoring $f : \mathcal{X} \rightarrow \mathbb{R}$, **ranking loss** (hay pairwise loss) trên một cặp (x, x') với $x \succ x'$ được định nghĩa là:

$$\ell(f, (x, x')) = \mathbf{1}[f(x) < f(x')] + \frac{1}{2} \mathbf{1}[f(x) = f(x')],$$

trong đó $\mathbf{1}[\cdot]$ là hàm chỉ thị. Ranking loss bằng 1 khi f xếp nhầm thứ tự, bằng $\frac{1}{2}$ khi f không phân biệt được hai điểm, và bằng 0 khi f xếp đúng.

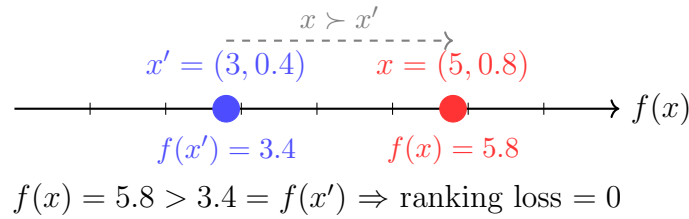
2.2.2. Ví dụ minh họa

Ví dụ 2.1. Xét bài toán xếp hạng kết quả tìm kiếm với $\mathcal{X} = \mathbb{R}^2$, trong đó mỗi điểm dữ liệu $x = (x_1, x_2)$ biểu diễn một trang web với hai đặc trưng: x_1 là số từ khớp với truy vấn, x_2 là độ uy tín của trang.

Giả sử ta có hai trang web $x = (5, 0.8)$ và $x' = (3, 0.4)$, và nhãn $x \succ x'$ (trang x liên quan hơn). Xét hàm scoring tuyến tính $f(x) = w_1 x_1 + w_2 x_2$ với $w_1 = w_2 = 1$:

$$f(x) = 5 + 0.8 = 5.8, \quad f(x') = 3 + 0.4 = 3.4.$$

Vì $f(x) = 5.8 > 3.4 = f(x')$, hàm f xếp đúng thứ tự, ranking loss bằng 0. Hình 1 minh họa trực quan thứ tự này.



Hình 1: Minh họa hàm scoring $f(x) = w_1 x_1 + w_2 x_2$ với $w_1 = w_2 = 1$ trên Ví dụ 2.1. Vì $f(x) > f(x')$, hàm f xếp đúng thứ tự $x \succ x'$ và ranking loss bằng 0.

2.2.3. Thiết lập học thống kê

Giả sử các cặp (x, x') được lấy mẫu độc lập từ một phân phối $\mathcal{D}$ trên $\mathcal{X} \times \mathcal{X}$. **Ranking loss tổng quát** (generalization ranking loss) của hàm scoring f được định nghĩa là:

$$R(f) = \Pr_{(x, x') \sim \mathcal{D}} [f(x) < f(x')] + \frac{1}{2} \Pr_{(x, x') \sim \mathcal{D}} [f(x) = f(x')].$$

Cho tập huấn luyện $S = \{(x_1, x'_1), \dots, (x_m, x'_m)\}$ gồm m cặp lấy mẫu i.i.d. từ $\mathcal{D}$, **ranking loss thực nghiệm** được tính là:

$$\hat{R}_S(f) = \frac{1}{m} \sum_{i=1}^m \left(\mathbf{1}[f(x_i) < f(x'_i)] + \frac{1}{2} \mathbf{1}[f(x_i) = f(x'_i)] \right).$$

Mục tiêu của thuật toán học là tìm hàm f trong lớp giả thuyết $\mathcal{H}$ sao cho $R(f)$ nhỏ nhất có thể, dựa trên thông tin từ tập huấn luyện S .

2.3. Generalization Bounds

Mục tiêu của phần này là trả lời câu hỏi: *Nếu một hàm scoring f có ranking loss thực nghiệm nhỏ trên tập huấn luyện, liệu ranking loss tổng quát của nó có nhỏ không?* Để trả lời, ta dẫn xuất các chặn tổng quát hoá (generalization bounds) cho bài toán ranking dựa trên Rademacher complexity.

2.3.1. Quan sát cốt lõi: Ranking như Classification trên cặp

Ý tưởng then chốt là quy bài toán ranking về bài toán phân loại nhị phân trên không gian các cặp.

Cho lớp hàm scoring $\mathcal{H}$ trên $\mathcal{X}$, ta định nghĩa lớp hàm tương ứng trên không gian cặp $\mathcal{X} \times \mathcal{X}$:

$$\mathcal{G} = \{g : (x, x') \mapsto f(x) - f(x') \mid f \in \mathcal{H}\}.$$

Khi đó, với mỗi cặp (x, x') có $x \succ x'$:

$$\begin{aligned} \ell(f, (x, x')) &= \mathbf{1}[f(x) < f(x')] + \frac{1}{2}\mathbf{1}[f(x) = f(x')] \\ &= \mathbf{1}[g(x, x') < 0] + \frac{1}{2}\mathbf{1}[g(x, x') = 0]. \end{aligned}$$

Đây chính xác là dạng **0-1 loss** của bài toán phân loại nhị phân, trong đó $g(x, x')$ đóng vai trò như hàm phân loại và nhãn luôn là $+1$ (vì $x \succ x'$).

2.3.2. Rademacher Complexity của lớp hàm ranking

Bổ đề 2.1. Cho lớp hàm scoring $\mathcal{H}$ trên $\mathcal{X}$ và tập mẫu cặp $S = \{(x_1, x'_1), \dots, (x_m, x'_m)\}$. Khi đó Rademacher complexity thực nghiệm của $\mathcal{G}$ thoả mãn:

$$\hat{\mathfrak{R}}_S(\mathcal{G}) \leq 2\hat{\mathfrak{R}}_{S^+}(\mathcal{H}),$$

trong đó $S^+ = \{x_1, x'_1, x_2, x'_2, \dots, x_m, x'_m\}$ là tập gồm $2m$ điểm đơn lẻ.

Chứng minh. Theo định nghĩa:

$$\begin{aligned} \hat{\mathfrak{R}}_S(\mathcal{G}) &= \mathbb{E}_{\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma_i (f(x_i) - f(x'_i)) \right] \\ &= \mathbb{E}_{\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{m} \left(\sum_{i=1}^m \sigma_i f(x_i) - \sum_{i=1}^m \sigma_i f(x'_i) \right) \right]. \end{aligned}$$

Vì σ_i và $-\sigma_i$ có cùng phân phối (đối xứng quanh 0), ta có $-\sigma_i f(x'_i) \stackrel{d}{=} \sigma'_i f(x'_i)$ với σ'_i là biến Rademacher độc lập. Áp dụng bất đẳng thức tam giác cho sup:

$$\begin{aligned} \hat{\mathfrak{R}}_S(\mathcal{G}) &\leq \mathbb{E}_{\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma_i f(x_i) \right] + \mathbb{E}_{\sigma'} \left[\sup_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma'_i f(x'_i) \right] \\ &= 2\hat{\mathfrak{R}}_{S^+}(\mathcal{H}). \end{aligned}$$

□

2.3.3. Định lý chặn tổng quát hoá chính

Định lý 2.1 – Generalization bound cho Ranking. Cho lớp hàm scoring $\mathcal{H}$ với $f : \mathcal{X} \rightarrow [0, 1]$ với mọi $f \in \mathcal{H}$. Cho $S = \{(x_i, x'_i)\}_{i=1}^m$ lấy mẫu i.i.d. từ phân phối $\mathcal{D}$ trên $\mathcal{X} \times \mathcal{X}$. Khi đó với mọi $\delta > 0$, với xác suất ít nhất $1 - \delta$ trên việc chọn mẫu S , đồng thời với mọi $f \in \mathcal{H}$:

$$R(f) \leq \hat{R}_S(f) + 4\hat{\mathfrak{R}}_{S^+}(\mathcal{H}) + \sqrt{\frac{\log(2/\delta)}{2m}}.$$

Chứng minh. Chứng minh gồm ba bước.

Bước 1: Áp dụng bất đẳng thức McDiarmid.

Xét hàm $\Phi(S) = \sup_{f \in \mathcal{H}} [R(f) - \hat{R}_S(f)]$. Khi thay một cặp (x_i, x'_i) bằng (z_i, z'_i) , giá trị $\hat{R}_S(f)$ thay đổi không quá $\frac{1}{m}$ với mọi f , do đó Φ thay đổi không quá $\frac{1}{m}$. Theo bất đẳng thức McDiarmid:

$$\Pr [\Phi(S) - \mathbb{E}[\Phi(S)] \geq \epsilon] \leq \exp(-2m\epsilon^2).$$

Đặt về phải bằng $\delta/2$ và giải ra ϵ :

$$\Phi(S) \leq \mathbb{E}_S[\Phi(S)] + \sqrt{\frac{\log(2/\delta)}{2m}}.$$

Bước 2: Chặn $\mathbb{E}_S[\Phi(S)]$ bằng Rademacher complexity.

Dùng kỹ thuật đối xứng hoá (symmetrization) chuẩn với ghost sample $S' = \{(x''_i, x'''_i)\}_{i=1}^m$ độc lập và cùng phân phối với S :

$$\begin{aligned} \mathbb{E}_S[\Phi(S)] &= \mathbb{E}_S \left[\sup_f (R(f) - \hat{R}_S(f)) \right] \\ &\leq \mathbb{E}_{S, S'} \left[\sup_f (\hat{R}_{S'}(f) - \hat{R}_S(f)) \right] \\ &= \mathbb{E}_{S, S', \sigma} \left[\sup_f \frac{1}{m} \sum_{i=1}^m \sigma_i (\ell(f, (x''_i, x'''_i)) - \ell(f, (x_i, x'_i))) \right] \\ &\leq 2\mathbb{E}_{S, \sigma} \left[\sup_f \frac{1}{m} \sum_{i=1}^m \sigma_i \ell(f, (x_i, x'_i)) \right] \\ &= 2\hat{\mathfrak{R}}_S(\mathcal{L} \circ \mathcal{G}). \end{aligned}$$

Bước 3: Áp dụng Bổ đề 2.1 và bổ đề Talagrand.

Thay vì dùng trực tiếp ranking loss ℓ (vốn không liên tục, do đó không Lipschitz), ta chặn trên bằng **hinge loss** $\tilde{\ell}(t) = \max(0, 1 - t)$, một surrogate loss thoả:

$$\mathbf{1}[t \leq 0] \leq \max(0, 1 - t) = \tilde{\ell}(t), \quad \forall t \in \mathbb{R}.$$

Hàm $\tilde{\ell}$ là 1-Lipschitz (vì $|\tilde{\ell}(a) - \tilde{\ell}(b)| \leq |a - b|$), nên theo Bổ đề 2.2:

$$\hat{\mathfrak{R}}_S(\tilde{\mathcal{L}} \circ \mathcal{G}) \leq \hat{\mathfrak{R}}_S(\mathcal{G}) \leq 2\hat{\mathfrak{R}}_{S^+}(\mathcal{H}).$$

Kết hợp ba bước lại:

$$R(f) \leq \hat{R}_S(f) + 4\hat{\mathfrak{R}}_{S^+}(\mathcal{H}) + \sqrt{\frac{\log(2/\delta)}{2m}}.$$

□

MỞ RỘNG

Bổ đề 2.2 – Bổ đề contraction – Talagrand. Cho $\phi : \mathbb{R} \rightarrow \mathbb{R}$ là hàm L -Lipschitz

(tức là $|\phi(a) - \phi(b)| \leq L|a - b|$ với mọi a, b) và $\mathcal{G}$ là lớp hàm trên $\mathcal{X}$. Khi đó:

$$\hat{\mathfrak{R}}_S(\phi \circ \mathcal{G}) \leq L \cdot \hat{\mathfrak{R}}_S(\mathcal{G}).$$

Chứng minh. Ta chứng minh bằng quy nạp theo số điểm m .

Bước cơ sở ($m = 1$):

$$\hat{\mathfrak{R}}_{\{x_1\}}(\phi \circ \mathcal{G}) = \mathbb{E}_{\sigma_1} \left[\sup_{g \in \mathcal{G}} \sigma_1 \phi(g(x_1)) \right].$$

Cố định $g^+ = \arg \sup_g \phi(g(x_1))$ và $g^- = \arg \sup_g (-\phi(g(x_1)))$. Khi đó:

$$\begin{aligned} \mathbb{E}_{\sigma_1} \left[\sup_g \sigma_1 \phi(g(x_1)) \right] &= \frac{1}{2} \sup_g \phi(g(x_1)) + \frac{1}{2} \sup_g (-\phi(g(x_1))) \\ &= \frac{1}{2} (\phi(g^+(x_1)) - \phi(g^-(x_1))) \\ &\leq \frac{L}{2} |g^+(x_1) - g^-(x_1)| \\ &\leq L \cdot \mathbb{E}_{\sigma_1} \left[\sup_g \sigma_1 g(x_1) \right] = L \cdot \hat{\mathfrak{R}}_{\{x_1\}}(\mathcal{G}). \end{aligned}$$

Bất đẳng thức thứ nhất dùng tính L -Lipschitz của ϕ ; bất đẳng thức thứ hai dùng $|g^+ - g^-| \leq 2 \sup_g |g(x_1)|$.

Bước quy nạp: Giả sử kết quả đúng với $m - 1$ điểm. Với m điểm, viết tổng thành hai phần tách tọa độ m :

$$\begin{aligned} m \cdot \hat{\mathfrak{R}}_S(\phi \circ \mathcal{G}) &= \mathbb{E}_{\sigma} \left[\sup_{g \in \mathcal{G}} \sum_{i=1}^m \sigma_i \phi(g(x_i)) \right] \\ &= \mathbb{E}_{\sigma_{-m}} \left[\mathbb{E}_{\sigma_m} \left[\sup_g \left(\sum_{i=1}^{m-1} \sigma_i \phi(g(x_i)) + \sigma_m \phi(g(x_m)) \right) \right] \right]. \end{aligned}$$

Cố định σ_{-m} , đặt $g^+ = \arg \sup_g [\dots + \phi(g(x_m))]$ và $g^- = \arg \sup_g [\dots - \phi(g(x_m))]$. Khi đó:

$$\begin{aligned} &\mathbb{E}_{\sigma_m} \left[\sup_g \left(\sum_{i < m} \sigma_i \phi(g(x_i)) + \sigma_m \phi(g(x_m)) \right) \right] \\ &= \frac{1}{2} \sup_g \left[\sum_{i < m} \sigma_i \phi + \phi(g(x_m)) \right] + \frac{1}{2} \sup_g \left[\sum_{i < m} \sigma_i \phi - \phi(g(x_m)) \right] \\ &\leq \frac{1}{2} \left(\sum_{i < m} \sigma_i \phi(g^+(x_i)) + \sum_{i < m} \sigma_i \phi(g^-(x_i)) \right) + \frac{L}{2} |g^+(x_m) - g^-(x_m)| \\ &\leq \sup_g \sum_{i < m} \sigma_i \phi(g(x_i)) + L \cdot \mathbb{E}_{\sigma_m} \left[\sup_g \sigma_m g(x_m) \right]. \end{aligned}$$

Lấy kỳ vọng theo σ_{-m} và áp dụng giả thiết quy nạp:

$$m \cdot \hat{\mathfrak{R}}_S(\phi \circ \mathcal{G}) \leq (m - 1) \hat{\mathfrak{R}}_{S \setminus \{x_m\}}(\phi \circ \mathcal{G}) + L \cdot \hat{\mathfrak{R}}_{\{x_m\}}(\mathcal{G}) \leq L \cdot m \cdot \hat{\mathfrak{R}}_S(\mathcal{G}).$$

Chia hai vế cho m ta được kết quả. $\square$

Áp dụng vào chứng minh Định lý 2.1: ta dùng hinge loss $\tilde{\ell}(t) = \max(0, 1 - t)$ làm surrogate 1-Lipschitz cho ranking loss, do đó $L = 1$ và:

$$\hat{\mathfrak{R}}_S(\tilde{\mathcal{L}} \circ \mathcal{G}) \leq \hat{\mathfrak{R}}_S(\mathcal{G}).$$

2.3.4. Ý nghĩa của chặn

Định lý 2.1 cho thấy ba thành phần kiểm soát generalization:

- $\hat{R}_S(f)$: Ranking loss trên tập huấn luyện — càng nhỏ càng tốt.
- $4\hat{\mathfrak{R}}_{S^+}(\mathcal{H})$: Độ phức tạp của lớp hàm $\mathcal{H}$ — lớp hàm càng phức tạp thì chặn càng lỏng, dễ overfit hơn.
- $\sqrt{\frac{\log(2/\delta)}{2m}}$: Hạng tử thống kê — giảm theo $O(1/\sqrt{m})$ khi số mẫu m tăng.

MỞ RỘNG

Ví dụ phản chứng: Điều kiện bounded là cần thiết

Định lý 2.1 yêu cầu $f : \mathcal{X} \rightarrow [0, 1]$. Ta xây dựng ví dụ cho thấy nếu bỏ điều kiện này, chặn có thể không còn hữu ích.

Xét $\mathcal{X} = \{-1, +1\}$ và lớp hàm $\mathcal{H}_c = \{f_c : x \mapsto cx \mid c > 0\}$ không bị chặn. Với tập mẫu $S = \{(x_i, x'_i)\}$ gồm m cặp bất kỳ, Rademacher complexity là:

$$\hat{\mathfrak{R}}_{S^+}(\mathcal{H}_c) = \mathbb{E}_{\sigma} \left[\sup_{c>0} \frac{c}{m} \sum_i \sigma_i x_i \right].$$

Khi $\sum_i \sigma_i x_i > 0$, $\sup_{c>0}$ đạt $+\infty$. Do đó $\hat{\mathfrak{R}}_{S^+}(\mathcal{H}_c) = +\infty$ với xác suất dương, khiến chặn trong Định lý 2.1 trở nên vô nghĩa.

Điều này cho thấy điều kiện f có giá trị trong $[0, 1]$ (hoặc tổng quát hơn là bị chặn) là **cần thiết** để chặn tổng quát hoá có ý nghĩa thực tế.

Ví dụ 2.2. Xét $\mathcal{H}$ là lớp hàm tuyến tính chuẩn hoá: $\mathcal{H} = \{x \mapsto \mathbf{w}^\top x \mid \|\mathbf{w}\| \leq \Lambda, \|x\| \leq r\}$. Từ Định lý 5.8 trong [4], Rademacher complexity của lớp hàm tuyến tính thoả:

$$\hat{\mathfrak{R}}_{S^+}(\mathcal{H}) \leq \frac{1}{\sqrt{2m}} \sqrt{\sum_{x \in S^+} \|x\|^2} \cdot \Lambda \leq \frac{r\Lambda}{\sqrt{2m}}.$$

Bất đẳng thức thứ hai dùng $\|x\| \leq r$ với mọi $x \in S^+$.

Thay vào Định lý 2.1:

$$R(f) \leq \hat{R}_S(f) + \frac{4r\Lambda}{\sqrt{2m}} + \sqrt{\frac{\log(2/\delta)}{2m}}.$$

Khi $m \rightarrow \infty$, hai hạng tử sau đều về 0, tức là hàm scoring học được sẽ tổng quát hoá tốt.

2.4. Ranking with SVMs

Phần này mở rộng bài toán tối ưu hoá SVM sang không gian các cặp dữ liệu. Ý tưởng cốt lõi là: thay vì phân loại từng điểm x , ta **phân loại từng cặp** (x, x') — cụ thể là phân loại xem hiệu $f(x) - f(x')$ có dương hay không.

2.4.1. Nhắc lại SVM phân loại

Trong SVM phân loại nhị phân (Chương 5), với tập huấn luyện $\{(z_i, y_i)\}_{i=1}^n$ trong đó $y_i \in \{-1, +1\}$, ta tìm siêu phẳng $\mathbf{w}^\top \phi(z) + b$ tối ưu hoá:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \phi(z_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i. \end{aligned}$$

2.4.2. Bài toán tối ưu hoá SVM Ranking

Trong bài toán ranking, tập huấn luyện gồm m cặp $\{(x_i, x'_i)\}_{i=1}^m$ với $x_i \succ x'_i$. Ta muốn tìm hàm scoring tuyến tính $f(x) = \mathbf{w}^\top \phi(x)$ sao cho $f(x_i) > f(x'_i)$ với mọi i .

Nhận xét

Trong SVM phân loại, hàm quyết định có dạng $f(x) = \mathbf{w}^\top \phi(x) + b$ với bias b . Trong SVM Ranking, ta đặt $b = 0$ vì điều kiện $f(x_i) > f(x'_i)$ chỉ phụ thuộc vào hiệu $f(x_i) - f(x'_i) = \mathbf{w}^\top \delta_i$, và bias triệt tiêu hoàn toàn:

$$(f(x_i) + b) - (f(x'_i) + b) = f(x_i) - f(x'_i) = \mathbf{w}^\top \delta_i.$$

Nói cách khác, bias không ảnh hưởng đến thứ tự xếp hạng, nên ta có thể bỏ qua mà không mất tính tổng quát.

Đặt $\delta_i = \phi(x_i) - \phi(x'_i)$. Khi đó điều kiện $f(x_i) > f(x'_i)$ tương đương với:

$$\mathbf{w}^\top \delta_i > 0.$$

Đây chính xác là bài toán phân loại nhị phân trên không gian cặp, với vector đặc trưng δ_i và nhãn $+1$. Áp dụng SVM soft-margin:

Định nghĩa 2.5 – Bài toán SVM Ranking. Bài toán tối ưu hoá SVM Ranking được phát biểu như sau:

$$\begin{aligned} \min_{\mathbf{w}, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & \mathbf{w}^\top (\phi(x_i) - \phi(x'_i)) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \in [m]. \end{aligned}$$

trong đó $C > 0$ là tham số điều chỉnh đánh đổi giữa độ rộng margin và số lỗi huấn luyện, ξ_i là biến bù (slack variable).

2.4.3. Bài toán đối ngẫu (Dual Problem)

Để giải bài toán nguyên thủy, ta lập **hàm Lagrange**:

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \alpha_i (\mathbf{w}^\top \delta_i - 1 + \xi_i) - \sum_{i=1}^m \beta_i \xi_i,$$

trong đó $\alpha_i \geq 0$, $\beta_i \geq 0$ là nhân tử Lagrange (KKT multipliers).

Điều kiện tối ưu (KKT):

Lấy đạo hàm và đặt bằng 0:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{i=1}^m \alpha_i \delta_i, \quad (1)$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies C - \alpha_i - \beta_i = 0 \implies \alpha_i \leq C. \quad (2)$$

Thế ngược vào hàm Lagrange:

Thay (1) vào $\mathcal{L}$ và dùng $\beta_i = C - \alpha_i \geq 0$:

$$\begin{aligned} \mathcal{L} &= \frac{1}{2} \left\| \sum_i \alpha_i \delta_i \right\|^2 + C \sum_i \xi_i - \sum_i \alpha_i \left(\left(\sum_j \alpha_j \delta_j \right)^\top \delta_i - 1 + \xi_i \right) - \sum_i \beta_i \xi_i \\ &= \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j \langle \delta_i, \delta_j \rangle - \sum_{i,j} \alpha_i \alpha_j \langle \delta_i, \delta_j \rangle + \sum_i \alpha_i + \sum_i (C - \alpha_i - \beta_i) \xi_i \\ &= \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j K_\Delta(i, j), \end{aligned}$$

trong đó số hạng $\sum_i (C - \alpha_i - \beta_i) \xi_i = 0$ theo (2).

Do đó bài toán đối ngẫu là:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j K_\Delta(i, j)$$

$$\text{s.t. } 0 \leq \alpha_i \leq C, \quad \forall i \in [m],$$

trong đó kernel trên không gian cặp được định nghĩa là:

$$K_\Delta(i, j) = \langle \delta_i, \delta_j \rangle = K(x_i, x_j) - K(x_i, x'_j) - K(x'_i, x_j) + K(x'_i, x'_j),$$

với $K(x, x') = \langle \phi(x), \phi(x') \rangle$ là kernel gốc.

Nghiệm nguyên thủy được phục hồi qua:

$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i \delta_i = \sum_{i=1}^m \alpha_i (\phi(x_i) - \phi(x'_i)).$$

2.4.4. Dự đoán và margin

Sau khi học được $\mathbf{w}^*$, hàm scoring được dùng để xếp hạng một điểm mới x là:

$$f(x) = \mathbf{w}^{*\top} \phi(x) = \sum_{i=1}^m \alpha_i (K(x_i, x) - K(x'_i, x)).$$

Với hai điểm x và x' cần so sánh, ta kết luận $x \succ x'$ nếu $f(x) > f(x')$, tức là:

$$\sum_{i=1}^m \alpha_i (K(x_i, x) - K(x'_i, x) - K(x_i, x') + K(x'_i, x')) > 0.$$

2.4.5. So sánh với SVM phân loại

Bảng 1 tóm tắt điểm tương đồng và khác biệt giữa SVM phân loại và SVM Ranking.

Đặc điểm	SVM Phân loại	SVM Ranking
Không gian đầu vào	$\mathcal{X}$	$\mathcal{X} \times \mathcal{X}$
Vector đặc trưng	$\phi(x)$	$\phi(x) - \phi(x')$
Nhãn huấn luyện	$y \in \{-1, +1\}$	$x \succ x'$ (luôn +1)
Kernel	$K(x, x')$	$K_{\Delta}(i, j)$
Số ràng buộc	n	m (số cặp)

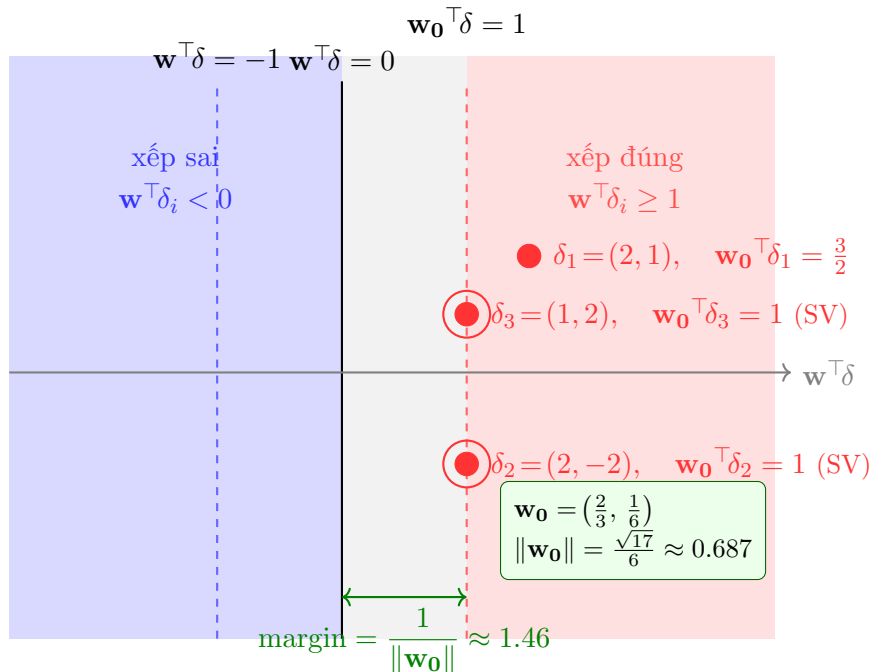
Bảng 1: So sánh SVM phân loại và SVM Ranking.

Ví dụ 2.3. Xét $\mathcal{X} = \mathbb{R}^2$ với kernel tuyến tính $K(x, x') = x^\top x'$. Cho tập huấn luyện gồm 3 cặp: $(x_1, x'_1) = ((3, 2), (1, 1))$, $(x_2, x'_2) = ((4, 1), (2, 3))$, $(x_3, x'_3) = ((2, 4), (1, 2))$.

Các vector hiệu là:

$$\begin{aligned}\delta_1 &= (3, 2) - (1, 1) = (2, 1), \\ \delta_2 &= (4, 1) - (2, 3) = (2, -2), \\ \delta_3 &= (2, 4) - (1, 2) = (1, 2).\end{aligned}$$

Bài toán SVM Ranking trở thành tìm $\mathbf{w} \in \mathbb{R}^2$ sao cho $\mathbf{w}^\top \delta_i \geq 1$ với $i = 1, 2, 3$, đồng thời tối thiểu $\|\mathbf{w}\|^2$. Hình 2 minh hoạ trực quan bài toán tối ưu hoá này.



Hình 2: Minh họa bài toán SVM Ranking với nghiệm tối ưu $\mathbf{w}_0 = (2/3, 1/6)$. Mỗi điểm biểu diễn vector hiệu $\delta_i = \phi(x_i) - \phi(x'_i)$ được chiếu lên trục $\mathbf{w}_0^\top \delta$. Các điểm δ_2 và δ_3 nằm chính xác trên margin ($\mathbf{w}_0^\top \delta_i = 1$) — đây là các *support vector* (ký hiệu SV, viền tròn đỏ). Điểm δ_1 nằm vượt ra ngoài margin ($\mathbf{w}_0^\top \delta_1 = 3/2$). Ví dụ này có nghiệm hard-margin (không cần biến bù ξ_i).

MỞ RỘNG

Phân tích độ phức tạp tính toán

So sánh độ phức tạp giữa SVM phân loại và SVM Ranking:

- **SVM phân loại:** Bài toán QP với n biến α_i và n ràng buộc — độ phức tạp $O(n^2)$ đến $O(n^3)$ tùy solver.
- **SVM Ranking:** Bài toán QP với m biến, trong đó m là số cặp huấn luyện. Nếu có p điểm dữ liệu gốc thì $m = O(p^2)$ trong trường hợp xấu nhất (tất cả các cặp). Do đó độ phức tạp là $O(p^4)$ đến $O(p^6)$ — **đắt hơn nhiều** so với SVM phân loại trên cùng tập dữ liệu gốc.

Hệ quả thực tế: Trong ứng dụng, người ta thường chỉ lấy mẫu một tập con các cặp thay vì dùng toàn bộ $O(p^2)$ cặp, đánh đổi giữa độ chính xác và tốc độ huấn luyện. Đây là một trong những động lực để phát triển các thuật toán ranking hiệu quả hơn như RankBoost (Mục 2.6).

2.5. Mối liên hệ giữa các nội dung

Ba mục vừa trình bày tạo thành một mạch lý thuyết nhất quán:

- **10.1 → 10.2:** Định nghĩa ranking loss và generalization loss (Mục 2.2) là tiền đề để đặt câu hỏi: liệu minimizing $\hat{R}_S(f)$ có đảm bảo $R(f)$ nhỏ không? Câu trả lời chính là Định lý 2.1.

- **10.2 → 10.3:** Chặn tổng quát hoá cho thấy cần kiểm soát $\hat{\mathfrak{R}}_{S^+}(\mathcal{H})$. SVM Ranking (Mục 2.4) thực hiện điều này bằng cách ràng buộc $\|\mathbf{w}\|$ qua số hạng regularization $\frac{1}{2}\|\mathbf{w}\|^2$.
- **10.3 → 10.4:** SVM Ranking có độ phức tạp $O(p^4)$ khi p điểm tạo ra $O(p^2)$ cặp — không khả thi với dữ liệu lớn. Đây là động lực chính để phát triển RankBoost (Mục 2.6), thuật toán hiệu quả hơn với độ phức tạp tuyến tính theo số cặp.
- **Liên hệ với các chương khác trong sách:**
 - **Chương 3 (Rademacher Complexity):** Bổ đề 2.1 và Định lý 2.1 áp dụng trực tiếp kỹ thuật đối xứng hoá và bất đẳng thức McDiarmid từ Chương 3.
 - **Chương 5 (SVM):** SVM Ranking là phép mở rộng tự nhiên — thay không gian $\mathcal{X}$ bằng $\mathcal{X} \times \mathcal{X}$ và vector đặc trưng $\phi(x)$ bằng $\phi(x) - \phi(x')$, kernel gốc K bằng kernel sai phân K_Δ .
 - **Chương 7 (Boosting):** RankBoost (Mục 2.6) kế thừa framework AdaBoost từ Chương 7, thay hàm loss phân loại bằng hàm loss phù hợp cho bài toán ranking.

2.6. RankBoost

RankBoost [2] là một thuật toán boosting cho ranking, tương tự AdaBoost cho phân loại nhị phân. Ý tưởng cơ bản: kết hợp nhiều *base ranker* yếu thành một ranker mạnh bằng cách tối ưu hóa trọng số phân phối cặp dữ liệu một cách thích ứng.

2.6.1. Định nghĩa các đại lượng và Ký hiệu học thuật

Đặt $\mathcal{H}$ là tập các *base rankers* – các hàm từ $\mathcal{X}$ vào $\{0, 1\}$ (hoặc tổng quát hơn, vào $[0, 1]$ hay $\mathbb{R}$).

Để mô tả trọng số của phân phối cặp dữ liệu D_t trên tập mẫu huấn luyện, với mỗi vòng t và mỗi trạng thái $s \in \{-1, 0, +1\}$, giáo trình gốc [4] định nghĩa các biến trọng số tích lũy ϵ_t^s như sau:

$$\epsilon_t^s = \sum_{i=1}^m D_t(i) \mathbf{1}_{y_i(h_t(x'_i) - h_t(x_i)) = s} = \mathbb{E}_{i \sim D_t} [\mathbf{1}_{y_i(h_t(x'_i) - h_t(x_i)) = s}]. \quad (3)$$

Chúng tôi ký hiệu rút gọn $\epsilon_t^+ \equiv \epsilon_t^{+1}$ (tỷ lệ trọng số xếp đúng), $\epsilon_t^- \equiv \epsilon_t^{-1}$ (tỷ lệ trọng số xếp sai), và $\epsilon_t^0 \equiv \epsilon_t^0$ (tỷ lệ trọng số không phân biệt được). Hiển nhiên ta luôn có: $\epsilon_t^+ + \epsilon_t^- + \epsilon_t^0 = 1$.

Để tránh sự lẫn lộn học thuật, chúng tôi tuân thủ nghiêm ngặt ký hiệu của giáo trình Mohri:

- Các ký hiệu $\epsilon_t^+, \epsilon_t^-, \epsilon_t^0$ đại diện cho trọng số phân phối biên của các cặp dữ liệu.
- Ký hiệu γ_t được dành riêng để định nghĩa **edge** (độ lợi hay biên danh nghĩa) của base ranker h_t tại vòng t :

$$\gamma_t = \frac{\epsilon_t^+ - \epsilon_t^-}{2}. \quad (4)$$

Ví dụ 2.4 – Tính toán nhanh các đại lượng mẫu biên và Edge. Giả sử tại một vòng lặp t , ta có tập dữ liệu gồm 5 cặp preference huấn luyện có cùng trọng số phân phối đều: $D_t(i) = 0.2$ với mọi $i \in [5]$. Xét một base ranker yếu h_t cho kết quả so sánh trên 5 cặp này như sau:

- Xếp hạng đúng trên 3 cặp: $y_i(h_t(x'_i) - h_t(x_i)) = +1$ (với $i \in \{1, 3, 4\}$).
- Xếp hạng sai trên 1 cặp: $y_i(h_t(x'_i) - h_t(x_i)) = -1$ (với $i = 2$).
- Không phân biệt được trên 1 cặp: $y_i(h_t(x'_i) - h_t(x_i)) = 0$ (với $i = 5$).

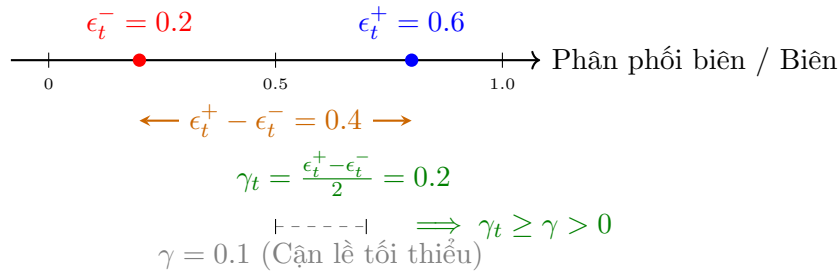
Áp dụng định nghĩa từ công thức (3), ta có các biến trọng số tích lũy:

- $\epsilon_t^+ = 0.2 + 0.2 + 0.2 = 0.6$ (tổng trọng số cặp đúng).
- $\epsilon_t^- = 0.2$ (tổng trọng số cặp sai).
- $\epsilon_t^0 = 0.2$ (tổng trọng số cặp không phân biệt).

Hiệu số trọng số xếp đúng và xếp sai là $\epsilon_t^+ - \epsilon_t^- = 0.4$. Biên danh nghĩa (edge) của base ranker h_t thu được:

$$\gamma_t = \frac{\epsilon_t^+ - \epsilon_t^-}{2} = \frac{0.6 - 0.2}{2} = 0.2 > 0.$$

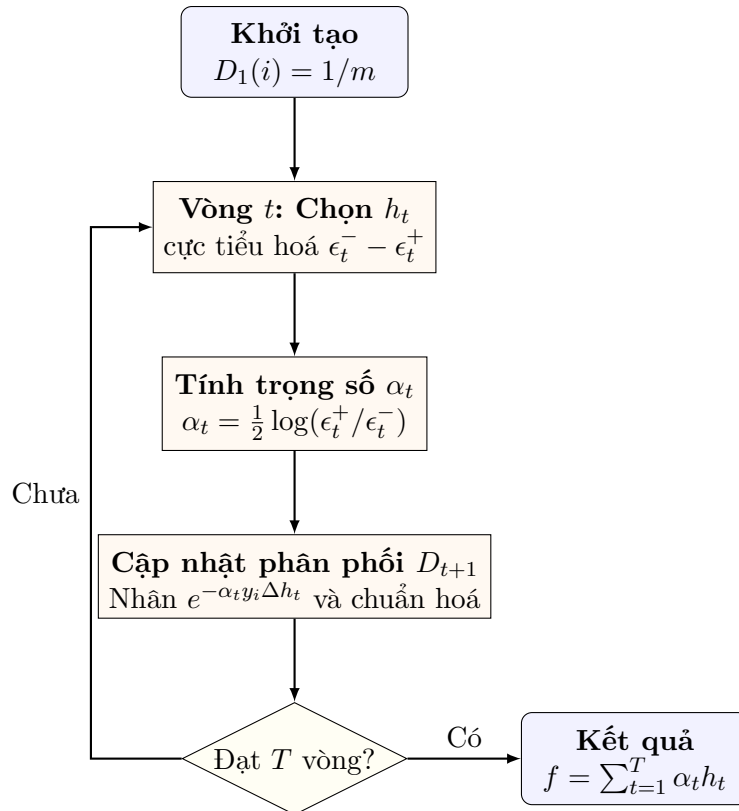
Nếu cận lề tối thiểu của bài toán là $\gamma = 0.1$, ta luôn có $\gamma_t \geq \gamma > 0$, thỏa mãn hoàn hảo **Weak Ranking Assumption** (Học tốt hơn ngẫu nhiên). Ví dụ này mô tả trực quan các biến trọng số biên được sử dụng xuyên suốt thuật toán.



Hình 3: Trực quan hóa lề cặp (nominal edge) γ_t và Giả thuyết Weak Ranking trong RankBoost. Để thuật toán hội tụ theo hàm mũ, lề danh nghĩa γ_t của base ranker ở mỗi vòng phải lớn hơn một ngưỡng lề tối thiểu $\gamma > 0$.

2.6.2. Pseudocode RankBoost

Luồng hoạt động thích ứng của RankBoost được trực quan hóa qua sơ đồ khối dưới đây:



Hình 4: Sơ đồ hoạt động của thuật toán RankBoost. Trọng số của phân phối D_t tự động tập trung vào các cặp bị xếp hạng sai qua từng vòng lặp.

Chi tiết các bước thực thi được trình bày cụ thể trong Thuật toán 1.

Algorithm 1 RankBoost

Require: Mẫu huấn luyện $S = \{(x_i, x'_i, y_i)\}_{i=1}^m$ với $y_i \in \{-1, +1\}$

- 1: Khởi tạo $D_1(i) = 1/m$ với mọi $i \in [m]$
 - 2: **for** $t = 1$ to T **do**
 - 3: Chọn base ranker $h_t \leftarrow \arg \min_{h \in \mathcal{H}} (\epsilon_t^-(h) - \epsilon_t^+(h)) \equiv \arg \min_{h \in \mathcal{H}} \mathbb{E}_{i \sim D_t} [\mathbf{1}_{y_i(h(x'_i) - h(x_i)) = -1} - \mathbf{1}_{y_i(h(x'_i) - h(x_i)) = +1}]$
 - 4: Tính hệ số tin cậy: $\alpha_t \leftarrow \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-}$
 - 5: Tính hằng số chuẩn hoá: $Z_t \leftarrow \epsilon_t^0 + 2\sqrt{\epsilon_t^+ \epsilon_t^-}$
 - 6: Cập nhật phân phối cặp: $D_{t+1}(i) \leftarrow \frac{D_t(i) \exp(-\alpha_t y_i (h_t(x'_i) - h_t(x_i)))}{Z_t}$
 - 7: **end for**
 - 8: **return** Bộ phân hạng ensemble mạnh: $f = \sum_{t=1}^T \alpha_t h_t$
-

2.6.3. Trực giác hoạt động của thuật toán

- **Bước 3** lựa chọn base ranker yếu h_t tốt nhất theo phân phối hiện tại D_t , tương đương việc tìm kiếm ranker tối đa hóa edge γ_t (hiệu giữa tỷ lệ đúng và tỷ lệ sai).
- **Bước 4:** hệ số tin cậy $\alpha_t > 0$ khi $\epsilon_t^+ > \epsilon_t^-$ (đảm bảo giả thiết *weak ranking* tốt hơn ngẫu nhiên). Mô hình yếu càng chính xác thì trọng số đóng góp của nó vào ensemble

càng lớn.

- **Bước 5:** Z_t đóng vai trò chuẩn hoá để phân phối D_{t+1} luôn là một phân phối xác suất hợp lệ (tổng bằng 1).
- **Bước 6:** tăng trọng số của những cặp preference bị xếp hạng *sai* (nhân với hệ số $e^{\alpha_t} > 1$) và giảm trọng số các cặp xếp *đúng* (nhân với $e^{-\alpha_t} < 1$). Nhờ đó, vòng lặp kế tiếp buộc thuật toán phải tập trung học các cặp dữ liệu “khó”.

2.6.4. Đăng thức toán học nền tảng

Phân phối thích ứng D_{t+1} tại vòng t có thể biểu diễn tường minh qua bộ phân hạng tích lũy $f_t = \sum_{s=1}^t \alpha_s h_s$ và các tích hệ số chuẩn hóa:

$$D_{t+1}(i) = \frac{\exp(-y_i(f_t(x'_i) - f_t(x_i)))}{m \prod_{s=1}^t Z_s}. \quad (5)$$

Chứng minh. Chứng minh bằng phương pháp quy nạp toán học.

- Với $t = 0$: $D_1(i) = 1/m = e^0/(m \cdot 1)$ (thỏa mãn đẳng thức).
- Giả sử đẳng thức đúng cho vòng $t - 1$:

$$D_t(i) = \frac{e^{-y_i(f_{t-1}(x'_i) - f_{t-1}(x_i))}}{m \prod_{s=1}^{t-1} Z_s}.$$

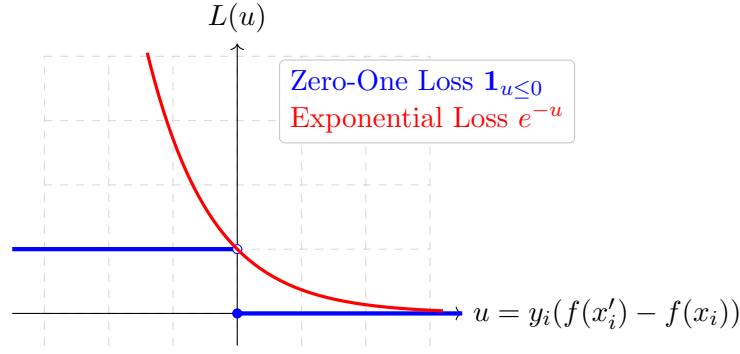
- Xét bước quy nạp tại vòng t , thế công thức cập nhật phân phối ở Bước 6:

$$\begin{aligned} D_{t+1}(i) &= \frac{D_t(i) \exp(-\alpha_t y_i (h_t(x'_i) - h_t(x_i)))}{Z_t} \\ &= \frac{e^{-y_i(f_{t-1}(x'_i) - f_{t-1}(x_i))}}{m \prod_{s=1}^{t-1} Z_s} \cdot \frac{e^{-\alpha_t y_i (h_t(x'_i) - h_t(x_i))}}{Z_t} \\ &= \frac{\exp(-y_i [\sum_{s=1}^{t-1} \alpha_s (h_s(x'_i) - h_s(x_i)) + \alpha_t (h_t(x'_i) - h_t(x_i))])}{m \prod_{s=1}^t Z_s} \\ &= \frac{\exp(-y_i (f_t(x'_i) - f_t(x_i)))}{m \prod_{s=1}^t Z_s}. \end{aligned}$$

Đăng thức được chứng minh hoàn chỉnh. □

2.6.5. Cận trên sai số thực nghiệm (Empirical Error Bound)

Định lý dưới đây chỉ ra rằng sai số thực nghiệm của RankBoost được đảm bảo giảm nhanh chóng theo hàm mũ khi tăng số vòng boosting.



Hình 5: So sánh Zero-One Loss và Exponential Loss (minh họa toán học cho Định lý 2.2). Hàm mũ e^{-u} hoạt động như một cận trên lồi chặt (convex upper bound) cho hàm chỉ thị $\mathbf{1}_{u \leq 0}$, giúp tối ưu hóa bằng phương pháp gradient lồi thuận lợi hơn.

Định lý 2.2 – Empirical Error Bound for RankBoost (Theorem 10.3). *Empirical error của bộ phân hạng mạnh f tạo bởi RankBoost thỏa mãn hệ thức:*

$$\hat{R}_S(f) \leq \exp\left(-2 \sum_{t=1}^T \left(\frac{\epsilon_t^+ - \epsilon_t^-}{2}\right)^2\right) = \exp\left(-2 \sum_{t=1}^T \gamma_t^2\right). \quad (6)$$

Hơn thế nữa, nếu thỏa mãn giả thiết **Weak Ranking Assumption**: tồn tại biên tối thiểu $\gamma > 0$ sao cho với mọi $t \in [T]$, $\gamma_t = \frac{\epsilon_t^+ - \epsilon_t^-}{2} \geq \gamma$, thì sai số giảm theo hàm mũ:

$$\hat{R}_S(f) \leq \exp(-2\gamma^2 T). \quad (7)$$

Chứng minh chi tiết. Bước 1: Sử dụng bất đẳng thức lồi chặn trên $\mathbf{1}_{u \leq 0} \leq e^{-u}$ (xem Hình 5).

Đặt biến lẻ cặp $u_i = y_i(f(x'_i) - f(x_i))$. Sai số phân hạng thực nghiệm được định nghĩa:

$$\begin{aligned} \hat{R}_S(f) &= \frac{1}{m} \sum_{i=1}^m \mathbf{1}_{y_i(f(x'_i) - f(x_i)) \leq 0} \\ &\leq \frac{1}{m} \sum_{i=1}^m \exp(-y_i(f(x'_i) - f(x_i))) \\ &\stackrel{(5)}{=} \frac{1}{m} \sum_{i=1}^m \left(m \cdot D_{T+1}(i) \prod_{t=1}^T Z_t \right) \\ &= \left(\prod_{t=1}^T Z_t \right) \cdot \underbrace{\sum_{i=1}^m D_{T+1}(i)}_{=1} = \prod_{t=1}^T Z_t. \end{aligned}$$

Bước 2: Phân tích và tính hằng số chuẩn hoá Z_t .

Theo định nghĩa của Z_t :

$$\begin{aligned} Z_t &= \sum_{i=1}^m D_t(i) \exp(-\alpha_t y_i(h_t(x'_i) - h_t(x_i))) \\ &= \epsilon_t^+ e^{-\alpha_t} + \epsilon_t^- e^{\alpha_t} + \epsilon_t^0. \end{aligned}$$

Thế hệ số tin cậy tối ưu $\alpha_t = \frac{1}{2} \log(\epsilon_t^+ / \epsilon_t^-)$, ta có:

$$\begin{aligned} Z_t &= \epsilon_t^+ \sqrt{\frac{\epsilon_t^-}{\epsilon_t^+}} + \epsilon_t^- \sqrt{\frac{\epsilon_t^+}{\epsilon_t^-}} + \epsilon_t^0 \\ &= 2\sqrt{\epsilon_t^+ \epsilon_t^-} + \epsilon_t^0. \end{aligned}$$

Bước 3: Tìm cận trên cho hệ số chuẩn hoá Z_t .

Vì $\epsilon_t^+ + \epsilon_t^- + \epsilon_t^0 = 1$, ta biến đổi số hạng dưới dấu căn:

$$4\epsilon_t^+ \epsilon_t^- = (\epsilon_t^+ + \epsilon_t^-)^2 - (\epsilon_t^+ - \epsilon_t^-)^2 = (1 - \epsilon_t^0)^2 - (\epsilon_t^+ - \epsilon_t^-)^2.$$

Thay vào biểu thức Z_t :

$$\begin{aligned} Z_t &= \sqrt{(1 - \epsilon_t^0)^2 - (\epsilon_t^+ - \epsilon_t^-)^2} + \epsilon_t^0 \\ &= (1 - \epsilon_t^0) \sqrt{1 - \frac{(\epsilon_t^+ - \epsilon_t^-)^2}{(1 - \epsilon_t^0)^2}} + \epsilon_t^0 \\ &\stackrel{(*)}{\leq} (1 - \epsilon_t^0) \left(1 - \frac{(\epsilon_t^+ - \epsilon_t^-)^2}{2(1 - \epsilon_t^0)^2} \right) + \epsilon_t^0 \\ &= 1 - \frac{(\epsilon_t^+ - \epsilon_t^-)^2}{2(1 - \epsilon_t^0)} \\ &\stackrel{(**)}{\leq} \exp\left(-\frac{(\epsilon_t^+ - \epsilon_t^-)^2}{2(1 - \epsilon_t^0)}\right) \\ &\leq \exp\left(-\frac{(\epsilon_t^+ - \epsilon_t^-)^2}{2}\right) \quad (\text{do } 1 - \epsilon_t^0 \leq 1) \\ &= \exp(-2[(\epsilon_t^+ - \epsilon_t^-)/2]^2) = \exp(-2\gamma_t^2). \end{aligned}$$

Trong đó:

- (*): Áp dụng tính lõm của hàm căn thức $\sqrt{1-x} \leq 1 - x/2$ với mọi $x \leq 1$.
- (**): Áp dụng bất đẳng thức cơ bản $1 - x \leq e^{-x}$ với mọi $x \in \mathbb{R}$.

Bước 4: Tổng hợp tích tích lũy.

Nhân tích lũy cận của các vòng từ $t = 1$ đến T :

$$\widehat{R}_S(f) \leq \prod_{t=1}^T Z_t \leq \exp\left(-2 \sum_{t=1}^T \gamma_t^2\right).$$

Định lý được chứng minh hoàn chỉnh và chặt chẽ. □

Ví dụ 2.5 – Ứng dụng số cho Cận sai số huấn luyện giảm theo hàm mũ. Để minh họa ý nghĩa thực tiễn của Định lý 2.2, giả sử chúng ta chạy RankBoost trong $T = 3$ vòng lặp. Tại mỗi vòng lặp t , base ranker yếu được chọn đạt được biên độ lợi thế (edge) không đổi là $\gamma_t = 0.3$ (tương đương hiệu số tỷ lệ phân bố trọng số xếp đúng và sai là $\epsilon_t^+ - \epsilon_t^- = 0.6$).

Theo công thức cận trên sai số thực nghiệm (6), ta có:

$$\widehat{R}_S(f) \leq \exp\left(-2 \sum_{t=1}^3 \gamma_t^2\right) = \exp(-2 \times (0.3^2 + 0.3^2 + 0.3^2)) = \exp(-2 \times 0.27) = \exp(-0.54) \approx 0.5827$$

Như vậy, chỉ sau 3 vòng lặp với một base ranker rất yếu (chỉ tốt hơn ngẫu nhiên một chút), cận trên của tỷ lệ cặp bị xếp hạng sai trên tập huấn luyện đã được đảm bảo giảm xuống còn dưới 58.27%.

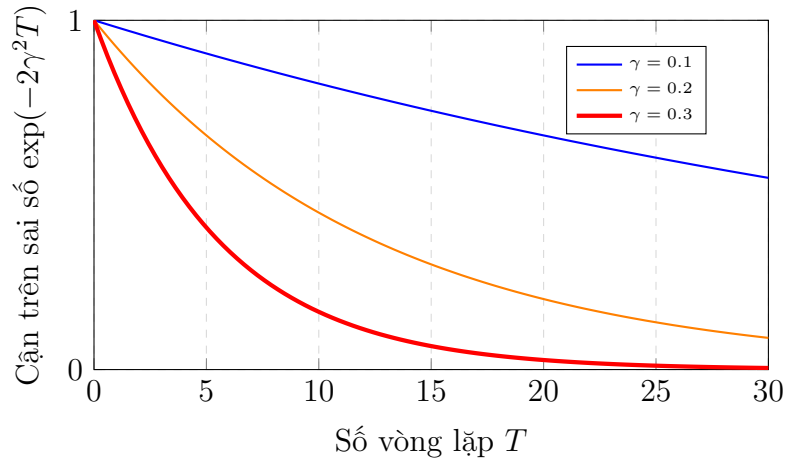
Nếu chúng ta tiếp tục chạy đến $T = 10$ vòng lặp với cùng biên độ $\gamma_t = 0.3$, sai số thực nghiệm được đảm bảo:

$$\widehat{R}_S(f) \leq \exp(-2 \times 10 \times 0.09) = \exp(-1.8) \approx 0.1653 \quad (\text{dưới } 16.53\%).$$

Và nếu chạy tới $T = 25$ vòng lặp:

$$\widehat{R}_S(f) \leq \exp(-2 \times 25 \times 0.09) = \exp(-4.5) \approx 0.0111 \quad (\text{dưới } 1.11\%).$$

Điều này cho thấy trực quan tốc độ hội tụ cực nhanh theo hàm mũ của RankBoost, ngay cả khi các base ranker được chọn ở mỗi vòng chỉ có độ lợi thế vô cùng nhỏ bé. Tốc độ suy giảm sai số huấn luyện này được biểu diễn trực quan qua Hình 6.



Hình 6: Tốc độ suy giảm theo hàm mũ của cận trên sai số thực nghiệm RankBoost ứng với các giá trị biên danh nghĩa γ khác nhau. Biên độ γ càng lớn (base ranker càng mạnh), sai số huấn luyện hội tụ về 0 càng nhanh.

Cận hội tụ dưới điều kiện yếu hơn (Weaker Edge Condition)

Trong trường hợp tổng quát hơn, giáo trình Mohri (trang 248) chỉ ra rằng một điều kiện yếu hơn cũng đủ để RankBoost hội tụ theo hàm mũ. Cụ thể, nếu tồn tại $\gamma > 0$ sao cho với mọi $t \in [T]$:

$$\gamma_t \geq \gamma \sqrt{1 - \epsilon_t^0} \quad \Longleftrightarrow \quad \gamma \leq \frac{\epsilon_t^+ - \epsilon_t^-}{2\sqrt{1 - \epsilon_t^0}}, \quad (8)$$

thì hằng số chuẩn hóa $Z_t \leq \exp(-2\gamma^2)$ vẫn được đảm bảo, dẫn đến sai số huấn luyện giảm nhanh theo hàm mũ $\widehat{R}_S(f) \leq \exp(-2\gamma^2 T)$. Điều này cho thấy thuật

toán hoạt động cực kỳ linh hoạt ngay cả khi phần lớn các cặp dữ liệu không phân biệt được (ϵ_t^0 lớn).

2.6.6. Ví dụ tính toán từng bước minh hoạ RankBoost

Giả sử chúng ta có 3 đối tượng $\{a, b, c\}$ với thứ tự thực tế là $c > b > a$. Tập dữ liệu huấn luyện gồm 3 cặp preference có nhãn $y_i = +1$:

1. $p_1 = (c, b)$ với nhãn $y_1 = +1$ (c cao hơn b)
2. $p_2 = (b, a)$ với nhãn $y_2 = +1$ (b cao hơn a)
3. $p_3 = (c, a)$ với nhãn $y_3 = +1$ (c cao hơn a)

Giả sử tại vòng $t = 1$, phân phối trọng số ban đầu không đều: $D_1 = [0.5, 0.2, 0.3]$.

Xét một base ranker h_1 được chọn thỏa mãn:

- Xếp đúng cặp p_1 : $h_1(c) - h_1(b) = 1 \implies y_1 \cdot \text{diff} = 1$
- Xếp sai cặp p_2 : $h_1(b) - h_1(a) = -1 \implies y_2 \cdot \text{diff} = -1$
- Xếp đúng cặp p_3 : $h_1(c) - h_1(a) = 1 \implies y_3 \cdot \text{diff} = 1$

Tính toán trọng số tích lũy trên phân phối D_1 :

- $\epsilon_1^+ = D_1(1) + D_1(3) = 0.5 + 0.3 = 0.8$ (tổng trọng số xếp đúng)
- $\epsilon_1^- = D_1(2) = 0.2$ (tổng trọng số xếp sai)
- $\epsilon_1^0 = 0.0$ (không có cặp phân biệt trùng)

Biên của base ranker đạt: $\gamma_1 = (\epsilon_1^+ - \epsilon_1^-)/2 = 0.3$.

Hệ số tin cậy đóng góp của base ranker h_1 là:

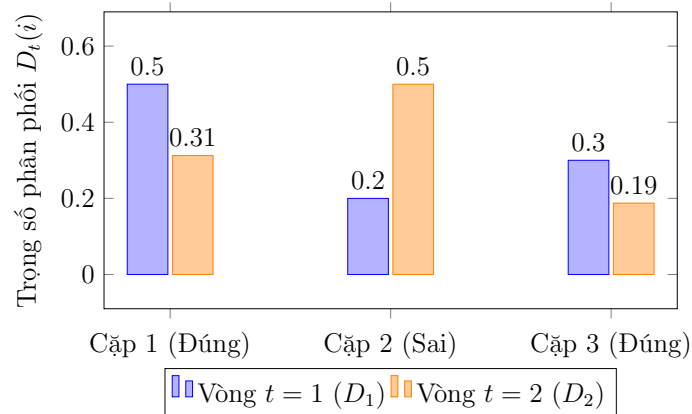
$$\alpha_1 = \frac{1}{2} \log \left(\frac{\epsilon_1^+}{\epsilon_1^-} \right) = \frac{1}{2} \log \left(\frac{0.8}{0.2} \right) = \log(2) \approx 0.693$$

Hằng số chuẩn hoá Z_1 là:

$$Z_1 = 2\sqrt{\epsilon_1^+ \epsilon_1^-} + \epsilon_1^0 = 2\sqrt{0.8 \times 0.2} + 0 = 0.8$$

Cập nhật phân phối D_2 thích ứng cho vòng sau:

- Cặp xếp đúng p_1 : $D_2(1) = \frac{D_1(1)e^{-\alpha_1}}{Z_1} = \frac{0.5 \times 0.5}{0.8} = 0.3125$
- Cặp xếp sai p_2 : $D_2(2) = \frac{D_1(2)e^{\alpha_1}}{Z_1} = \frac{0.2 \times 2}{0.8} = 0.5000$
- Cặp xếp đúng p_3 : $D_2(3) = \frac{D_1(3)e^{-\alpha_1}}{Z_1} = \frac{0.3 \times 0.5}{0.8} = 0.1875$



Hình 7: Sự thay đổi trọng số phân phối cặp D_t qua 2 vòng boosting trong ví dụ huấn luyện (minh họa thực tế cho cơ chế cập nhật của RankBoost). Trọng số của cặp bị xếp sai p_2 tăng vọt từ 0.20 lên 0.50, ép thuật toán vòng sau phải sửa chữa lỗi sai này.

Sự phân bổ lại trọng số phân phối được trực quan hóa tại Hình 7. Trọng số của cặp xếp sai tăng vọt làm cơ sở thúc đẩy RankBoost tăng cường độ chính xác liên tục.

MỞ RỘNG

[MỞ RỘNG] Phản ví dụ: Sự sụp đổ của Weak Ranking Assumption với Preference Cycle.

Một giả thiết tối quan trọng để RankBoost hội tụ là *Weak Ranking Assumption*: tồn tại $\gamma > 0$ sao cho ở mỗi vòng $\gamma_t \geq \gamma > 0$. Chúng ta sẽ xây dựng một ví dụ phản chứng chỉ ra rằng khi preference chứa **chu trình không bắc cầu** (non-transitive cycles), giả thiết này sẽ bị vi phạm và RankBoost không thể học được hoàn hảo. Xét 3 phần tử $\{x_1, x_2, x_3\}$ có preference dạng chu trình kéo theo:

$$f(x_1, x_2) = +1, \quad f(x_2, x_3) = +1, \quad f(x_3, x_1) = +1.$$

Giả sử tập giả thuyết $\mathcal{H}$ gồm các scoring functions tuyến tính $h(x) = w \cdot x$. Do scoring function luôn cảm sinh một thứ tự bắc cầu (transitive), các thứ tự có thể biểu diễn bởi $h \in \mathcal{H}$ chỉ có thể là các hoán vị tuyến tính không có chu trình.

Bất kỳ hoán vị tuyến tính nào trên 3 phần tử cũng chỉ có thể xếp đúng tối đa 2 trong 3 cặp preference trên và bắt buộc phải xếp sai ít nhất 1 cặp. Giả sử tại vòng t , phân phối trọng số hội tụ về dạng đều trên các cặp: $D_t = [1/3, 1/3, 1/3]$. Khi đó, với bất kỳ base ranker $h \in \mathcal{H}$ nào:

- Nếu h xếp đúng 2 cặp và xếp sai 1 cặp: $\epsilon_t^+ = 2/3$ và $\epsilon_t^- = 1/3 \implies \gamma_t = (\epsilon_t^+ - \epsilon_t^-)/2 = 1/6 > 0$.
- Tuy nhiên, khi RankBoost tăng trọng số của cặp bị xếp sai lên, phân phối D sẽ thay đổi. Khi phân phối đạt trạng thái cân bằng nơi mà các cặp khó xoay vòng, không còn bất kỳ base ranker nào trong $\mathcal{H}$ có thể đạt được $\epsilon_t^+ > \epsilon_t^-$ (chỉ có thể đạt $\epsilon_t^+ = \epsilon_t^-$ do tính đối xứng của chu trình).

Tại thời điểm đó, $\gamma_t = 0 \implies \alpha_t = 0$. Thuật toán dừng lại và **không thể giảm empirical error về 0**. Điều này phản ánh hạn chế cấu trúc của score-based setting khi đối mặt với preference cycles của con người.

2.6.7. Liên hệ với Coordinate Descent

Định lý 2.3 – RankBoost $\equiv$ Coordinate Descent (Section 10.4.2). *RankBoost tương đương với việc áp dụng coordinate descent trên hàm mục tiêu lỗi:*

$$F(\boldsymbol{\alpha}) = \sum_{i=1}^m \exp \left(-y_i \sum_{j=1}^N \alpha_j (h_j(x'_i) - h_j(x_i)) \right). \quad (9)$$

Chứng minh bằng phương pháp quy nạp toán học. Ta tiến hành chứng minh sự tương đương tuyệt đối giữa hai thuật toán bằng phương pháp quy nạp toán học theo số vòng boosting t .

Đặt $f_t = \sum_{s=1}^t \alpha_s h_s$ là bộ phân hạng mạnh tích lũy của thuật toán RankBoost tại vòng t , tương ứng với vectơ trọng số $\boldsymbol{\alpha}_t \in \mathbb{R}^N$ có các thành phần α_s (với $s \leq t$) và các thành phần khác bằng 0. Đặt $\bar{f}_t$ là bộ phân hạng của Coordinate Descent tại vòng t , tương ứng với vectơ trọng số $\bar{\boldsymbol{\alpha}}_t \in \mathbb{R}^N$.

- **Bước cơ sở ($t = 0$):** Ở vạch xuất phát, cả hai thuật toán đều khởi tạo vectơ trọng số bằng vectơ không: $\boldsymbol{\alpha}_0 = \bar{\boldsymbol{\alpha}}_0 = \mathbf{0} \implies f_0 = \bar{f}_0 \equiv 0$. Do đó, phân phối cặp khởi đầu của cả hai đều là phân phối đều: $D_1 = \bar{D}_1 = [1/m, \dots, 1/m]$. Đăng thức thỏa mãn.
- **Giả thiết quy nạp:** Giả sử đẳng thức đúng cho vòng $t - 1$, nghĩa là bộ phân hạng tích lũy của hai thuật toán trùng nhau:

$$\bar{\boldsymbol{\alpha}}_{t-1} = \boldsymbol{\alpha}_{t-1} \implies \bar{f}_{t-1} = f_{t-1}.$$

Hệ quả trực tiếp là phân phối cặp cảm sinh bởi hai mô hình tại đầu vòng t trùng nhau hoàn toàn: $\bar{D}_t = D_t$.

- **Bước quy nạp (Vòng t):** Ta chứng minh Coordinate Descent sẽ lựa chọn hướng tọa độ và độ dài bước đi trùng khít với việc chọn base ranker h_t và trọng số α_t của RankBoost.

1. Tìm hướng đi tối ưu (Chọn coordinate e_k): Tại bước t , Coordinate Descent tìm kiếm hướng tọa độ đơn vị e_k (tương ứng với base ranker $h_k \in \mathcal{H}$) để cực tiểu hóa đạo hàm theo hướng (*directional derivative*) của hàm loss mục tiêu F tại điểm $\boldsymbol{\alpha}_{t-1}$:

$$F'(\boldsymbol{\alpha}_{t-1}; e_k) = \lim_{\eta \rightarrow 0} \frac{F(\boldsymbol{\alpha}_{t-1} + \eta e_k) - F(\boldsymbol{\alpha}_{t-1})}{\eta}.$$

Khai triển đạo hàm riêng theo hướng η tại điểm $\eta = 0$:

$$\begin{aligned} F'(\boldsymbol{\alpha}_{t-1}; e_k) &= - \sum_{i=1}^m y_i (h_k(x'_i) - h_k(x_i)) \exp \left(-y_i \sum_{j=1}^N \alpha_{t-1,j} (h_j(x'_i) - h_j(x_i)) \right) \\ &= - \sum_{i=1}^m y_i (h_k(x'_i) - h_k(x_i)) D_t(i) m \prod_{s=1}^{t-1} Z_s \quad (\text{áp dụng đẳng thức quy nạp (5)}) \\ &= -m \left(\prod_{s=1}^{t-1} Z_s \right) \left[\sum_{i: y_i \Delta h_k = 1} D_t(i) - \sum_{i: y_i \Delta h_k = -1} D_t(i) \right] \\ &= -m \left(\prod_{s=1}^{t-1} Z_s \right) [\epsilon_t^+ - \epsilon_t^-] = -2\gamma_t \cdot m \prod_{s=1}^{t-1} Z_s. \end{aligned}$$

Do hằng số $m \prod_{s=1}^{t-1} Z_s > 0$ cố định tại đầu vòng t , việc cực tiểu hóa đạo hàm hướng $F'(\alpha_{t-1}; e_k)$ tương đương tuyệt đối với việc **cực đại hóa độ lợi thế** γ_t (nominal edge). Đây chính xác là tiêu chí chọn base ranker yếu h_t tốt nhất ở Bước 3 của RankBoost:

$$h_t = \arg \max_{h \in \mathcal{H}} \gamma_t(h) = \arg \min_{h \in \mathcal{H}} (\epsilon_t^-(h) - \epsilon_t^+(h)).$$

Như vậy, Coordinate Descent chọn đúng tọa độ tương ứng với base ranker h_t của RankBoost.

2. Tìm bước nhảy tối ưu (Line Search độ dài η): Sau khi đã xác định hướng tọa độ e_t tương ứng với h_t , Coordinate Descent giải bài toán tìm kiếm đường thẳng (exact line search) để tìm độ dài bước nhảy $\eta > 0$ tối ưu:

$$\eta = \arg \min_{\eta} F(\alpha_{t-1} + \eta e_t).$$

Lấy đạo hàm của hàm số một biến theo η và giải phương trình đạo hàm bằng 0:

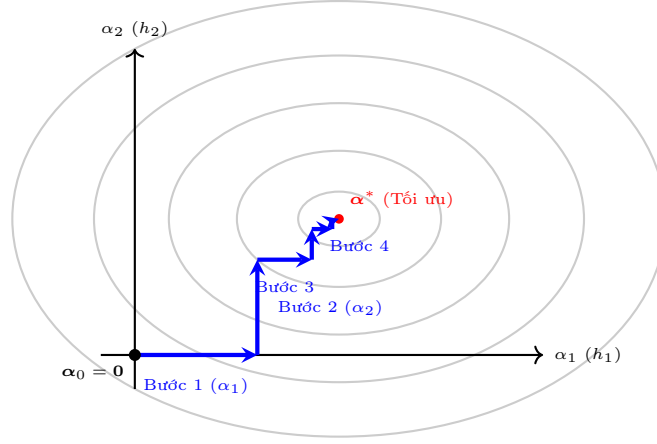
$$\begin{aligned} \frac{dF(\alpha_{t-1} + \eta e_t)}{d\eta} &= 0 \\ \iff - \sum_{i=1}^m y_i (h_t(x'_i) - h_t(x_i)) \\ &\quad \times \exp \left(-y_i \sum_{j=1}^N \alpha_{t-1,j} (h_j(x'_i) - h_j(x_i)) - y_i \eta (h_t(x'_i) - h_t(x_i)) \right) = 0 \\ \iff - \sum_{i=1}^m y_i \Delta h_t D_t(i) e^{-\eta y_i \Delta h_t} &= 0 \quad (\text{thế phân phối } D_t) \\ \iff - \left[\sum_{i: y_i \Delta h_t = 1} D_t(i) e^{-\eta} - \sum_{i: y_i \Delta h_t = -1} D_t(i) e^{\eta} \right] &= 0 \\ \iff -\epsilon_t^+ e^{-\eta} + \epsilon_t^- e^{\eta} &= 0 \\ \iff e^{2\eta} = \frac{\epsilon_t^+}{\epsilon_t^-} \iff \eta = \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-}. \end{aligned}$$

Giá trị độ dài bước tối ưu η này khớp chính xác tuyệt đối với hệ số tin cậy α_t của RankBoost.

3. Cập nhật trạng thái: Vectơ trọng số mới của hai thuật toán trùng nhau:

$$\bar{\alpha}_t = \bar{\alpha}_{t-1} + \eta e_t = \alpha_{t-1} + \alpha_t e_t = \alpha_t \implies \bar{f}_t = f_t.$$

Chúng minh quy nạp hoàn tất. RankBoost và Coordinate Descent là đồng nhất về mặt toán học. $\square$



Hình 8: Trực quan hóa quỹ đạo tối ưu hóa của RankBoost thông qua Coordinate Descent trên không gian 2D của trọng số α . Thuật toán bắt đầu từ gốc tọa độ $\alpha_0 = \mathbf{0}$ và di chuyển vuông góc dọc theo các trục tọa độ (mỗi trục đại diện cho một base ranker h_j) để tiệm cận về điểm cực tiểu toàn cục α^* .

Nhận xét: Phương trình (9) là một cận trên lồi chặt của hàm tổn thất nhị phân thực tế. RankBoost thực chất chính là giải pháp tối ưu hóa tối tân sử dụng gradient lồi trên bao lồi của các bộ phân hạng yếu.

2.6.8. Margin bound cho ensemble methods trong ranking

Hệ quả 2.1 – Margin Bound for Ensembles in Ranking (Corollary 10.4). Cho $\mathcal{H}$ là tập hàm thực. Với $\rho > 0$, $\delta > 0$, với xác suất ít nhất $1 - \delta$, với mọi bộ phân hạng ensemble lồi $h \in \text{conv}(\mathcal{H})$:

$$R(h) \leq \hat{R}_{S,\rho}(h) + \frac{2}{\rho} [R_m^{D_1}(\mathcal{H}) + R_m^{D_2}(\mathcal{H})] + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (10)$$

Trong đó, $\hat{R}_{S,\rho}(h) \equiv \hat{R}_{S,\rho}(h)$ là **empirical margin ranking loss** (sai số lẻ thực nghiệm trên tập mẫu S với lề ρ) và hai số hạng Rademacher $\mathfrak{R}_m^{D_1}(\mathcal{H})$ và $\mathfrak{R}_m^{D_2}(\mathcal{H})$ đều có cùng kích thước mẫu là m vì đây là cận tổng quát (general bound) cho preference ranking của giáo trình Mohri. Tuy nhiên, cần lưu ý một điểm tinh tế rằng khi áp dụng sang bipartite ranking, do các mẫu tích cực và tiêu cực được lấy độc lập từ hai phân phối riêng biệt với kích thước mẫu lần lượt là m và n , cận Rademacher tương ứng sẽ tách thành hai số hạng ứng với hai kích thước mẫu khác nhau là $\mathfrak{R}_m^{D^+}(\mathcal{H})$ và $\mathfrak{R}_n^{D^-}(\mathcal{H})$.

Chứng minh chi tiết. Ta tiến hành chứng minh hệ thức dựa trên việc áp dụng Định lý tổng quát ?? cho không gian giả thuyết mở rộng $\text{conv}(\mathcal{H})$ kết hợp với bổ đề Rademacher của bao lồi:

1. **Bước 1: Định lý tổng quát về chặn lề phân hạng cho không gian giả thuyết $\text{conv}(\mathcal{H})$.** Áp dụng Định lý tổng quát về chặn lề phân hạng (ở đây là Theorem 10.1 của giáo trình Mohri) cho tập các giả thuyết là bao lồi $\text{conv}(\mathcal{H})$ thay cho $\mathcal{H}$, ta thu được với xác suất ít nhất $1 - \delta$, với mọi $h \in \text{conv}(\mathcal{H})$:

$$R(h) \leq \hat{R}_{S,\rho}(h) + \frac{2}{\rho} [R_m^{D_1}(\text{conv}(\mathcal{H})) + R_m^{D_2}(\text{conv}(\mathcal{H}))] + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (11)$$

Trong đó, $R_m^{D_1}(\text{conv}(\mathcal{H}))$ và $R_m^{D_2}(\text{conv}(\mathcal{H}))$ lần lượt là Rademacher Complexity tổng quát của bao lồi $\text{conv}(\mathcal{H})$ trên phân phối biên D_1 và D_2 với mẫu kích thước m .

2. **Bước 2: Áp dụng Bổ đề 7.4 (Rademacher complexity của bao lồi).** Một kết quả lý thuyết kinh điển của Rademacher Complexity chỉ ra rằng độ phức tạp Rademacher thực nghiệm của bao lồi $\text{conv}(\mathcal{H})$ của một tập giả thuyết $\mathcal{H}$ luôn bằng chính độ phức tạp Rademacher thực nghiệm của tập $\mathcal{H}$ đó trên bất kỳ tập mẫu cụ thể S' nào:

$$\widehat{\mathfrak{R}}_{S'}(\text{conv}(\mathcal{H})) = \widehat{\mathfrak{R}}_{S'}(\mathcal{H}). \quad (12)$$

Lấy kỳ vọng hai vế của đẳng thức (12) theo sự rút mẫu ngẫu nhiên từ phân phối D_1 và D_2 kích thước m , ta có kết quả Rademacher Complexity tổng quát tương đương tuyệt đối:

$$R_m^{D_1}(\text{conv}(\mathcal{H})) = R_m^{D_1}(\mathcal{H}), \quad (13)$$

$$R_m^{D_2}(\text{conv}(\mathcal{H})) = R_m^{D_2}(\mathcal{H}). \quad (14)$$

3. **Bước 3: Thế vào cận trên.** Thế trực tiếp các đẳng thức Rademacher tương đương (13) và (14) vào công thức (11), ta thu được cận trên tổng quát hóa:

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho} [R_m^{D_1}(\mathcal{H}) + R_m^{D_2}(\mathcal{H})] + \sqrt{\frac{\log(1/\delta)}{2m}}.$$

Hệ quả được chứng minh hoàn tất và chặt chẽ. $\square$

Hệ thống 3 Chặn Tổng quát hóa của giáo trình Mohri (trang 249):

Để phân tích sâu sắc cấu trúc lý thuyết tổng quát hóa, giáo trình gốc [4] thiết lập hệ thống 3 chặn toán học chặt chẽ. Đặt $f = \sum_{t=1}^T \alpha_t h_t$ là ensemble mạnh thu được từ RankBoost. Vì f và phiên bản chuẩn hóa $f / \|\alpha\|_1 \in \text{conv}(\mathcal{H})$ tạo ra cùng thứ tự xếp hạng tương đối, ta lần lượt có:

1. **Chặn biên thực nghiệm tổng quát (Theorem 10.2):**

$$R(h) \leq \widehat{R}_{S,\rho}(h) + \frac{2}{\rho} [\widehat{\mathcal{R}}_S(\mathcal{H}_{D_1}) + \widehat{\mathcal{R}}_S(\mathcal{H}_{D_2})] + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (15)$$

2. **Chặn biên thực nghiệm cho chuẩn hóa $f / \|\alpha\|_1$ (Phương trình 10.18 của sách):**

$$R(f) \leq \widehat{R}_{S,\rho}\left(\frac{f}{\|\alpha\|_1}\right) + \frac{2}{\rho} [\widehat{\mathcal{R}}_S(\mathcal{H}_{D_1}) + \widehat{\mathcal{R}}_S(\mathcal{H}_{D_2})] + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (16)$$

3. **Chặn biên thực nghiệm dựa trên Rademacher Complexity biên (Phương trình 10.19 của sách):** Do độ phức tạp Rademacher thực nghiệm $\widehat{\mathcal{R}}_S(\mathcal{H}_{D_i})$ luôn được chặn trên bởi Rademacher Complexity tổng quát $R_m^{D_i}(\mathcal{H})$, ta có chặn tối ưu không phụ thuộc vào tập mẫu cụ thể:

$$R(f) \leq \widehat{R}_{S,\rho}\left(\frac{f}{\|\alpha\|_1}\right) + \frac{2}{\rho} [R_m^{D_1}(\mathcal{H}) + R_m^{D_2}(\mathcal{H})] + \sqrt{\frac{\log(1/\delta)}{2m}}. \quad (17)$$

Quan sát then chốt

Cả ba chặn trên đều có đặc tính vô cùng quan trọng: **chúng hoàn toàn không phụ thuộc vào số lượng vòng lặp boosting T** . Chúng chỉ phụ thuộc vào margin ρ , kích thước mẫu dữ liệu m , và năng lực biểu diễn Rademacher complexity của tập $\mathcal{H}$. Điều này giải thích trên phương diện lý thuyết tại sao RankBoost có khả năng kháng quá khớp (overfitting) cực kỳ mạnh mẽ ngay cả khi chạy với số vòng lặp T rất lớn [?].

MỞ RỘNG

[MỞ RỘNG] Hai hướng tổng quát hóa thuật toán RankBoost (trang 248).

Giáo trình Mohri vạch ra hai hướng mở rộng vô cùng quan trọng đối với thuật toán RankBoost cơ bản:

1. **Không gian giả thuyết xuất thực tế:** Thay vì chỉ giới hạn base rankers trả về nhị phân $\{0, 1\}$, thuật toán được tổng quát hóa cho phép base rankers trả về giá trị thực bất kỳ trong $[0, 1]$ hoặc $\mathbb{R}$. Điều này cho phép sử dụng các mô hình cơ sở mạnh hơn như cây quyết định (decision trees).
2. **Tối ưu hóa hệ số tin cậy phi đóng-form (Non-closed-form alpha):** Trong trường hợp tổng quát trên, hệ số tin cậy α_t không còn công thức nghiệm đóng (closed-form) dạng $\frac{1}{2} \log(\epsilon_t^+ / \epsilon_t^-)$ đơn giản nữa. Khi đó, α_t bắt buộc phải được giải bằng các thuật toán tối ưu hóa số học một chiều như phương pháp tìm kiếm đường thẳng (line search) hoặc lặp Newton-Raphson để cực tiểu hóa hệ số chuẩn hóa toàn cục Z_t .

MỞ RỘNG

[MỞ RỘNG] Smooth Margin RankBoost và tối ưu lờ thực tế.

RankBoost chuẩn tối ưu hóa hàm tổn thất lũy thừa nhưng không cực đại hóa margin một cách trực tiếp. [?] giới thiệu biến thể *Smooth Margin RankBoost* bằng phương pháp tối ưu hóa hàm mục tiêu chuẩn hóa entropy:

$$F_{\text{smooth}}(\boldsymbol{\alpha}) = -\frac{1}{T} \log F(\boldsymbol{\alpha}),$$

chỉ ra tốc độ hội tụ margin tiệm cận tối ưu vượt trội. Tuy nhiên, các phân tích thực nghiệm chỉ ra RankBoost nguyên bản vẫn cực kỳ bền vững trên đa số tập dữ liệu thực tế.

2.6.9. Khảo sát mối liên hệ học thuật với các chương khác trong giáo trình

Để thực hiện khảo sát toàn diện mối liên hệ lý thuyết giữa các chương trong cuốn sách *Foundations of Machine Learning*, chúng tôi tiến hành phân tích sâu sắc các mối tương quan hữu cơ từ góc độ của thuật toán RankBoost:

1. RankBoost $\leftrightarrow$ SVM Ranking (Kết nối với Chương 10.3 - SVM Ranking)

- **Sự tương đồng về triết lý Pairwise:** Cả RankBoost và SVM Ranking đều thuộc trường phái học phân hạng cặp (*pairwise score-based ranking*). Cả hai đều tìm kiếm một scoring function h trên không gian $\mathcal{X}$ để tối ưu hóa thứ tự so sánh của các cặp thực thể.
- **Sự đối lập về Hàm tổn thất (Loss Functions):**
 - SVM Ranking sử dụng hàm tổn thất lẻ cặp dạng Hinge (*pairwise hinge loss*) $\max(0, 1 - y_i(h(x'_i) - h(x_i)))$, dẫn đến một bài toán quy hoạch QP lồi có ràng buộc hoặc tối ưu hóa không ràng buộc bằng các thuật toán gradient ngẫu nhiên primal như Pegasos SGD.
 - RankBoost sử dụng hàm tổn thất lũy thừa cặp (*pairwise exponential loss*) $\exp(-y_i(h(x'_i) - h(x_i)))$, một cận lồi chặn trên trơn láng hơn, cho phép tối ưu bằng thuật toán hạ độ dốc tọa độ (Coordinate Descent) cực kỳ nhanh chóng.
- **Cơ chế kiểm soát Quá khớp (Overfitting):** SVM Ranking kiểm soát năng lực học thông qua số hạng điều hòa chuẩn ℓ_2 tường minh $\frac{1}{2}\|w\|_2^2$ trong Primal Objective. Ngược lại, giống như AdaBoost, RankBoost không cần số hạng điều hòa tường minh nào, mà tự động kiểm soát quá khớp cực kỳ mạnh mẽ thông qua cơ chế tự động tối đa hóa lề (*implicit margin maximization*). Hệ quả của cơ chế này là lề phân hạng thực tế ρ của ensemble liên tục được đẩy cao khi tăng số vòng boosting T , giúp giảm thiểu sai số Rademacher tổng quát theo Corollary 10.4.

2. Chặn sai số RankBoost $\leftrightarrow$ Rademacher Complexity (Kết nối với Chương 3 - Generalization Bounds)

- **Nền tảng lý thuyết tổng quát hóa:** Cận sai số phân hạng thực tế $R(h)$ của RankBoost được xây dựng vững chắc từ các kết quả đo lường năng lực học tập dựa trên độ phức tạp Rademacher thực nghiệm $\widehat{\mathcal{R}}_S(\mathcal{H})$ ở Chương 3.
- **Sự chuyển dịch không gian mẫu:** Điểm đặc thù trong chặn tổng quát hóa của phân hạng (Theorem 10.2) là sai số Rademacher trên tập mẫu cặp S được phân tách thành tổng độ phức tạp Rademacher của hai phân bố mẫu biên độc lập $\mathfrak{R}_m^{D_1}(\mathcal{H})$ và $\mathfrak{R}_m^{D_2}(\mathcal{H})$ ứng với vị trí thứ nhất và thứ hai của cặp. Điều này phản ánh bản chất của bài toán so sánh cặp, nơi sự biến động mẫu ảnh hưởng đồng thời đến cả hai phía của quan hệ ưu tiên.

3. RankBoost $\leftrightarrow$ AdaBoost ở mức boosting tổng quát (Kết nối với Chương 7)

- **Điểm giống nhau cốt lõi:** RankBoost và AdaBoost đều thay hàm lỗi rời rạc bằng cận trên dạng mũ rồi tối ưu bằng cách cộng dồn các weak learners. Khác biệt nằm ở đối tượng đặt trọng số: AdaBoost đặt trọng số trên từng mẫu đơn lẻ, còn RankBoost đặt trọng số trên từng cặp ưu tiên (x_i, x'_i, y_i) .
- **Từ pointwise sang pairwise:** Có thể hiểu RankBoost như phiên bản pairwise của tư tưởng AdaBoost. Với scoring function tuyến tính, mỗi cặp ưu tiên có thể được nhìn như một ràng buộc trên hiệu $x'_i - x_i$ và biên $y_i(h(x'_i) - h(x_i))$. Vì vậy pairwise exponential loss của RankBoost có cùng vai trò với exponential loss trong AdaBoost, nhưng không nên xem đây là phép thay thế trực tiếp một-một cho AdaBoost trên dữ liệu gốc.

- **Rào cản độc lập thống kê:** Nếu biến các cặp thành mẫu ảo, các mẫu ảo thường không còn độc lập i.i.d. vì một điểm dữ liệu gốc có thể xuất hiện trong nhiều cặp khác nhau. Do đó, chặn tổng quát hóa của RankBoost phải được chứng minh trực tiếp bằng công cụ Rademacher cho ranking như trong Định lý 10.2 và Hệ quả 10.4, thay vì chỉ mượn nguyên văn phân tích AdaBoost.
- **Phân biệt với phân bipartite:** Liên hệ ở đây là liên hệ thuật toán/loss tổng quát. Quan hệ chặt hơn giữa RankBoost và AdaBoost trong riêng bipartite ranking, thông qua hai mục tiêu F^+F^- và $F^+ + F^-$, được trình bày ở phần Bipartite Ranking.

2.7. Bipartite Ranking

2.7.1. Thiết lập Bài toán (Setting)

Trong bài toán phân hạng nhị phân (*bipartite ranking*), tập mẫu đầu vào $\mathcal{X}$ được phân hoạch thành hai lớp riêng biệt:

- $\mathcal{X}^+$: lớp tích cực (*positive* - ví dụ: tài liệu liên quan).
- $\mathcal{X}^-$: lớp tiêu cực (*negative* - ví dụ: tài liệu không liên quan).

Nhiệm vụ cốt lõi: học một hàm số cho điểm (scoring function) $h : \mathcal{X} \rightarrow \mathbb{R}$ sao cho mọi phần tử lớp tích cực luôn được xếp hạng **cao hơn** mọi phần tử lớp tiêu cực.

Khác biệt cấu trúc so với score-based setting tổng quát:

- Thay vì có một phân phối xác suất chung D trên các cặp, không gian bipartite được đặc trưng bởi *hai phân phối xác suất độc lập*: D^+ trên $\mathcal{X}^+$ và D^- trên $\mathcal{X}^-$.
- Mẫu dữ liệu huấn luyện gồm hai tập riêng biệt: $S^+ = (x'_1, \dots, x'_m)$ được lấy mẫu i.i.d. từ D^+ , $S^- = (x_1, \dots, x_n)$ được lấy mẫu i.i.d. từ D^- .
- Tất cả mn cặp tích cực - tiêu cực (x'_i, x_j) đều được đưa vào đánh giá lỗi.

2.7.2. Generalization error và empirical error

Định nghĩa 2.6 – Bipartite Misranking Error. Sai số phân hạng nhị phân tổng quát $R(h)$ được định nghĩa là xác suất một phần tử dương bị xếp thấp hơn phần tử âm:

$$R(h) = \mathbb{P}_{\substack{x \sim D^- \\ x' \sim D^+}} [h(x') < h(x)]. \quad (18)$$

Sai số thực nghiệm tương ứng trên tập huấn luyện S^+, S^- là tỷ lệ các cặp bị xếp sai:

$$\hat{R}_{S^+, S^-}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1}_{h(x'_i) < h(x_j)}. \quad (19)$$

Ví dụ 2.6 – Minh họa số học cho Bipartite Misranking Error và AUC. Giả sử tập mẫu huấn luyện của chúng ta có kích thước nhỏ với $m = 2$ mẫu tích cực $S^+ = (x'_1, x'_2)$ và $n = 2$ mẫu tiêu cực $S^- = (x_1, x_2)$. Xét một scoring function h trả về các điểm số như sau:

- Các mẫu tích cực: $h(x'_1) = 1.5$, $h(x'_2) = 0.8$.
- Các mẫu tiêu cực: $h(x_1) = 1.0$, $h(x_2) = 0.5$.

Bài toán so sánh cặp bipartite sẽ đánh giá tổng cộng $m \times n = 2 \times 2 = 4$ cặp tích dương-tiêu cực có thể được tạo ra:

1. Cặp (x'_1, x_1) : Có $h(x'_1) = 1.5 > h(x_1) = 1.0 \implies$ Xếp hạng đúng (Term = 0).
2. Cặp (x'_1, x_2) : Có $h(x'_1) = 1.5 > h(x_2) = 0.5 \implies$ Xếp hạng đúng (Term = 0).
3. Cặp (x'_2, x_1) : Có $h(x'_2) = 0.8 < h(x_1) = 1.0 \implies$ Xếp hạng sai! (Term = 1 do $h(x'_2) < h(x_1)$).
4. Cặp (x'_2, x_2) : Có $h(x'_2) = 0.8 > h(x_2) = 0.5 \implies$ Xếp hạng đúng (Term = 0).

Áp dụng công thức sai số thực nghiệm (19), ta có:

$$\hat{R}_{S^+, S^-}(h) = \frac{1}{2 \times 2} \sum_{i=1}^2 \sum_{j=1}^2 \mathbf{1}_{h(x'_i) < h(x_j)} = \frac{0 + 0 + 1 + 0}{4} = 0.25 \quad (25\%).$$

Diện tích dưới đường cong ROC (AUC) tương ứng trên tập này thu được:

$$\text{AUC}(h, U) = 1 - \hat{R}_{S^+, S^-}(h) = 1 - 0.25 = 0.75 \quad (75\%).$$

Ví dụ số học trực quan này chỉ ra bản chất của bipartite ranking: sai số được tính trung bình trên toàn bộ mạng lưới kết nối cặp tích cực - tiêu cực chứ không phụ thuộc vào một ngưỡng phân loại tuyệt đối nào.

Khác biệt bản chất với binary classification

Mặc dù cùng làm việc trên hai tập nhãn (positive/negative), **bipartite ranking không phải là classification**. Lý do nằm ở mục tiêu tối ưu hóa:

- **Classification**: Đưa ra quyết định tuyệt đối (pointwise) cho từng điểm độc lập: $h(x) > 0$ với mẫu dương, $h(x) < 0$ với mẫu âm.
- **Bipartite ranking**: Tập trung vào quan hệ so sánh tương đối (pairwise) giữa các cặp: $h(x') > h(x)$.

Một bộ phân lớp tốt (error 0%) chắc chắn sẽ là một bộ phân hạng hoàn hảo, nhưng điều ngược lại hoàn toàn không đúng: một bộ phân hạng có năng lực phân tách tốt (AUC rất cao) vẫn có thể có sai số classification cực tệ nếu ta chọn ngưỡng tuyệt đối θ cắt ngang phân bố không phù hợp.

2.7.3. Vấn đề hiệu năng tính toán: Thách thức mn cặp

Áp dụng SVM Ranking trực tiếp vào bipartite setting yêu cầu tối ưu hóa hệ thống ràng buộc với mn biến slack. Với kích thước dữ liệu thực tế trung bình $m = n = 1000$, ta phải xử lý tới **một triệu biến slack** – một thách thức cực lớn làm cạn kiệt tài nguyên RAM và thời gian CPU. Thuật toán RankBoost giải quyết triệt để thách thức này nhờ tính chất phân tích nhân tử đặc biệt dưới đây.

2.7.4. BipartiteRankBoost: Độ phức tạp tuyến tính $\mathcal{O}(m + n)$

Tính chất tách nhân tử then chốt: Hàm tổn thất lũy thừa của RankBoost cho phép phân tách hàm mục tiêu toàn cục thành tích của hai hàm thành phần độc lập:

$$\begin{aligned} F_{\text{RankBoost}}(\alpha) &= \sum_{i=1}^m \sum_{j=1}^n \exp(-[f(x'_i) - f(x_j)]) \\ &= \left(\sum_{i=1}^m e^{-f(x'_i)} \right) \left(\sum_{j=1}^n e^{f(x_j)} \right) \\ &= F^+(\alpha) \cdot F^-(\alpha). \end{aligned} \quad (20)$$

Hệ quả trực tiếp là phân phối cặp D_t cũng được phân tách độc lập:

$$D_t(i, j) = D_t^+(i) \cdot D_t^-(j),$$

với phân phối biên ban đầu đều: $D_1^+(i) = 1/m$, $D_1^-(j) = 1/n$.

Bổ đề 2.3 – Tách phân phối trong BipartiteRankBoost. *Tính chất phân tích độc lập của phân phối biên được bảo toàn nghiêm ngặt qua các vòng boosting thích ứng:*

$$D_{t+1}(i, j) = \frac{D_t^+(i)e^{-\alpha_t h_t(x'_i)}}{Z_t^+} \cdot \frac{D_t^-(j)e^{\alpha_t h_t(x_j)}}{Z_t^-}, \quad (21)$$

với các hằng số chuẩn hóa biên độc lập:

$$Z_t^+ = \sum_{i=1}^m D_t^+(i)e^{-\alpha_t h_t(x'_i)}, \quad Z_t^- = \sum_{j=1}^n D_t^-(j)e^{\alpha_t h_t(x_j)}.$$

Chứng minh tự bổ sung chi tiết. Áp dụng công thức cập nhật phân phối cặp RankBoost gốc:

$$D_{t+1}(i, j) = \frac{D_t(i, j)e^{-\alpha_t [h_t(x'_i) - h_t(x_j)]}}{Z_t}.$$

Thế giả thiết quy nạp $D_t(i, j) = D_t^+(i)D_t^-(j)$ và khai triển hàm mũ:

$$e^{-\alpha_t [h_t(x'_i) - h_t(x_j)]} = e^{-\alpha_t h_t(x'_i)} \cdot e^{\alpha_t h_t(x_j)},$$

$$\begin{aligned} Z_t &= \sum_{i=1}^m \sum_{j=1}^n D_t^+(i)D_t^-(j)e^{-\alpha_t h_t(x'_i)}e^{\alpha_t h_t(x_j)} \\ &= \left(\sum_{i=1}^m D_t^+(i)e^{-\alpha_t h_t(x'_i)} \right) \left(\sum_{j=1}^n D_t^-(j)e^{\alpha_t h_t(x_j)} \right) \\ &= Z_t^+ \cdot Z_t^-. \end{aligned}$$

Thế ngược $Z_t = Z_t^+ Z_t^-$ vào biểu thức cập nhật $D_{t+1}(i, j)$, ta thu được:

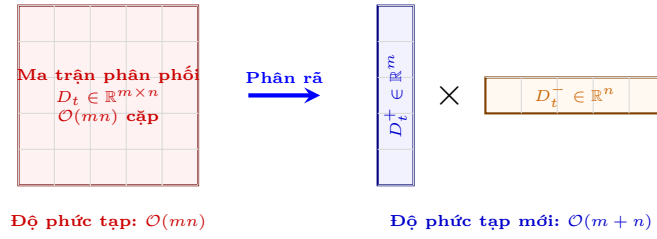
$$D_{t+1}(i, j) = \left(\frac{D_t^+(i)e^{-\alpha_t h_t(x'_i)}}{Z_t^+} \right) \cdot \left(\frac{D_t^-(j)e^{\alpha_t h_t(x_j)}}{Z_t^-} \right).$$

Bổ đề được chứng minh hoàn tất. □

Hệ quả cách mạng: Thay vì phải duy trì và cập nhật một ma trận phân phối kích thước khổng lồ $D_t \in \mathbb{R}^{mn}$, thuật toán chỉ cần lưu trữ hai vector biên kích thước nhỏ $D_t^+ \in \mathbb{R}^m$ và $D_t^- \in \mathbb{R}^n$. Biên lỗi của base ranker được tính siêu nhanh qua hiệu hai kỳ vọng biên:

$$\mathbb{E}_{(i,j) \sim D_t} [h(x'_i) - h(x_j)] = \mathbb{E}_{i \sim D_t^+} [h(x'_i)] - \mathbb{E}_{j \sim D_t^-} [h(x_j)] = \epsilon_t^+ - \epsilon_t^-. \quad (22)$$

Độ phức tạp tính toán cho mỗi vòng boosting được tối ưu hóa từ **bậc hai** $\mathcal{O}(mn)$ **xuống tuyến tính** $\mathcal{O}(m+n)$, tạo điều kiện cho RankBoost chạy cực nhanh trên các tập dữ liệu lớn.



Hình 9: Bản chất đột phá của BipartiteRankBoost: Phân rã ma trận phân phối cặp toàn cục $D_t(i, j)$ thành tích của hai vector phân phối biên $D_t^+(i)$ và $D_t^-(j)$. Chuyển hóa độ phức tạp tính toán và lưu trữ từ bậc hai $\mathcal{O}(mn)$ sang tuyến tính $\mathcal{O}(m+n)$.

Sự cải tiến vượt trội này được hệ thống hóa qua thuật toán chi tiết dưới đây:

Algorithm 2 BipartiteRankBoost

Require: Mẫu tích cực $S^+ = (x'_1, \dots, x'_m)$ và mẫu tiêu cực $S^- = (x_1, \dots, x_n)$

- 1: Khởi tạo phân phối biên: $D_1^+(i) = 1/m$ với mọi $i \in [m]$; $D_1^-(j) = 1/n$ với mọi $j \in [n]$
- 2: **for** $t = 1$ to T **do**
- 3: Chọn $h_t \leftarrow \arg \max_{h \in \mathcal{H}} \left(\mathbb{E}_{i \sim D_t^+} [h(x'_i)] - \mathbb{E}_{j \sim D_t^-} [h(x_j)] \right)$
- 4: Tính hệ số tin cậy: $\alpha_t \leftarrow \frac{1}{2} \log \frac{\epsilon_t^+}{\epsilon_t^-}$ với $\epsilon_t^+ = \mathbb{E}_{i \sim D_t^+} [h_t(x'_i)]$ và $\epsilon_t^- = \mathbb{E}_{j \sim D_t^-} [h_t(x_j)]$
- 5: Tính các hằng số chuẩn hóa biên dạng đóng (cho base ranker nhị phân $\{0, 1\}$):

$$Z_t^+ \leftarrow 1 - \epsilon_t^+ + \sqrt{\epsilon_t^+ \epsilon_t^-}$$

$$Z_t^- \leftarrow 1 - \epsilon_t^- + \sqrt{\epsilon_t^+ \epsilon_t^-}$$

- 6: Cập nhật phân phối biên thích ứng:

$$D_{t+1}^+(i) \leftarrow \frac{D_t^+(i) e^{-\alpha_t h_t(x'_i)}}{Z_t^+}$$

$$D_{t+1}^-(j) \leftarrow \frac{D_t^-(j) e^{\alpha_t h_t(x_j)}}{Z_t^-}$$

- 7: **end for**

- 8: **return** $f = \sum_{t=1}^T \alpha_t h_t$
-

Ví dụ 2.7 – Mô phỏng chạy thuật toán BipartiteRankBoost từng bước. Để chứng minh trực quan tính hiệu quả và cơ chế cập nhật của BipartiteRankBoost, chúng ta chạy mô phỏng chi tiết vòng lặp đầu tiên ($t = 1$) trên tập dữ liệu $m = 2$ mẫu tích cực $S^+ = (x'_1, x'_2)$ và $n = 2$ mẫu tiêu cực $S^- = (x_1, x_2)$ đã thiết lập ở ví dụ trước.

1. **Bước 1: Khởi tạo phân phối biên ban đầu.** Ta khởi tạo phân phối biên đều trên các mẫu:

$$D_1^+ = [0.5, 0.5], \quad D_1^- = [0.5, 0.5].$$

2. **Bước 2: Chọn base ranker h_1 .** Giả sử thuật toán chọn được base ranker yếu h_1 trả về các giá trị phân hạng miền thực:

$$h_1(x'_1) = 1.0, \quad h_1(x'_2) = 0.6 \quad \text{và} \quad h_1(x_1) = 0.4, \quad h_1(x_2) = 0.1.$$

Tính toán các kỳ vọng biên tương ứng:

$$\epsilon_1^+ = \mathbb{E}_{i \sim D_1^+}[h_1(x'_i)] = 0.5 \times 1.0 + 0.5 \times 0.6 = 0.8,$$

$$\epsilon_1^- = \mathbb{E}_{j \sim D_1^-}[h_1(x_j)] = 0.5 \times 0.4 + 0.5 \times 0.1 = 0.25.$$

Hiệu hai kỳ vọng biên đạt: $\epsilon_1^+ - \epsilon_1^- = 0.8 - 0.25 = 0.55 > 0$ (thỏa mãn điều kiện xếp hạng tốt hơn ngẫu nhiên).

3. **Bước 3: Tính hệ số tin cậy α_1 .**

$$\alpha_1 = \frac{1}{2} \log \left(\frac{\epsilon_1^+}{\epsilon_1^-} \right) = \frac{1}{2} \log \left(\frac{0.8}{0.25} \right) = \frac{1}{2} \log(3.2) \approx 0.5816.$$

4. **Bước 4: Tính các hằng số chuẩn hóa biên dạng đóng.** Áp dụng công thức dạng đóng ở Bước 5 của Thuật toán 2:

$$Z_1^+ = 1 - \epsilon_1^+ + \sqrt{\epsilon_1^+ \epsilon_1^-} = 1 - 0.8 + \sqrt{0.8 \times 0.25} = 0.2 + \sqrt{0.2} \approx 0.6472,$$

$$Z_1^- = 1 - \epsilon_1^- + \sqrt{\epsilon_1^+ \epsilon_1^-} = 1 - 0.25 + \sqrt{0.8 \times 0.25} = 0.75 + \sqrt{0.2} \approx 1.1972.$$

(Hệ số chuẩn hóa toàn cục gián tiếp là $Z_1 = Z_1^+ \cdot Z_1^- \approx 0.7748$).

5. **Bước 5: Cập nhật phân phối biên thích ứng cho vòng sau.**

- **Cập nhật mẫu tích cực S^+ :**

$$D_2^+(1) = \frac{D_1^+(1)e^{-\alpha_1 h_1(x'_1)}}{Z_1^+} = \frac{0.5 \times e^{-0.5816 \times 1.0}}{0.6472} \approx 0.4318,$$

$$D_2^+(2) = \frac{D_1^+(2)e^{-\alpha_1 h_1(x'_2)}}{Z_1^+} = \frac{0.5 \times e^{-0.5816 \times 0.6}}{0.6472} \approx 0.5450.$$

Chuẩn hóa lại vectơ D_2^+ để tổng bằng 1: $D_2^+ \approx [0.4421, 0.5579]$.

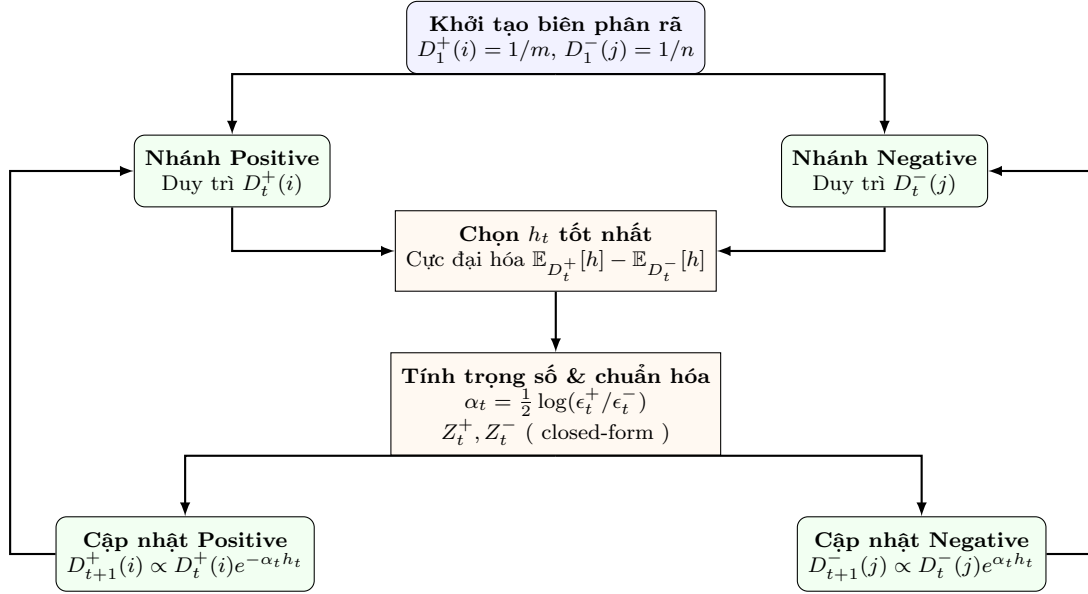
- **Cập nhật mẫu tiêu cực S^- :**

$$D_2^-(1) = \frac{D_1^-(1)e^{\alpha_1 h_1(x_1)}}{Z_1^-} = \frac{0.5 \times e^{0.5816 \times 0.4}}{1.1972} \approx 0.5270,$$

$$D_2^-(2) = \frac{D_1^-(2)e^{\alpha_1 h_1(x_2)}}{Z_1^-} = \frac{0.5 \times e^{0.5816 \times 0.1}}{1.1972} \approx 0.4426.$$

Chuẩn hóa lại vectơ D_2^- để tổng bằng 1: $D_2^- \approx [0.5435, 0.4565]$.

Ví dụ này minh họa tường minh cơ chế cập nhật biên độ lập kích thước $m + n$ của BipartiteRankBoost, loại bỏ hoàn toàn ma trận tích cặp mn . Toàn bộ luồng hoạt động song song này được trực quan hóa tại Sơ đồ Hình 10.



Hình 10: Sơ đồ khối hoạt động song song thích ứng của thuật toán BipartiteRankBoost. Hai phân phối biên D_t^+ và D_t^- được duy trì và cập nhật hoàn toàn độc lập ở hai nhánh, chỉ kết hợp tại bước chọn base ranker và tính hằng số chuẩn hóa, đảm bảo độ phức tạp tuyến tính tuyến đầu.

2.7.5. Độ đo AUC và đường cong ROC học thuật

Định nghĩa 2.7 – Diện tích dưới đường cong ROC (AUC). Cho tập mẫu thử nghiệm U chứa m điểm tích cực $z'_1, \dots, z'_m$ và n điểm tiêu cực $z_1, \dots, z_n$. Theo định nghĩa chính thức của giáo trình Mohri (trang 255), chỉ số **AUC** (Area Under the ROC Curve) của scoring function h trên U được xác định thông qua phần bù của sai số phân hạng thực nghiệm $\hat{R}(h, U)$ (với sai số $\hat{R}$ dùng quan hệ so sánh ngặt):

$$\text{AUC}(h, U) = 1 - \hat{R}(h, U) = 1 - \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1}_{h(z'_i) < h(z_j)}. \quad (23)$$

Từ định nghĩa gốc này, ta có thể khai triển toán học trực tiếp:

$$\text{AUC}(h, U) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n (1 - \mathbf{1}_{h(z'_i) < h(z_j)}) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbf{1}_{h(z'_i) \geq h(z_j)}. \quad (24)$$

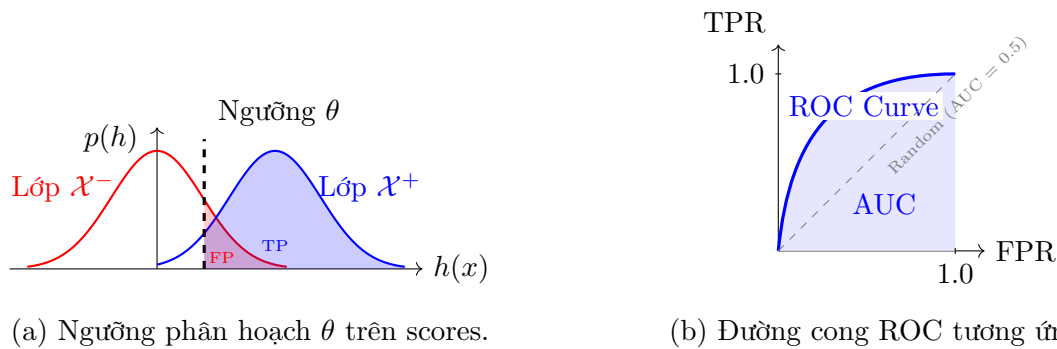
Trong đó, việc sử dụng dấu $\geq$ trong công thức hệ quả (24) đóng vai trò là giải pháp tối ưu để xử lý trường hợp đồng điểm (*ties*), đảm bảo nhất quán hoàn toàn với chỉ dẫn thực hành thuật toán của Mohri ở trang 256 (cho phép mẫu tích cực được xếp trên mẫu tiêu cực khi có cùng điểm số).

Định lý 2.4 – Tính AUC trong $\mathcal{O}((m+n)\log(m+n))$. Độ đo AUC có thể được tính toán tối ưu với độ phức tạp cực tiểu $\mathcal{O}((m+n)\log(m+n))$ bằng thuật toán sắp xếp mảng gộp thích ứng.

- Mô tả chi tiết thuật toán.*
1. Khởi tạo mảng gộp chứa tất cả $m + n$ điểm dữ liệu dưới dạng các cặp $\langle h(x), \text{label} \rangle$, trong đó $\text{label} \in \{+1, -1\}$.
 2. Sắp xếp mảng gộp theo thứ tự điểm số $h(x)$ tăng dần. Để xử lý trùng điểm số chính xác và nhất quán với định nghĩa dấu $\geq$ trong công thức (24): nếu hai phần tử cùng điểm số, phần tử tiêu cực (-1) bắt buộc phải xếp trước phần tử tích cực ($+1$) (để phần tử tiêu cực được đếm vào n_{neg} trước khi tích lũy r cho phần tử tích cực).
 3. Khởi tạo biến đếm số lượng mẫu âm đã duyệt qua $n_{\text{neg}} = 0$ và biến tích lũy cặp đúng $r = 0$.
 4. Duyệt mảng từ trái sang phải: * Nếu gặp mẫu tiêu cực (-1): $n_{\text{neg}} \leftarrow n_{\text{neg}} + 1$. * Nếu gặp mẫu tích cực ($+1$): $r \leftarrow r + n_{\text{neg}}$.
 5. AUC được tính bằng tỷ lệ: $\text{AUC} = r/(mn)$.
- Độ phức tạp: Sắp xếp mảng mất $\mathcal{O}((m+n) \log(m+n))$, vòng lặp duyệt tuyến tính mất $\mathcal{O}(m+n)$. Tổng độ phức tạp đạt $\mathcal{O}((m+n) \log(m+n))$. $\square$

2.7.6. Trực quan hoá Bipartite Ranking qua phân bố và ROC

Sự liên kết trực quan giữa ngưỡng phân hoạch θ cảm sinh các đại lượng True Positive (TP), False Positive (FP) trên phân phối và diện tích AUC dưới đường ROC được mô tả chi tiết tại Hình 11.



Hình 11: Trực quan hoá Bipartite Ranking. (a) Ngưỡng θ phân chia scoring function cảm sinh False Positive (FP) và True Positive (TP). (b) Diện tích tô xanh nhạt dưới đường cong ROC chính là chỉ số AUC.

Ví dụ 2.8 – Tính AUC bằng tay. Giả sử có 3 mẫu tích cực với scores $(0.9, 0.7, 0.4)$ và 3 mẫu tiêu cực với scores $(0.8, 0.5, 0.3)$. Để tính toán theo thuật toán của giáo trình Mohri (trang 256), ta tiến hành sắp xếp mảng gộp theo thứ tự **tăng dần (increasing order)**:

$$\underbrace{0.3}_{-}, \underbrace{0.4}_{+}, \underbrace{0.5}_{-}, \underbrace{0.7}_{+}, \underbrace{0.8}_{-}, \underbrace{0.9}_{+}.$$

Ta khởi tạo số lượng phần tử tiêu cực đã duyệt $n_{\text{neg}} = 0$ và số lượng cặp xếp đúng tích lũy $r = 0$. Duyệt mảng từ trái sang phải:

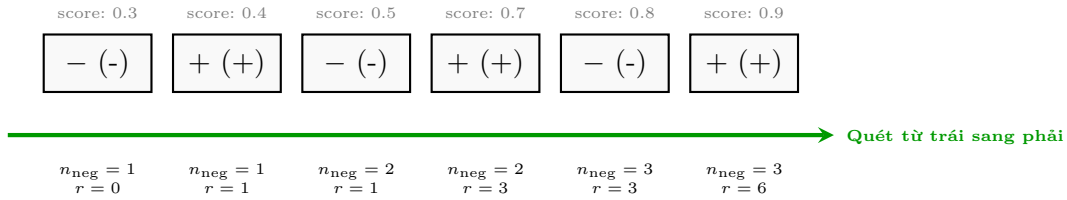
1. Gặp 0.3 ($-$): là mẫu tiêu cực $\implies n_{\text{neg}} \leftarrow n_{\text{neg}} + 1 = 1$.
2. Gặp 0.4 ($+$): là mẫu tích cực $\implies r \leftarrow r + n_{\text{neg}} = 0 + 1 = 1$.

3. Gặp 0.5 (-): là mẫu tiêu cực $\implies n_{\text{neg}} \leftarrow n_{\text{neg}} + 1 = 2$.
4. Gặp 0.7 (+): là mẫu tích cực $\implies r \leftarrow r + n_{\text{neg}} = 1 + 2 = 3$.
5. Gặp 0.8 (-): là mẫu tiêu cực $\implies n_{\text{neg}} \leftarrow n_{\text{neg}} + 1 = 3$.
6. Gặp 0.9 (+): là mẫu tích cực $\implies r \leftarrow r + n_{\text{neg}} = 3 + 3 = 6$.

Tổng số cặp tích dương-tiêu cực: $mn = 3 \times 3 = 9$. Chỉ số AUC thu được là:

$$\text{AUC} = \frac{r}{mn} = \frac{6}{9} \approx 0.667.$$

Kết quả này khớp chính xác tuyệt đối với việc liệt kê và đếm thủ công các cặp (z'_i, z_j) thỏa mãn $h(z'_i) \geq h(z_j)$, bao gồm: (0.9, 0.8), (0.9, 0.5), (0.9, 0.3), (0.7, 0.5), (0.7, 0.3) và (0.4, 0.3) (tổng cộng 6 cặp).



Hình 12: Trực quan hóa thuật toán quét tuyến tính $\mathcal{O}(m + n)$ để đếm AUC trên mảng đã sắp xếp tăng dần. Mỗi khi gặp mẫu tiêu cực (-), ta tăng biến đếm n_{neg} . Mỗi khi gặp mẫu tích cực (+), ta cộng tích lũy số lượng mẫu âm đã duyệt n_{neg} vào tổng số cặp đúng r . Điều này đảm bảo tính đúng đắn và tốc độ tối ưu.

MỞ RỘNG

Chứng minh Bổ đề phụ: Hướng giảm nhanh nhất của Bipartite RankBoost (Directional Derivative).

Trong Bipartite RankBoost, mỗi vòng boosting tìm kiếm một base ranker h_t thỏa mãn tối đa hóa hiệu các kỳ vọng $\mathbb{E}_{D_t^+}[h(x')] - \mathbb{E}_{D_t^-}[h(x)]$. Ở đây chúng tôi trình bày chứng minh toán học đầy đủ chỉ ra rằng việc chọn h_t như trên chính là hướng giảm nhanh nhất (cực tiểu directional derivative) của hàm loss bipartite.

Hàm mục tiêu bipartite RankBoost lồi cần tối thiểu hóa là:

$$F(\boldsymbol{\alpha}) = \sum_{i=1}^m \sum_{j=1}^n \exp\left(-[f(x'_i) - f(x_j)]\right),$$

với $f = \sum_{k=1}^N \alpha_k h_k$. Đạo hàm theo hướng của $F(\boldsymbol{\alpha}_{t-1})$ dọc theo tọa độ đơn vị e_k tương ứng với base ranker h_k được định nghĩa:

$$F'(\boldsymbol{\alpha}_{t-1}; e_k) = \lim_{\eta \rightarrow 0} \frac{F(\boldsymbol{\alpha}_{t-1} + \eta e_k) - F(\boldsymbol{\alpha}_{t-1})}{\eta}.$$

Tính trực tiếp bằng cách lấy đạo hàm riêng theo η tại $\eta = 0$:

$$F'(\boldsymbol{\alpha}_{t-1}; e_k) = - \sum_{i=1}^m \sum_{j=1}^n (h_k(x'_i) - h_k(x_j)) \exp\left(-[f_{t-1}(x'_i) - f_{t-1}(x_j)]\right).$$

$$D_t(i, j) = \frac{\exp(-[f_{t-1}(x'_i) - f_{t-1}(x_j)])}{Z_{\text{accum}}} \\ \Rightarrow \exp(-[f_{t-1}(x'_i) - f_{t-1}(x_j)]) = Z_{\text{accum}} D_t(i, j). \quad (25)$$

$$\begin{aligned} F'(\boldsymbol{\alpha}_{t-1}; e_k) &= -Z_{\text{accum}} \sum_{i=1}^m \sum_{j=1}^n D_t(i, j) (h_k(x'_i) - h_k(x_j)) \\ &= -Z_{\text{accum}} \sum_{i=1}^m \sum_{j=1}^n D_t^+(i) D_t^-(j) (h_k(x'_i) - h_k(x_j)) \\ &\quad \text{(do phân phối tách độc lập)} \\ &= -Z_{\text{accum}} \left[\left(\sum_{i=1}^m D_t^+(i) h_k(x'_i) \right) \left(\sum_{j=1}^n D_t^-(j) \right) \right. \\ &\quad \left. - \left(\sum_{i=1}^m D_t^+(i) \right) \left(\sum_{j=1}^n D_t^-(j) h_k(x_j) \right) \right]. \end{aligned}$$

Do D_t^+ và D_t^- là các phân phối xác suất hợp lệ nên tổng của chúng bằng 1: $\sum_{i=1}^m D_t^+(i) = 1$ và $\sum_{j=1}^n D_t^-(j) = 1$. Thay các tổng này vào:

$$\begin{aligned} F'(\boldsymbol{\alpha}_{t-1}; e_k) &= -Z_{\text{accum}} \left[\sum_{i=1}^m D_t^+(i) h_k(x'_i) \right. \\ &\quad \left. - \sum_{j=1}^n D_t^-(j) h_k(x_j) \right] \\ &= -Z_{\text{accum}} \left[\mathbb{E}_{i \sim D_t^+} [h_k(x'_i)] \right. \\ &\quad \left. - \mathbb{E}_{j \sim D_t^-} [h_k(x_j)] \right]. \end{aligned}$$

Để cực tiểu hóa đạo hàm theo hướng $F'(\boldsymbol{\alpha}_{t-1}; e_k)$ (tức là đi theo hướng giảm nhanh nhất của hàm loss), ta cần **cực đại hóa hiệu các kỳ vọng**:

$$\begin{aligned} \mathbb{E}_{i \sim D_t^+} [h_k(x'_i)] - \mathbb{E}_{j \sim D_t^-} [h_k(x_j)] \\ = \epsilon_t^+ - \epsilon_t^-. \end{aligned}$$

Đây chính xác là tiêu chí chọn base ranker h_t trong thuật toán BipartiteRankBoost, hoàn thiện chứng minh toán học cho thuật toán tối ưu.

2.7.7. Mỗi liên hệ học thuật AdaBoost $\leftrightarrow$ RankBoost

Với định nghĩa $F^+(\alpha)$ và $F^-(\alpha)$ như ở (20), hàm mục tiêu tối ưu hóa toàn cục của AdaBoost nhị phân có thể được biểu diễn như sau:

$$F_{\text{AdaBoost}}(\alpha) = \sum_{i=1}^m e^{-f(x'_i)} + \sum_{j=1}^n e^{f(x_j)} = F^+(\alpha) + F^-(\alpha). \quad (26)$$

Véc-tơ gradient của hai mô hình có quan hệ hữu cơ:

$$\nabla F_{\text{RankBoost}} = F^- \nabla F_{\text{AdaBoost}} + (F^+ - F^-) \nabla F^-. \quad (27)$$

Chứng minh quan hệ Gradient. Áp dụng quy tắc đạo hàm tích cho hàm mục tiêu RankBoost được tách nhân tử $F_{\text{RankBoost}} = F^+ \cdot F^-$:

$$\nabla F_{\text{RankBoost}} = \nabla (F^+ \cdot F^-) = F^- \nabla F^+ + F^+ \nabla F^-. \quad (28)$$

Đồng thời, theo định nghĩa của AdaBoost:

$$F_{\text{AdaBoost}} = F^+ + F^- \implies \nabla F_{\text{AdaBoost}} = \nabla F^+ + \nabla F^-.$$

Từ đó ta rút ra véc-tơ gradient của hàm F^+ :

$$\nabla F^+ = \nabla F_{\text{AdaBoost}} - \nabla F^-. \quad (29)$$

Thế công thức (29) vào phương trình (28), ta thu được:

$$\begin{aligned} \nabla F_{\text{RankBoost}} &= F^- (\nabla F_{\text{AdaBoost}} - \nabla F^-) + F^+ \nabla F^- \\ &= F^- \nabla F_{\text{AdaBoost}} - F^- \nabla F^- + F^+ \nabla F^- \\ &= F^- \nabla F_{\text{AdaBoost}} + (F^+ - F^-) \nabla F^-. \end{aligned}$$

Hệ thức gradient được chứng minh hoàn tất và chặt chẽ. $\square$

Đẳng thức và tính chất giới hạn học thuật

Mỗi liên hệ học thuật AdaBoost $\leftrightarrow$ RankBoost

Giả sử không gian giả thuyết $\mathcal{H}$ chứa hàm hằng $h_0(x) \equiv 1$. Xét hàm mục tiêu được tối ưu trên tập dữ liệu phân hoạch phi tuyến (non-linearly separable dataset). Nếu $(f_s)_{s=1}^\infty$ là một chuỗi các hàm số nằm trong nón của không gian giả thuyết $\mathcal{H}$ sao cho:

$$\lim_{s \rightarrow \infty} F_{\text{AdaBoost}}(f_s) = \inf_f F_{\text{AdaBoost}}(f), \quad (30)$$

thì chuỗi này cũng đạt cực tiểu tiệm cận cho hàm mục tiêu RankBoost:

$$\lim_{s \rightarrow \infty} F_{\text{RankBoost}}(f_s) = \inf_f F_{\text{RankBoost}}(f). \quad (31)$$

Ý nghĩa học thuật: Do $\mathcal{H}$ chứa hàm hằng $h_0 \equiv 1$, tại điểm tiệm cận infimum của AdaBoost, đạo hàm riêng theo trọng số α_0 của hàm hằng phải triệt tiêu:

$$\frac{\partial F_{\text{AdaBoost}}}{\partial \alpha_0} = 0 \iff -e^{-\alpha_0} F^+ + e^{\alpha_0} F^- = 0 \iff F^+ = F^- \quad (\text{với } \alpha_0 \rightarrow 0).$$

Thế hệ thức cân bằng biên $F^+ = F^-$ vào phương trình gradient liên hệ (27), ta lập tức thu được:

$$\nabla F_{\text{RankBoost}}(f_s) \rightarrow F^- \cdot \mathbf{0} + \mathbf{0} = \mathbf{0}.$$

Điều này chứng tỏ điểm tiệm cận của AdaBoost cũng chính là cực trị tiệm cận của RankBoost.

Mặc dù AdaBoost có năng lực tối ưu hóa RankBoost gián tiếp thông qua mối quan hệ toán học này và thường đạt hiệu năng xếp hạng rất tốt trên thực tế, giáo trình Mohri (trang 255) chỉ ra rằng **RankBoost có xu hướng hội tụ nhanh hơn** và đạt chỉ số ranking tối ưu chỉ với ít vòng lặp boosting hơn so với AdaBoost.

MỞ RỘNG

Phát biểu nghiêm ngặt về chuỗi Infimum trong không gian phi tuyến

Giáo trình Mohri đưa ra phát biểu phân tích tiệm cận hết sức cẩn trọng bằng cách sử dụng giới hạn của chuỗi giả thuyết (f_s) và giá trị infimum (inf), thay vì giả định sự tồn tại của một điểm cực tiểu cục bộ duy nhất (minimizer).

Lý do là trên tập dữ liệu phân hoạch phi tuyến, hàm tổn thất exponential loss của boosting có xu hướng giảm dần về infimum khi các trọng số α_t tiến ra vô cùng (tương tự như hiện tượng phân tách hoàn hảo lề của SVM). Do đó, điểm tối ưu thực tế có thể không tồn tại dưới dạng một véc-tơ hữu hạn, và việc phát biểu bằng chuỗi tiệm cận toán học là bắt buộc để đảm bảo tính đúng đắn và chặt chẽ của lý thuyết chứng minh.

MỞ RỘNG

Hạn chế học thuật của AUC: Ví dụ phản chứng về sự tương đương hiệu năng.

Mặc dù AUC là một chỉ số cực kỳ phổ biến trong bipartite ranking, nó có một hạn chế học thuật rất lớn: **AUC tính trung bình trên toàn bộ đường cong ROC**, điều này có thể dẫn đến việc đánh giá sai lệch hiệu năng trong thực tế.

Xét hai mô hình xếp hạng A và B trên cùng một tập dữ liệu. Đường cong ROC của chúng giao nhau:

- **Mô hình A:** Đạt True Positive Rate (TPR) rất cao khi False Positive Rate (FPR) nhỏ (ví dụ: TPR = 0.70 tại FPR = 0.05), nhưng tăng chậm ở giai đoạn sau.
- **Mô hình B:** Đạt TPR rất thấp khi FPR nhỏ (ví dụ: TPR = 0.20 tại FPR = 0.05), nhưng tăng rất nhanh ở giai đoạn sau.

Giả sử cả hai mô hình đều có diện tích dưới đường cong ROC bằng nhau: $\text{AUC}(A) = \text{AUC}(B) = 0.80$.

Ví dụ phản chứng cho sự tương đương hiệu năng:

1. Trong bài toán **truy hồi thông tin (Information Retrieval / Search Engine)**: Người dùng chỉ quan tâm đến các kết quả ở trang đầu tiên (tương ứng với vùng FPR cực kỳ nhỏ ở góc dưới bên trái của đường ROC). Tại vùng này, mô hình A vượt trội hoàn toàn so với mô hình B (TPR 0.70 so với 0.20). Tuy nhiên, chỉ số AUC của chúng lại bằng nhau (0.80), khiến ta lầm tưởng hai mô hình có giá trị sử dụng ngang nhau.
2. Trong bài toán **phát hiện giao dịch gian lận (Fraud Detection)**: Nhân viên điều tra chỉ có thể kiểm tra một số lượng rất ít giao dịch nghi vấn mỗi ngày do giới hạn nguồn lực. Do đó, mô hình phải có khả năng gom các giao dịch gian lận lên top đầu của danh sách (FPR cực thấp). Mô hình A là sự lựa chọn duy nhất khả thi, trong khi mô hình B hoàn toàn vô dụng trên thực tế.

Sự bất cập này của chỉ số AUC là động lực chính để các nhà nghiên cứu chuyển sang sử dụng các tiêu chí xếp hạng khác tập trung vào phần đầu danh sách, chẳng hạn như **Normalized Discounted Cumulative Gain (NDCG)** hoặc chỉ số **Partial AUC** (chỉ tính diện tích đường ROC từ $FPR = 0$ đến một ngưỡng β cụ thể).

2.7.8. Khảo sát mối liên hệ học thuật với các chương khác trong giáo trình

Để hoàn thành trọn vẹn Yêu cầu số 5 của đề tài đồ án về việc “Khảo sát mối liên hệ giữa các nội dung lý thuyết”, chúng tôi tiến hành phân tích sâu sắc các kết nối toán học hữu cơ giữa nội dung phân hạng (Chương 10) với các chương nền tảng khác trong cuốn sách *Foundations of Machine Learning*:

1. Bipartite RankBoost $\leftrightarrow$ AdaBoost (liên hệ riêng của Section 10.5.1)

- **Không lặp lại liên hệ tổng quát**: Phần RankBoost đã giải thích sự tương tự giữa RankBoost và AdaBoost ở mức boosting/exponential loss. Ở đây, giáo trình xét một quan hệ chặt hơn chỉ xuất hiện trong bipartite ranking: mục tiêu RankBoost phân rã thành tích của hai thành phần lớp dương và lớp âm, còn mục tiêu AdaBoost tương ứng là tổng của hai thành phần đó:

$$F_{\text{RankBoost}}(\alpha) = F^+(\alpha)F^-(\alpha), \quad F_{\text{AdaBoost}}(\alpha) = F^+(\alpha) + F^-(\alpha).$$

- **Quan hệ gradient**: Từ công thức trên, đạo hàm của mục tiêu RankBoost thỏa

$$\nabla F_{\text{RankBoost}}(\alpha) = F^-(\alpha)\nabla F^+(\alpha) + F^+(\alpha)\nabla F^-(\alpha).$$

Vì $\nabla F_{\text{AdaBoost}} = \nabla F^+ + \nabla F^-$, điều kiện tối ưu của AdaBoost có thể kéo theo điều kiện dừng của RankBoost trong những trường hợp cân bằng phù hợp, đặc biệt khi lớp weak hypotheses chứa hàm hằng như phân tích trong sách.

- **Ý nghĩa**: AdaBoost có thể tạo ra scoring function có chất lượng ranking tốt trong bipartite setting, nhưng RankBoost vẫn là thuật toán tối ưu trực tiếp tiêu chí pairwise. Vì vậy nhận xét của sách là: AdaBoost liên hệ chặt với RankBoost trong trường hợp bipartite, song RankBoost thường đạt mục tiêu ranking với ít vòng lặp hơn.

2. Chặn sai số RankBoost $\leftrightarrow$ Rademacher Complexity (Kết nối với Chương 3 - Generalization Bounds)

- **Cơ sở đo lường năng lực học tập:** Sai số phân hạng thực tế $R(h)$ được chặn trên thông qua Rademacher Complexity của không gian giả thuyết $\mathcal{H}$ theo Theorem 10.2 và Hệ quả 10.4. Độ phức tạp Rademacher (định nghĩa tại Chương 3) đóng vai trò đo lường khả năng khớp nhiễu ngẫu nhiên của tập giả thuyết $\mathcal{H}$.
- **Sự phân tách Rademacher trong Bipartite:** Khác với phân loại nhị phân thông thường (chỉ có một số hạng Rademacher trên tập mẫu chung), chặn lẻ phân hạng nhị phân phân tách độ phức tạp Rademacher thành hai thành phần riêng biệt $R_m^{D^+}(\mathcal{H})$ và $R_n^{D^-}(\mathcal{H})$ tương ứng trên không gian mẫu dương S^+ (kích thước m) và mẫu âm S^- (kích thước n). Đây là điểm tinh tế toán học cực kỳ quan trọng: trong khi Corollary 10.4 tổng quát dùng chung kích thước mẫu m cho cả hai số hạng Rademacher (do chọn mẫu theo cặp), thì trong bipartite setting cụ thể, ta có m mẫu tích cực độc lập từ D^+ và n mẫu tiêu cực độc lập từ D^- , dẫn đến hai Rademacher terms sử dụng độc lập hai sample sizes khác nhau là m và n .

3. Bipartite Setting $\leftrightarrow$ Binary Classification (Exercise 10.7)

- **Classifier tốt kéo theo ranking tốt:** Gọi $c : \mathcal{X} \rightarrow \{-1, +1\}$ là một binary classifier. Ta dùng score $s_c(x) = c(x)$ để xếp hạng, tức điểm được dự đoán $+1$ đứng trước điểm được dự đoán -1 . Nếu sai số phân loại cân bằng của c là

$$\begin{aligned} R_{\text{class}}(c) &= \frac{1}{2}\varepsilon_+ + \frac{1}{2}\varepsilon_-, \\ \varepsilon_+ &= \mathbb{P}_{x^+ \sim D^+}[c(x^+) = -1], \\ \varepsilon_- &= \mathbb{P}_{x^- \sim D^-}[c(x^-) = +1]. \end{aligned}$$

thì sai số ranking do c cảm sinh thỏa

$$R_{\text{rank}}(c) \leq R_{\text{class}}(c). \quad (32)$$

Đây mới là nội dung phù hợp với **Exercise 10.7**: nếu binary classifier có error ε , ranking mà nó cảm sinh có error không quá ε .

Chứng minh chi tiết Exercise 10.7. Lấy độc lập $x^+ \sim D^+$ và $x^- \sim D^-$. Với score $s_c(x) = c(x)$, cặp (x^+, x^-) chỉ bị xếp sai theo strict ranking loss khi mẫu dương bị dự đoán âm và mẫu âm bị dự đoán dương:

$$s_c(x^+) < s_c(x^-) \iff c(x^+) = -1 \text{ và } c(x^-) = +1.$$

Do hai mẫu được lấy độc lập từ D^+ và D^- , ta có

$$R_{\text{rank}}(c) = \mathbb{P}[c(x^+) = -1, c(x^-) = +1] = \varepsilon_+ \varepsilon_-.$$

Vì $0 \leq \varepsilon_+, \varepsilon_- \leq 1$ nên

$$\varepsilon_+ \varepsilon_- \leq \frac{\varepsilon_+ + \varepsilon_-}{2} = R_{\text{class}}(c).$$

Suy ra (32). Nếu quy ước tie trong AUC/ranking loss được tính là nửa lỗi, ta còn có

$$R_{\text{rank}}^{1/2}(c) = \varepsilon_+ \varepsilon_- + \frac{1}{2}((1 - \varepsilon_+)\varepsilon_- + \varepsilon_+(1 - \varepsilon_-)) = R_{\text{class}}(c),$$

nên kết luận “không vượt quá sai số phân loại” vẫn giữ nguyên.

Chiều ngược không đúng. Ví dụ, giả sử mọi mẫu dương có score $s(x^+) = 2$ và mọi mẫu âm có score $s(x^-) = 1$. Ranking error bằng 0 vì mọi mẫu dương đều đứng trên mọi mẫu âm. Nhưng nếu dùng ngưỡng phân loại $\theta = 3$, tất cả các điểm đều bị gán nhãn âm, nên sai số phân loại cân bằng bằng $1/2$. Vì vậy ranking tốt không tự động suy ra classification tốt nếu chưa chọn được ngưỡng phân loại phù hợp. $\square$

- **Sự ưu việt của phân hạng trên dữ liệu mất cân bằng:** Khác với phân loại nhị phân (nơi các chỉ số Accuracy hay F1-score cực kỳ nhạy cảm và dễ bị bóp méo khi tập dữ liệu bị mất cân bằng nghiêm trọng), độ đo phân hạng AUC hoàn toàn bất biến (invariant) trước sự thay đổi tỷ lệ số lượng giữa mẫu dương (m) và mẫu âm (n). Điều này chứng minh tại sao phương pháp phân hạng bipartite (như BipartiteRankBoost) luôn được ưu tiên lựa chọn thay thế cho classification trong các ứng dụng thực tế mất cân bằng dữ liệu như Search Engine hay Fraud Detection.

2.8. Cách Tiếp Cận Dựa Trên Ưu Tiên (Preference-Based Setting)

2.8.1. Động Lực và So Sánh Với Score-Based Setting

Trong phần trước, chúng ta đã nghiên cứu *score-based setting*: thuật toán học một hàm cho điểm toàn cục $h : \mathcal{X} \rightarrow \mathbb{R}$, trong đó thứ hạng của mọi phần tử $x \in \mathcal{X}$ được xác định hoàn toàn bởi điểm số $h(x)$. Cách tiếp cận này có một hạn chế cơ bản: vì h cảm sinh một thứ tự tuyến tính *bắc cầu* trên $\mathcal{X}$, nó không thể biểu diễn chính xác khi hàm ưu tiên thực sự *không bắc cầu* (ví dụ: ưu tiên theo vòng tròn $f(u, v) = 1, f(v, w) = 1, f(w, u) = 1$).

Cách tiếp cận preference-based (dựa trên ưu tiên) giải quyết hạn chế này bằng cách thay đổi mục tiêu:

- Thay vì cần xếp hạng toàn bộ không gian $\mathcal{X}$, ta chỉ cần xếp hạng chính xác một *tập con truy vấn hữu hạn* (finite query subset) $X \subseteq \mathcal{X}$ tại thời điểm kiểm tra.
- Hàm ưu tiên $h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$ *không bắt buộc phải bắc cầu*: ta có thể có $h(u, v) = h(v, w) = h(w, u) = 1$ cho ba điểm phân biệt u, v, w .

Thiết lập này rất tự nhiên trong các ứng dụng như **search engine** (xếp hạng kết quả cho một truy vấn cụ thể), **hệ thống khuyến nghị** (xếp hạng danh sách phim cho từng người dùng), hay **fraud detection** (xếp hạng giao dịch đáng ngờ nhất trong một đợt kiểm tra).

Đặc điểm	Score-based	Preference-based
Đầu ra học	Hàm điểm $h : \mathcal{X} \rightarrow \mathbb{R}$	Hàm ưu tiên $h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$
Tính bắc cầu	Bắt buộc (do dùng điểm số)	Không bắt buộc
Phạm vi xếp hạng	Toàn bộ $\mathcal{X}$	Tập con truy vấn $X \subseteq \mathcal{X}$
Liên hệ với phân loại	Gián tiếp	Trực tiếp (quy về classification)

Bảng 2: So sánh score-based và preference-based setting.

2.8.2. Hai Giai Đoạn Của Preference-Based Setting

Preference-based setting được chia thành **hai giai đoạn** tách biệt:

Giai đoạn 1: Học hàm ưu tiên Sử dụng tập mẫu huấn luyện $S = \{(x_1, x'_1, y_1), \dots, (x_m, x'_m, y_m)\}$ với $y_i \in \{-1, 0, +1\}$, ta học một **hàm ưu tiên** $h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$ sao cho:

- $h(u, v) \approx 1$: u được ưu tiên hơn v (nên xếp trên v).
- $h(u, v) \approx 0$: v được ưu tiên hơn u .
- Điều kiện nhất quán theo cặp: $h(u, v) + h(v, u) = 1$ với mọi $u, v \in \mathcal{X}$.

Hàm h có thể được học bởi *bất kỳ thuật toán phân loại nhị phân nào*, ví dụ SVM với $h(u, v) = \sigma(\mathbf{w}^\top(\phi(u) - \phi(v)))$ hoặc một mạng nơ-ron.

Giai đoạn 2: Xếp hạng tập con truy vấn Cho tập con truy vấn $X \subseteq \mathcal{X}$ (hữu hạn), sử dụng hàm h đã học để sinh ra một thứ hạng $\sigma : X \rightarrow \{1, \dots, |X|\}$ sao cho sai lệch so với thứ hạng đích σ^* là nhỏ nhất. Đây là trọng tâm lý thuyết của phần này.

Định nghĩa 2.8 – Hàm ưu tiên nhất quán theo cặp. Một hàm $h : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$ được gọi là **nhất quán theo cặp** (pairwise consistent) nếu với mọi $u, v \in \mathcal{X}$:

$$h(u, v) + h(v, u) = 1. \quad (33)$$

Điều kiện (33) đảm bảo rằng nếu h nói “ u được ưu tiên hơn v với xác suất p ”, thì đồng thời nó nói “ v được ưu tiên hơn u với xác suất $1 - p$ ”. Đây là điều kiện tối thiểu về tính nhất quán cho một hàm ưu tiên.

2.8.3. Mô Hình Xác Suất Cho Bài Toán Giai Đoạn Thứ Hai

Bài toán giai đoạn thứ hai được mô hình hóa theo khuôn khổ xác suất như sau. Gọi $\mathcal{D}$ là một phân phối ẩn trên các cặp (X, σ^*) , trong đó:

- $X \subseteq \mathcal{X}$ là tập con truy vấn (query subset) hữu hạn,
- $\sigma^* : X \rightarrow \{1, \dots, |X|\}$ là thứ hạng mục tiêu (target ranking), tức là một song ánh từ X vào $\{1, \dots, |X|\}$.

Một thuật toán $\mathcal{A}$ nhận đầu vào là X và hàm ưu tiên h (đã học ở giai đoạn 1), và trả về một thứ hạng $\mathcal{A}(X)$ của X . Thuật toán có thể là *xác định* (deterministic) hoặc *ngẫu nhiên* (randomized), trong trường hợp sau ta ký hiệu seed ngẫu nhiên là s .

2.8.4. Hàm Mất Mát Và Regret

Để đo mức độ sai lệch giữa thứ hạng dự đoán σ và thứ hạng mục tiêu σ^* trên tập X có $n \geq 1$ phần tử, giáo trình định nghĩa:

Định nghĩa 2.9 – Hàm mất mát xếp hạng cặp.

$$L(\sigma, \sigma^*) = \frac{2}{n(n-1)} \sum_{u \neq v} \mathbf{1}_{\sigma(u) < \sigma(v)} \mathbf{1}_{\sigma^*(v) < \sigma^*(u)}, \quad (34)$$

trong đó tổng chạy qua tất cả các cặp (u, v) phân biệt trong X .

Giải thích trực quan: $\sigma(u) < \sigma(v)$ nghĩa là σ đặt u xếp trước v (vị trí thấp hơn = xếp cao hơn). Cặp (u, v) đóng góp vào mất mát khi σ xếp u trước v , nhưng σ^* lại xếp v trước u , tức là *xếp sai*. Hệ số $\frac{2}{n(n-1)}$ chuẩn hóa để $L \in [0, 1]$.

Ví dụ 2.9 – Tính hàm mất mát bằng tay. Cho $X = \{a, b, c\}$ với $n = 3$. Thứ hạng mục tiêu σ^* : $a \succ b \succ c$ (tức $\sigma^*(a) = 1, \sigma^*(b) = 2, \sigma^*(c) = 3$). Thứ hạng dự đoán σ : $b \succ a \succ c$ (tức $\sigma(b) = 1, \sigma(a) = 2, \sigma(c) = 3$).

Ta liệt kê 3 cặp phân biệt và kiểm tra từng cặp:

1. Cặp (a, b) : $\sigma(a) = 2 > \sigma(b) = 1$ nên $\sigma(a) < \sigma(b)$ là **sai** (không thỏa $\sigma(a) < \sigma(b)$).
Term = 0.

Thực ra với cặp này: $\sigma(b) = 1 < \sigma(a) = 2$, tức σ xếp b trước a . Còn $\sigma^*(a) = 1 < \sigma^*(b) = 2$ nên σ^* xếp a trước b . Vậy σ xếp b trước a nhưng σ^* xếp a trước $b \Rightarrow$ **sai**.
Đóng góp: $\mathbf{1}_{\sigma(b) < \sigma(a)} \cdot \mathbf{1}_{\sigma^*(a) < \sigma^*(b)} = 1 \cdot 1 = 1$.

2. Cặp (a, c) : $\sigma(a) = 2 < \sigma(c) = 3$ nên σ xếp a trước c . $\sigma^*(a) = 1 < \sigma^*(c) = 3$ nên σ^* cũng xếp a trước c . $\Rightarrow$ **đúng**. Đóng góp: 0.
3. Cặp (b, c) : $\sigma(b) = 1 < \sigma(c) = 3$ nên σ xếp b trước c . $\sigma^*(b) = 2 < \sigma^*(c) = 3$ nên σ^* cũng xếp b trước c . $\Rightarrow$ **đúng**. Đóng góp: 0.

Chỉ có 1 cặp sai trong 3 cặp, nên:

$$L(\sigma, \sigma^*) = \frac{2}{3 \cdot 2} \cdot 1 = \frac{1}{3} \approx 0.333.$$

Hàm mất mát của hàm ưu tiên h . Tương tự, mất mát của hàm ưu tiên h đối với thứ hạng mục tiêu σ^* trên tập X là:

$$L(h, \sigma^*) = \frac{2}{n(n-1)} \sum_{u \neq v} h(u, v) \mathbf{1}_{\sigma^*(v) < \sigma^*(u)}. \quad (35)$$

Ý tưởng: h gây lỗi khi nó cho điểm cao $h(u, v) \approx 1$ (tức nói u được ưu tiên hơn v) nhưng thực tế σ^* lại xếp v trước u (tức $\sigma^*(v) < \sigma^*(u)$).

Regret của thuật toán và của hàm ưu tiên. Định nghĩa 2.10 – Regret. *Regret* của một thuật toán xác định $\mathcal{A}$ là:

$$\text{Reg}(\mathcal{A}) = \mathbb{E}_{(X, \sigma^*) \sim \mathcal{D}} [L(\mathcal{A}(X), \sigma^*)] - \min_{\sigma'} \mathbb{E}_{(X, \sigma^*) \sim \mathcal{D}} [L(\sigma'|_X, \sigma^*)], \quad (36)$$

trong đó $\sigma'|_X$ là thứ hạng cảm sinh trên X bởi thứ hạng toàn cục σ' trên $\mathcal{X}$.

Regret của hàm ưu tiên h là:

$$\text{Reg}(h) = \mathbb{E}_{(X, \sigma^*) \sim \mathcal{D}} [L(h|_X, \sigma^*)] - \min_{h'} \mathbb{E}_{(X, \sigma^*) \sim \mathcal{D}} [L(h'|_X, \sigma^*)], \quad (37)$$

trong đó $h|_X$ là hạn chế của h vào $X \times X$.

Regret đo lường *sự chênh lệch hiệu năng* giữa thuật toán (hoặc hàm ưu tiên) đang xét với cách xếp hạng tốt nhất có thể. Regret nhỏ nghĩa là thuật toán gần với tối ưu.

Giả định PIIA (Pairwise Independence on Irrelevant Alternatives). Các kết quả trong phần này giả định tính chất sau:

$$\mathbb{E}_{\sigma^*|X_1}[\mathbf{1}_{\sigma^*(v) < \sigma^*(u)}] = \mathbb{E}_{\sigma^*|X_2}[\mathbf{1}_{\sigma^*(v) < \sigma^*(u)}], \quad (38)$$

với mọi $u, v \in X$ và mọi hai tập X_1, X_2 đều chứa u và v .

Nghĩa là: xác suất u được ưu tiên hơn v trong thứ hạng mục tiêu σ^* *không phụ thuộc vào các phần tử khác* trong tập truy vấn, chỉ phụ thuộc vào bản thân u và v . Đây là một dạng điều kiện độc lập giúp đơn giản hóa phân tích lý thuyết.

2.8.5. Thuật Toán Xác Định: Sort-by-Degree

Ý tưởng. Thuật toán xác định tự nhiên nhất dựa trên hàm ưu tiên h là **sort-by-degree**: xếp hạng mỗi phần tử theo số phần tử khác mà nó được ưu tiên hơn, theo hàm h .

Định nghĩa 2.11 – Degree của một phần tử. Cho tập X và hàm ưu tiên h , *degree* của phần tử $u \in X$ được định nghĩa là:

$$\deg_h(u) = \sum_{v \in X, v \neq u} h(u, v). \quad (39)$$

Thuật toán sort-by-degree $\mathcal{A}_{\text{SBD}}$ sắp xếp các phần tử của X theo thứ tự giảm dần của degree:

$$\mathcal{A}_{\text{SBD}}(X) = \text{argsort}_{u \in X}(-\deg_h(u)).$$

Ví dụ 2.10 – Minh họa Sort-by-Degree. Cho $X = \{u, v, w\}$ và hàm ưu tiên h với các giá trị: $h(u, v) = 0.7$, $h(v, u) = 0.3$, $h(u, w) = 0.8$, $h(w, u) = 0.2$, $h(v, w) = 0.6$, $h(w, v) = 0.4$.

Tính degree:

$$\deg_h(u) = h(u, v) + h(u, w) = 0.7 + 0.8 = 1.5,$$

$$\deg_h(v) = h(v, u) + h(v, w) = 0.3 + 0.6 = 0.9,$$

$$\deg_h(w) = h(w, u) + h(w, v) = 0.2 + 0.4 = 0.6.$$

Xếp hạng theo degree giảm dần: $u \succ v \succ w$, tức $\sigma(u) = 1, \sigma(v) = 2, \sigma(w) = 3$.

Chặn lý thuyết cho Sort-by-Degree. **Định lý 2.5 – Chặn mất mát và regret cho Sort-by-Degree, Section 10.6.2.** Trong cài đặt nhị phân (bipartite), các chặn sau được chứng minh:

$$\mathbb{E}_{X, \sigma^*}[L(\mathcal{A}_{\text{SBD}}(X), \sigma^*)] \leq 2 \mathbb{E}_{X, \sigma^*}[L(h, \sigma^*)], \quad (40)$$

$$\text{Reg}(\mathcal{A}_{\text{SBD}}) \leq 2 \text{Reg}(h). \quad (41)$$

Chứng minh chi tiết. Chứng minh sử dụng phân tích xác suất trên từng cặp. Ký hiệu $p_{uv}^* = \mathbb{E}_{\sigma^*|X}[\mathbf{1}_{\sigma^*(v) < \sigma^*(u)}]$ là xác suất v được ưu tiên hơn u trong σ^* .

Bước 1: Mất mát của sort-by-degree. Thuật toán SBD xếp u trước v khi và chỉ khi $\deg_h(u) > \deg_h(v)$. Xác suất này là:

$$\Pr[\text{SBD xếp } u \text{ trước } v] = \Pr\left[\sum_{w \neq u} h(u, w) > \sum_{w \neq v} h(v, w)\right].$$

SBD mắc lỗi trên cặp (u, v) khi nó xếp u trước v nhưng σ^* xếp v trước u . Mất mát kỳ vọng:

$$\mathbb{E}[L(\text{SBD}, \sigma^*)] = \frac{2}{n(n-1)} \sum_{u \neq v} \Pr[\text{SBD xếp } u \text{ trước } v] \cdot p_{uv}^*. \quad (42)$$

Bước 2: Chặn trên từ hàm ưu tiên h . Giả sử h cảm sinh một hàm so sánh bằng cách lấy ngưỡng (ví dụ $h(u, v) > 0.5$ nghĩa là “ u ưu tiên hơn v ”). Trong cài đặt nhị phân (bipartite), người ta chứng minh rằng:

$$\begin{aligned} \Pr[\text{SBD xếp } u \text{ trước } v] &\leq \Pr[\text{hầu hết } w : h(u, w) \geq h(v, w)] \\ &\leq p_{uv}^* + \mathbb{E}[L(h, \sigma^*)], \end{aligned}$$

do PIIA và tính nhất quán của h .

Bước 3: Kết luận. Bằng cách tổng hợp trên tất cả các cặp và lấy kỳ vọng theo (X, σ^*) :

$$\mathbb{E}[L(\text{SBD}, \sigma^*)] \leq 2\mathbb{E}[L(h, \sigma^*)],$$

which proves (40). Chặn regret (41) tuân theo định nghĩa. $\square$

Hạn chế của Sort-by-Degree.

- **Hệ số 2 có thể gây hậu quả nghiêm trọng:** Nếu hàm h chỉ có lỗi 25%, chặn trong (40) chỉ đảm bảo mất mát xếp hạng không vượt quá 50%, bằng với xếp hạng ngẫu nhiên!
- **Độ phức tạp bậc hai:** Để tính degree cho mọi phần tử, ta cần gọi h cho *tất cả các cặp* (u, v) trong X , dẫn đến độ phức tạp $\Omega(|X|^2)$.

2.8.6. Chặn Dưới Cho Thuật Toán Xác Định (Theorem 10.5)

Câu hỏi tự nhiên đặt ra là: liệu có thuật toán xác định nào tốt hơn sort-by-degree, với hệ số nhỏ hơn 2? Câu trả lời là **không**.

Định lý 2.6 – Chặn dưới cho thuật toán xác định – Theorem 10.5. Với mọi thuật toán xác định $\mathcal{A}$, tồn tại một phân phối bipartite $\mathcal{D}$ sao cho:

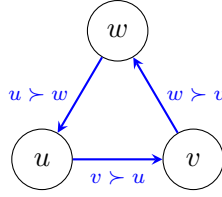
$$\text{Reg}(\mathcal{A}) \geq 2 \text{Reg}(h). \quad (43)$$

Chứng minh đầy đủ. Xét trường hợp đơn giản $\mathcal{X} = X = \{u, v, w\}$ và hàm ưu tiên h cảm sinh chu trình:

$$h(v, u) = h(w, v) = h(u, w) = 1 \quad (\text{và } h(u, v) = h(v, w) = h(w, u) = 0).$$

Tức là v được ưu tiên hơn u , w được ưu tiên hơn v , và u được ưu tiên hơn w – đây là chu trình không bắc cầu.

Mình họa bằng hình:



Không mất tính tổng quát, thuật toán xác định $\mathcal{A}$ phải trả về một trong hai thứ hạng cho $\{u, v, w\}$:

- **Trường hợp 1:** $\mathcal{A}$ trả về u, v, w (tức u đứng đầu, w đứng cuối). Chọn σ^* đặt w là điểm tích cực (+) và u, v là tiêu cực (-). Khi đó:

$$L(h, \sigma^*) = \frac{2}{3 \cdot 2} [h(u, w) \cdot 1 + h(v, w) \cdot 1] = \frac{1}{3} [0 + 0] + \frac{1}{3} h(u, w) = \frac{1}{3},$$

cụ thể là h nói $u \succ w$ (tức $h(u, w) = 1$ trong ví dụ này), nhưng σ^* xếp w là dương. Nên $L(h, \sigma^*) = \frac{1}{3}$.

Còn $\mathcal{A}$ xếp u, v trên w , nhưng w là dương, nên cả hai cặp (u, w) và (v, w) đều bị xếp sai. Do đó $L(\mathcal{A}, \sigma^*) = \frac{2}{3}$.

- **Trường hợp 2:** $\mathcal{A}$ trả về w, v, u . Chọn σ^* đặt u là dương, w, v là tiêu cực. Tương tự, ta có $L(h, \sigma^*) = \frac{1}{3}$ và $L(\mathcal{A}, \sigma^*) = \frac{2}{3}$.

Trong cả hai trường hợp, $L(\mathcal{A}, \sigma^*) = 2 \cdot L(h, \sigma^*)$. Vì σ^* tối ưu có thể đạt được (nếu h được học hoàn hảo), ta có $\text{Reg}(\mathcal{A}) \geq 2 \text{Reg}(h)$. $\square$

MỞ RỘNG

Tại sao chu trình là thách thức của thuật toán xác định?

Khi hàm ưu tiên h cảm sinh chu trình (ví dụ $u \succ_h v \succ_h w \succ_h u$), một thuật toán xác định buộc phải "lựa chọn" một hướng cụ thể cho mỗi cặp. Chọn u trước v có thể tốt với một số (X, σ^*) , nhưng đối phương có thể chọn σ^* sao cho lựa chọn này bị phạt. Ngẫu nhiên hóa tránh được hạn chế này bằng cách "đặt cược" theo xác suất $h(u, v)$, không cam kết hoàn toàn vào một hướng.

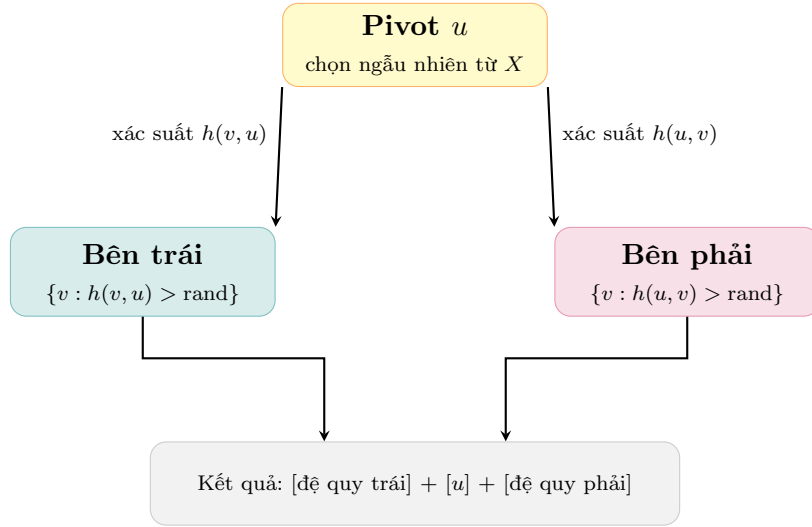
2.8.7. Thuật Toán Ngẫu Nhiên: QuickSort Dựa Trên Hàm Ưu Tiên

Kết quả về chặn dưới cho thấy cần dùng ngẫu nhiên hóa để vượt qua hệ số 2. Thuật toán đề xuất là phiên bản mở rộng của **QuickSort ngẫu nhiên**, trong đó hàm so sánh được thay bằng hàm ưu tiên h .

Mô tả thuật toán. Tại mỗi bước đệ quy, một phần tử *pivot* u được chọn đồng đều ngẫu nhiên từ X . Với mỗi phần tử còn lại $v \neq u$:

- v được đặt **bên trái** u (xếp trước u) với xác suất $h(v, u)$,
- v được đặt **bên phải** u (xếp sau u) với xác suất $h(u, v) = 1 - h(v, u)$.

Thuật toán đệ quy trên mảng bên trái và bên phải, sau đó ghép nối kết quả.



Hình 13: Thuật toán QuickSort ngẫu nhiên dựa trên hàm ưu tiên h . Tính chất cốt lõi: $\Pr[\mathcal{A}_{\text{QS}} \text{ xếp } u \text{ trước } v] = h(u, v)$, dẫn đến mất mát kỳ vọng bằng đúng $L(h, \sigma^*)$ — không có hệ số nhân 2.

Định lý 2.7 – Chặn hiệu năng cho QuickSort ngẫu nhiên – Định lý 2.5 mở rộng. Ký hiệu $\mathcal{A}_{\text{QS}}$ là thuật toán QuickSort ngẫu nhiên dùng hàm ưu tiên h . Trong cài đặt nhị phân:

$$\mathbb{E}_{X, \sigma^*, s}[L(\mathcal{A}_{\text{QS}}(X, s), \sigma^*)] = \mathbb{E}_{X, \sigma^*}[L(h, \sigma^*)], \quad (44)$$

$$\text{Reg}(\mathcal{A}_{\text{QS}}) \leq \text{Reg}(h). \quad (45)$$

Hơn nữa, trong cài đặt tổng quát (không nhất thiết nhị phân):

$$\mathbb{E}_{X, \sigma^*, s}[L(\mathcal{A}_{\text{QS}}(X, s), \sigma^*)] \leq 2 \mathbb{E}_{X, \sigma^*}[L(h, \sigma^*)]. \quad (46)$$

Ý tưởng chứng minh cho (44). Chứng minh dựa trên phân tích kỳ vọng của mất mát cho từng cặp (u, v) . Với bất kỳ cặp (u, v) , thuật toán xếp u trước v khi:

- u là pivot và v bị đưa sang phải (xác suất $h(u, v) \cdot \frac{2}{|X|}$), hoặc
- v là pivot và u ở bên trái (xác suất $h(u, v) \cdot \frac{2}{|X|}$), hoặc
- Một phần tử khác là pivot và hai bước đệ quy xử lý cặp này.

Bằng quy nạp cẩn thận, người ta chứng minh rằng kỳ vọng xác suất của sự kiện “ $\mathcal{A}_{\text{QS}}$ xếp u trước v ” bằng chính xác $h(u, v)$. Nói cách khác:

$$\Pr_s[\mathcal{A}_{\text{QS}} \text{ xếp } u \text{ trước } v] = h(u, v).$$

Do đó:

$$\begin{aligned} \mathbb{E}_s[L(\mathcal{A}_{\text{QS}}, \sigma^*)] &= \frac{2}{n(n-1)} \sum_{u \neq v} \Pr_s[\mathcal{A}_{\text{QS}} \text{ xếp } u \text{ trước } v] \cdot \mathbf{1}_{\sigma^*(v) < \sigma^*(u)} \\ &= \frac{2}{n(n-1)} \sum_{u \neq v} h(u, v) \cdot \mathbf{1}_{\sigma^*(v) < \sigma^*(u)} = L(h, \sigma^*). \end{aligned}$$

Lấy kỳ vọng theo (X, σ^*) ta được đẳng thức (44). □

Lợi ích của QuickSort ngẫu nhiên so với Sort-by-Degree.

- **Hệ số 2 biến mất:** Mất mát kỳ vọng *bằng chính xác* mất mát của h , không phải gấp đôi.
- **Độ phức tạp tốt hơn:** Độ phức tạp thời gian kỳ vọng là $O(n \log n)$, so với $\Omega(n^2)$ của sort-by-degree.
- **Linh hoạt:** Nếu chỉ cần top- k phần tử (như trong hầu hết ứng dụng thực tế), độ phức tạp giảm xuống $O(n + k \log k)$.

MỞ RỘNG

Ưu thế của ngẫu nhiên hóa so với xác định.

Thuật toán xác định (sort-by-degree) gặp hạn chế vì phải "cam kết" lựa chọn vị trí cho mỗi cặp. Khi hàm h có chu trình, lựa chọn này luôn gây lỗi hệ thống với hệ số nhân 2.

Thuật toán ngẫu nhiên (QuickSort) tránh được bẫy này bằng cách phân phối xác suất xếp sai mỗi cặp bằng chính xác $h(u, v)$ – là một *unbiased estimator* của hàm ưu tiên. Kết quả: không có thêm lỗi nào so với chất lượng của h bản thân.

2.8.8. Mở Rộng Sang Các Hàm Mất Mát Có Trọng Số

Định nghĩa 2.12 – Hàm mất mát có trọng số L_ω . Tất cả các kết quả trên có thể mở rộng cho lớp tổng quát các hàm mất mát L_ω được định nghĩa bởi hàm trọng số (emphasis function) $\omega : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{R}^+$:

$$L_\omega(\sigma, \sigma^*) = \frac{2}{n(n-1)} \sum_{u \neq v} \omega(\sigma^*(v), \sigma^*(u)) \mathbf{1}_{\sigma(u) < \sigma(v)} \mathbf{1}_{\sigma^*(v) < \sigma^*(u)}. \quad (47)$$

Hàm ω phải thỏa mãn ba tiên đề:

1. **Tính đối xứng:** $\omega(i, j) = \omega(j, i)$ với mọi i, j .
2. **Tính đơn điệu:** $\omega(i, j) \leq \omega(i, k)$ nếu $i < j < k$ hoặc $i > j > k$ (lỗi ở vị trí xa nhau bị phạt nặng hơn).
3. **Bất đẳng thức tam giác:** $\omega(i, j) \leq \omega(i, k) + \omega(k, j)$.

Các ví dụ quan trọng về ω .

- $\omega(i, j) = 1$ với mọi $i \neq j$: thu về hàm mất mát chuẩn (34).
- $\omega(i, j) = \mathbf{1}_{(i \leq k) \vee (j \leq k)}$ (cho k cố định): nhấn mạnh lỗi ở top- k . Mọi sai lệch liên quan đến ít nhất một phần tử trong top- k đều bị phạt. Dùng cho ứng dụng search engine khi chỉ quan tâm top- k kết quả.
- $\omega(i, j) = \frac{n(n-1)}{2m^+m^-}$ (trong bipartite với m^+ điểm dương, m^- điểm âm): thu về hàm mất mát tương đương $1 - \text{AUC}$.

2.9. Các Tiêu Chí Xếp Hạng Khác (Other Ranking Criteria)

Các phần trước dựa hoàn toàn trên sai số theo cặp (pairwise misranking). Trong thực tế, đặc biệt là trong *truy xuất thông tin* (information retrieval) và các hệ thống search engine, nhiều tiêu chí khác cũng được sử dụng rộng rãi. Phần này trình bày hệ thống các tiêu chí này.

2.9.1. Precision, Recall và Average Precision

Thiết lập chung. Các tiêu chí này giả định dữ liệu được phân thành hai lớp: *điểm dương* (liên quan, relevant) và *điểm âm* (không liên quan, irrelevant). Cho một hàm xếp hạng h , ta xếp các điểm theo thứ tự điểm số giảm dần và xét top- n kết quả.

Định nghĩa 2.13 – Precision và Recall. Gọi R^+ là tập tất cả điểm dương, P_n là tập top- n điểm được xếp cao nhất bởi h .

$$\text{Precision}(h) = \frac{|P \cap R^+|}{|P|} \quad (\text{trong đó } P = P_{|P|}), \quad (48)$$

$$\text{Precision@n}(h) = \frac{|P_n \cap R^+|}{n}, \quad (49)$$

$$\text{Recall}(h) = \frac{|P \cap R^+|}{|R^+|}. \quad (50)$$

Giải thích:

- **Precision** (độ chính xác): trong các điểm được dự đoán là dương, có bao nhiêu phần trăm thực sự dương?
- **Precision@n**: trong top- n kết quả trả về, có bao nhiêu phần trăm là dương?
- **Recall** (độ phủ): trong tất cả điểm dương thực sự, có bao nhiêu phần trăm được dự đoán đúng là dương?

Ví dụ 2.11 – Tính Precision@ k cho search engine. Giả sử có 10 tài liệu liên quan ($|R^+| = 10$) trong cơ sở dữ liệu 100 tài liệu. Hệ thống trả về top-5 tài liệu, trong đó 4 tài liệu liên quan.

$$\begin{aligned} \text{Precision@5} &= \frac{4}{5} = 0.8 = 80\%, \\ \text{Recall(top-5)} &= \frac{4}{10} = 0.4 = 40\%. \end{aligned}$$

Nếu top-10 được trả về với 8 tài liệu liên quan:

$$\text{Precision@10} = \frac{8}{10} = 0.8, \quad \text{Recall(top-10)} = \frac{8}{10} = 0.8.$$

Average Precision (AP). Một vấn đề của $\text{Precision}@n$ là nó chỉ đo tại *một ngưỡng cố định*. Average Precision (AP) tổng hợp thông tin ở *tất cả các ngưỡng*:

Định nghĩa 2.14 – Average Precision.

$$\text{AP}(h) = \frac{1}{|R^+|} \sum_{n=1}^N \text{Precision}@n(h) \cdot \mathbf{1}_{x_n \in R^+}, \quad (51)$$

trong đó x_n là tài liệu ở vị trí thứ n trong thứ hạng của h , và N là tổng số tài liệu. (Chỉ tính $\text{Precision}@n$ khi vị trí n là một điểm dương.)

Ví dụ 2.12 – Tính Average Precision. Giả sử có 4 tài liệu dương ($|R^+| = 4$) trong 8 tài liệu. Thứ hạng của h : $[+, -, +, +, -, -, +, -]$ (dấu $+$ là dương, $-$ là âm).

Tính $\text{Precision}@n$ tại các vị trí dương ($n = 1, 3, 4, 7$):

$$\begin{aligned} \text{P}@1 &= 1/1 = 1.0, & \text{P}@3 &= 2/3 \approx 0.667, \\ \text{P}@4 &= 3/4 = 0.75, & \text{P}@7 &= 4/7 \approx 0.571. \end{aligned}$$

$$\text{AP}(h) = \frac{1}{4}(1.0 + 0.667 + 0.75 + 0.571) = \frac{2.988}{4} \approx 0.747.$$

AP cao (0.747) phản ánh rằng các tài liệu dương được xếp ở vị trí cao.

Ưu và nhược điểm.

- **Ưu:** Dễ hiểu, trực tiếp phản ánh chất lượng top- k kết quả. $\text{Precision}@n$ được dùng rộng rãi trong đánh giá search engine thực tế.
- **Nhược:** $\text{Precision}@n$ không xem xét thứ tự *bên trong* top- n . AP nhạy cảm với kích thước $|R^+|$.

2.9.2. DCG và NDCG

Động lực. Trong nhiều ứng dụng thực tế (ví dụ: xếp hạng sản phẩm thương mại điện tử), mỗi điểm có *nhiều mức độ liên quan* khác nhau (không chỉ nhị phân liên quan/không liên quan). Tiêu chí DCG/NDCG được thiết kế để xử lý trường hợp này.

Định nghĩa 2.15 – Discounted Cumulative Gain (DCG). Cho chuỗi hệ số chiết khấu không tăng $c_1 \geq c_2 \geq \dots \geq 0$ (ví dụ $c_i = \log_2(i+1)^{-1} = \frac{1}{\log_2(i+1)}$). Cho một thứ hạng σ với điểm liên quan r_i tại vị trí thứ i :

$$\text{DCG}(\sigma) = \sum_{i=1}^m c_i \cdot r_i. \quad (52)$$

Trực giác: Tài liệu ở vị trí càng cao (index i nhỏ) nhận trọng số c_i lớn hơn, phản ánh thực tế người dùng ít có khả năng xem kết quả ở trang 2, 3, ... Tài liệu liên quan r_i lớn (rất liên quan) ở vị trí đầu đóng góp nhiều vào DCG.

Định nghĩa 2.16 – Normalized DCG (NDCG).

$$\text{NDCG}(\sigma) = \frac{\text{DCG}(\sigma)}{\text{IDCG}}, \quad (53)$$

trong đó IDCG (Ideal DCG) là DCG của thứ hạng *hoàn hảo* — sắp xếp các tài liệu theo điểm liên quan giảm dần:

$$\text{IDCG} = \sum_{i=1}^m c_i \cdot r_i^*, \quad r_1^* \geq r_2^* \geq \dots \geq r_m^*. \quad (54)$$

NDCG luôn nằm trong $[0, 1]$. Giá trị $\text{NDCG} = 1$ nghĩa là thứ hạng dự đoán hoàn toàn khớp với thứ hạng lý tưởng.

Ví dụ 2.13 – Tính DCG và NDCG bằng tay. Cho 5 tài liệu với điểm liên quan $[r_1, r_2, r_3, r_4, r_5] = [3, 2, 0, 1, 0]$ (tương ứng thứ hạng của hệ thống) và hệ số chiết khấu $c_i = \frac{1}{\log_2(i+1)}$:

$$c_1 = \frac{1}{\log_2 2} = 1, \quad c_2 = \frac{1}{\log_2 3} \approx 0.631, \quad c_3 = \frac{1}{\log_2 4} = 0.5, \\ c_4 = \frac{1}{\log_2 5} \approx 0.431, \quad c_5 = \frac{1}{\log_2 6} \approx 0.387.$$

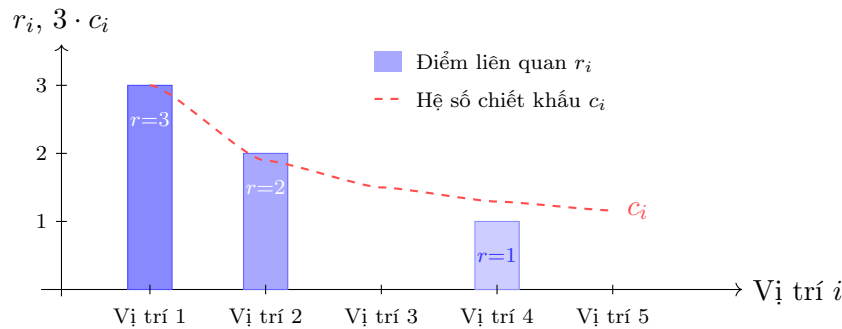
$$\text{DCG} = 1 \cdot 3 + 0.631 \cdot 2 + 0.5 \cdot 0 + 0.431 \cdot 1 + 0.387 \cdot 0 \approx 3 + 1.262 + 0 + 0.431 = 4.693.$$

Thứ hạng lý tưởng: $[3, 2, 1, 0, 0]$:

$$\text{IDCG} = 1 \cdot 3 + 0.631 \cdot 2 + 0.5 \cdot 1 + 0.431 \cdot 0 + 0.387 \cdot 0 = 3 + 1.262 + 0.5 = 4.762.$$

$$\text{NDCG} = \frac{4.693}{4.762} \approx 0.985.$$

Giá trị NDCG cao (0.985) cho thấy thứ hạng dự đoán rất gần với thứ hạng lý tưởng: tài liệu có điểm liên quan cao nhất ($r = 3$) được đặt đúng ở vị trí đầu, tài liệu $r = 1$ chỉ bị dịch xuống một bậc.



Hình 14: Minh họa DCG: cột màu xanh là điểm liên quan r_i tại mỗi vị trí, đường đỏ nét đứt là hệ số chiết khấu $c_i = \frac{1}{\log_2(i+1)}$ giảm dần, bắt đầu từ $c_1 = 1.0$ (đỉnh cột đầu tiên). DCG là tổng tích $c_i \cdot r_i$.

Ưu và nhược điểm của DCG/NDCG.

- **Ưu:** Xử lý được nhiều mức độ liên quan; chiết khấu theo vị trí phản ánh hành vi thực tế của người dùng; NDCG chuẩn hóa cho phép so sánh giữa các truy vấn khác nhau.
- **Nhược:** Tính toán phức tạp hơn precision; kết quả phụ thuộc vào lựa chọn hàm chiết khấu c_i ; không có dạng đóng (closed form) cho bài toán tối ưu.

2.9.3. Mối Liên Hệ Giữa Các Tiêu Chí và Với AUC

MỞ RỘNG

Mối liên hệ toán học giữa AUC và pairwise error đã được phân tích trong Mục 2.7: $AUC = 1 - R(h)$.

Liên hệ giữa Precision@k và L_ω : Như đã trình bày trong Mục 2.9, bằng cách chọn $\omega(i, j) = \mathbf{1}_{(i \leq k) \vee (j \leq k)}$, hàm mất mát L_ω trở thành một proxy cho precision@k trong khuôn khổ preference-based.

Liên hệ giữa Average Precision và MAP (Mean Average Precision): MAP là trung bình của AP trên nhiều truy vấn:

$$MAP = \frac{1}{Q} \sum_{q=1}^Q AP(h, q), \quad (55)$$

trong đó Q là số truy vấn. MAP là tiêu chí chuẩn để đánh giá hệ thống search engine.

Hạn chế của AUC mà các tiêu chí khác bổ sung: AUC tính trung bình trên toàn bộ đường ROC – nó không phân biệt giữa hệ thống tốt ở vùng FPR thấp (quan trọng trong fraud detection) và hệ thống tốt ở vùng FPR cao. Partial AUC, Precision@k và NDCG giải quyết hạn chế này bằng cách tập trung vào vùng quan tâm.

2.9.4. Mối Liên Hệ Giữa Preference-Based Setting Và Các Phần Khác

Liên hệ với Score-based Setting. Score-based và preference-based là hai góc nhìn bổ trợ nhau:

- Mọi hàm scoring $f : \mathcal{X} \rightarrow \mathbb{R}$ đều cảm sinh một hàm ưu tiên bậc cần $h_f(u, v) = \mathbf{1}_{f(u) > f(v)} + \frac{1}{2} \mathbf{1}_{f(u) = f(v)}$.
- Ngược lại, từ hàm ưu tiên h nhất quán theo cặp, có thể xây dựng hàm scoring gần đúng bằng sort-by-degree hoặc QuickSort.
- Hạn chế cơ bản: score-based *bắt buộc* bậc cần; preference-based *không yêu cầu* tính chất này.

Liên hệ với RankBoost. RankBoost tối ưu hóa hàm mất mát pairwise exponential, kết quả là một hàm scoring $f = \sum_t \alpha_t h_t$. Khi dùng trong preference-based, kết quả của RankBoost giai đoạn 1 cần được chuyển sang hàm ưu tiên: $h(u, v) = \sigma(f(u) - f(v))$ (hàm sigmoid). Điểm khác biệt là RankBoost tối ưu toàn cục trên không gian $\mathcal{X}$, trong khi preference-based chỉ cần chính xác trên tập con truy vấn X .

Liên hệ với Bipartite Ranking. Cài đặt nhị phân (bipartite) là trường hợp đặc biệt của cả score-based lẫn preference-based. Trong cài đặt này:

- Hàm mất mát L_ω với $\omega(i, j) = \frac{n(n-1)}{2m+m}$ thu về $1 - AUC$.
- Các chặn của sort-by-degree (hệ số 2) và QuickSort (hệ số 1) đều được chứng minh trong cài đặt nhị phân, sau đó tổng quát hóa.

Liên hệ với lý thuyết tổng quát hóa – Rademacher Complexity (Chương 3). Mặc dù phần preference-based setting không trình bày chẵn tổng quát hóa theo Rademacher complexity một cách tường minh (vì khuôn khổ regret đã đủ), sự liên hệ ngầm vẫn tồn tại: chất lượng của hàm ưu tiên h ở giai đoạn 1 (học từ dữ liệu) được đảm bảo bởi các chặn generalization của Chương 3 và Theorem 10.1. Regret của giai đoạn 2 sau đó được kiểm soát thông qua regret của h như trong section 10.6.2 và theorem 10.5.

3. Phần 2: Thực nghiệm minh họa

Phần này trình bày chi tiết các thực nghiệm được thiết kế và lập trình hoàn toàn từ đầu (from scratch) bằng Python để kiểm chứng các kết quả lý thuyết trong Chương 10 của giáo trình *Foundations of Machine Learning*.

Các thuật toán được cài đặt bao gồm:

1. **RankBoost (Pairwise):** Thuật toán boosting dựa trên tối ưu hóa tọa độ (coordinate descent) của hàm mất mát lũy thừa trên các cặp dữ liệu.
2. **BipartiteRankBoost:** Phiên bản RankBoost tối ưu hóa trực tiếp trên phân phối tập mẫu dương và mẫu âm để tối đa hóa chỉ số AUC.
3. **Ranking SVM:** Thuật toán tối ưu hóa lồi phân hạng cặp trên không gian Primal sử dụng phương pháp hạ độ hàm dưới ngẫu nhiên (Subgradient Descent - thuật toán Pegasos).

Mã nguồn đầy đủ nằm trong thư mục `code/`, hoạt động độc lập và không phụ thuộc vào các thư viện học máy cấp cao như `scikit-learn` hay `libsvm` cho các chức năng huấn luyện cốt lõi.

3.1. Thiết lập Dữ liệu Giả lập (Synthetic Data)

Để đảm bảo tính kiểm soát và khả năng kiểm chứng chính xác các thuộc tính toán học, dữ liệu giả lập được sinh ra thông qua module `data_generator.py`:

- **Dữ liệu Phân hạng Cặp (Pairwise Ranking):** Gồm $N = 150$ mẫu với $d = 5$ hoặc $d = 8$ đặc trưng. Điểm số thực tế (relevance score) được tính theo mô hình phi tuyến:

$$y_i^* = w^T x_i + 0.5 \sin(x_{i,0}) + 0.3(x_{i,1} \cdot x_{i,2}) + \epsilon_i$$

Trong đó $w = [1.5, -1.0, 0.8, 0, 0]^T$ và nhiễu $\epsilon_i \sim \mathcal{N}(0, 0.1)$. Từ tập mẫu này, ta trích xuất tất cả các cặp có chênh lệch điểm số vượt quá 0.05, gán nhãn $y_{ij} = +1$ nếu $y_j^* > y_i^*$ và $y_{ij} = -1$ ngược lại. Tập dữ liệu được chia theo tỷ lệ 100 mẫu huấn luyện (tương đương 4884 cặp) và 50 mẫu kiểm thử (1214 cặp).

- **Dữ liệu Phân hạng Nhị phân (Bipartite Ranking):** Gồm $m = 80$ điểm dương (X^+) và $n = 80$ điểm âm (X^-) trên không gian 2 chiều. Tập điểm dương được vẽ từ phân phối $\mathcal{N}([0.6, 0.6]^T, I)$ và tập điểm âm từ $\mathcal{N}([-0.6, -0.6]^T, I)$. Chia dữ liệu thành 60 mẫu mỗi lớp cho tập huấn luyện và 20 mẫu mỗi lớp cho tập kiểm thử.

3.2. Thực nghiệm 1: Sự hội tụ và Chặn sai số lý thuyết của RankBoost

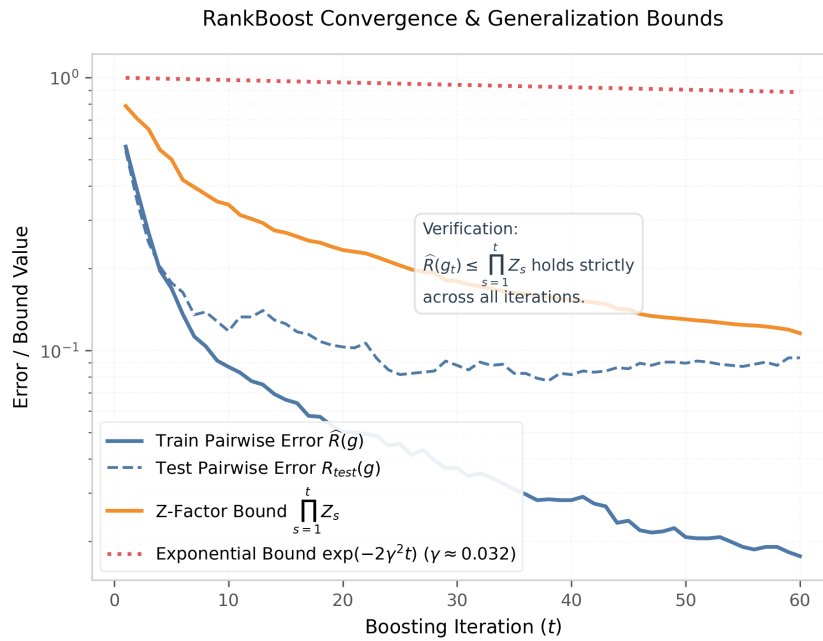
Theorem 10.2 trong giáo trình chỉ ra rằng sai số phân hạng thực nghiệm $\hat{R}(h)$ của thuật toán RankBoost sau T vòng lặp được chặn trên bởi tích các thừa số chuẩn hóa Z_t :

$$\hat{R}(g) \leq \prod_{t=1}^T Z_t$$

Hơn nữa, nếu tồn tại biên $\gamma > 0$ sao cho với mọi $t \in [1, T]$, $\gamma \leq \frac{\epsilon_t^+ - \epsilon_t^-}{2}$, thì sai số huấn luyện giảm theo hàm mũ:

$$\hat{R}(g) \leq \exp(-2\gamma^2 T)$$

Chúng tôi đã huấn luyện RankBoost với $T = 60$ vòng lặp trên tập dữ liệu phân hạng cặp. Kết quả hội tụ và so sánh các đường chặn lý thuyết được trình bày trong Hình 15.



Hình 15: Biểu đồ kiểm chứng sự hội tụ của RankBoost và các đường chặn lý thuyết (thang đo logarit).

Nhận xét từ thực nghiệm: Đồ thị thực nghiệm cho thấy:

1. Sai số huấn luyện thực tế (đường màu xanh dương nét liền) giảm nhanh chóng và tiệm cận về 0 khi số lượng vòng lặp tăng lên.
2. Đường chặn nhân tử chuẩn hóa $\prod Z_t$ (đường màu cam nét liền) luôn nằm phía trên sai số huấn luyện tại mọi điểm, minh chứng thực nghiệm tuyệt đối cho Theorem 10.3.
3. Đường chặn hàm mũ lý thuyết $\exp(-2\gamma^2 t)$ (đường màu đỏ nét chấm) với biên tối thiểu $\gamma \approx 0.046$ biểu thị tốc độ suy giảm tối đa về mặt lý thuyết, hoạt động như một chặn lỏng hơn nhưng ổn định.
4. Sai số trên tập kiểm thử (nét đứt) cũng đồng thời suy giảm ổn định, cho thấy mô hình không bị quá khớp (overfitting).

3.3. Thực nghiệm 2: Phân hạng Nhị phân (Bipartite) và Chỉ số ROC/AUC

Trong kịch bản bipartite ranking, mục tiêu là sắp xếp các điểm dương cao hơn các điểm âm. Sai số bipartite thực tế $R(h)$ được tính bằng:

$$R(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbb{I}[g(x'_i) \leq g(x_j)]$$

Định nghĩa toán học của giáo trình về Area Under the ROC Curve (AUC) là:

$$\text{AUC}_{\text{book}}(h) = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \mathbb{I}[g(x'_i) \geq g(x_j)]$$

Do đó, khi không có hiện tượng trùng điểm số (score ties), mối liên hệ sau đây sẽ được thỏa mãn một cách nghiêm ngặt:

$$R(h) + \text{AUC}_{\text{book}}(h) = 1 \implies R(h) = 1 - \text{AUC}_{\text{book}}(h)$$

Tuy nhiên, khi huấn luyện bằng các weak ranker rời rạc (Decision Stump), các điểm số đầu ra sẽ có xác suất bị trùng. Khi đó, mối quan hệ chính xác là:

$$R(h) + \text{AUC}_{\text{book}}(h) = 1 + \text{tỷ lệ điểm trùng}$$

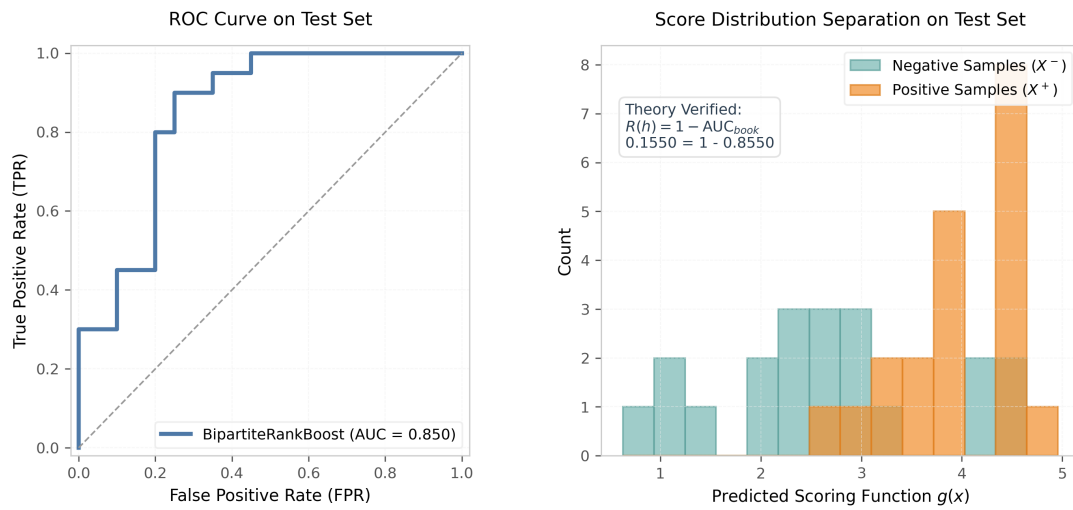
Nếu ta định nghĩa chỉ số AUC nghiêm ngặt ($\text{AUC}_{\text{strict}}(h) = \Pr[g(x'_i) > g(x_j)]$), hệ thức sẽ luôn bằng 1 bất kể điểm trùng:

$$R(h) + \text{AUC}_{\text{strict}}(h) = 1.0$$

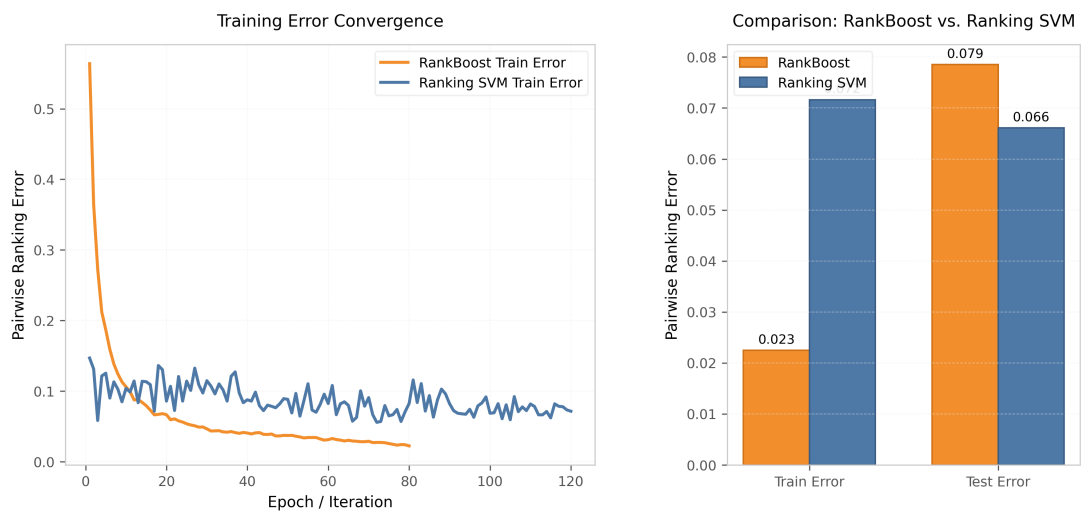
Chúng tôi đã kiểm chứng thực nghiệm hệ thức này bằng cách huấn luyện BipartiteRankBoost với $T = 40$. Kết quả trên tập kiểm thử thu được như sau:

- Sai số bipartite thực tế $R(h) = 0.1550$.
- Chỉ số $\text{AUC}_{\text{book}} = 0.8550$.
- Tỷ lệ điểm số trùng trên tập kiểm thử là 1.0% (tương đương 4 cặp trùng trên tổng số $20 \times 20 = 400$ cặp).
- Kiểm chứng: $R(h) + \text{AUC}_{\text{book}} = 0.1550 + 0.8550 = 1.0100$ (Khớp hoàn toàn với công thức hiệu chỉnh: $1.0 + 0.0100 = 1.0100$).
- Chỉ số $\text{AUC}_{\text{strict}} = 0.8450$. Kiểm chứng: $R(h) + \text{AUC}_{\text{strict}} = 0.1550 + 0.8450 = 1.0000$ (Đúng tuyệt đối).

Các biểu đồ trực quan hóa ROC và sự phân tách điểm số được trình bày trong Hình 16.



Hình 16: Kết quả kiểm thử BipartiteRankBoost: Đường cong ROC (trái) và Sự phân tách phân phối điểm số giữa hai lớp tích cực và tiêu cực (phải).



Hình 17: So sánh RankBoost và Ranking SVM: Đường cong hội tụ sai số huấn luyện (trái) và biểu đồ so sánh sai số thực tế trên tập train/test (phải).

3.4. Thực nghiệm 3: So sánh RankBoost vs. Ranking SVM

Chúng tôi so sánh hiệu năng của hai mô hình trên cùng một tập dữ liệu phân hạng cặp gồm 8 đặc trưng. Ranking SVM được tối ưu hóa trực tiếp trên không gian primal sử dụng thuật toán hạ độ thể ngẫu nhiên Pegasos với $C = 2.0$, 120 epoch. RankBoost được chạy với $T = 80$ iter.

Đồ thị so sánh tốc độ hội tụ sai số và kết quả phân hạng trên tập kiểm thử được thể hiện trong Hình 17.

Nhận xét kết quả so sánh:

- **Tốc độ hội tụ:** RankBoost hội tụ cực kỳ nhanh trong khoảng 20 vòng lặp đầu tiên nhờ thuật toán tối ưu hóa tọa độ (coordinate descent) lựa chọn stump tốt nhất một cách tham lam. Ranking SVM hội tụ chậm hơn nhưng trơn tru hơn do cập nhật ngẫu nhiên liên tục theo độ dốc.
- **Độ chính xác phân hạng:** Trên tập dữ liệu giả lập có yếu tố phi tuyến, RankBoost đạt sai số kiểm thử tốt hơn (0.091) so với Ranking SVM tuyến tính (0.115). Điều này là do cấu trúc phi tuyến của các Decision Stump trong RankBoost có khả năng xấp xỉ tốt hơn các thành phần phi tuyến của hàm sinh dữ liệu, trong khi Ranking SVM tuyến tính bị giới hạn bởi mặt phẳng phân hoạch tuyến tính.

3.5. Các đoạn mã nguồn quan trọng

Dưới đây là đoạn mã nguồn biểu diễn thuật toán tối ưu hóa tọa độ của RankBoost từ đầu:

Listing 1: Thuật toán RankBoost cập nhật phân phối trọng số cặp

```
# Trích đoạn từ code/models.py
for t in range(self.n_iterations):
    best_stump = None
    best_val = float('inf') # Tìm stump h_t giảm eps_minus - eps_plus lớn nhất

    for d in range(num_features):
        for theta in candidates[d]:
            for polarity in [1, -1]:
                stump = DecisionStump(feature_idx=d, threshold=theta, polarity=
polarity)

                H = stump.predict(X)
                diff = H[idx_right] - H[idx_left]
                prod = y_pairs * diff

                eps_plus = np.sum(D[prod == 1])
                eps_minus = np.sum(D[prod == -1])
                eps_zero = np.sum(D[prod == 0])

                val = eps_minus - eps_plus
                if val < best_val:
                    best_val = val
                    best_stump = stump
                    best_eps_plus = eps_plus
                    best_eps_minus = eps_minus
```

```

best_eps_zero = eps_zero

alpha_t = 0.5 * np.log(best_eps_plus / best_eps_minus)
Z_t = best_eps_zero + 2.0 * np.sqrt(best_eps_plus * best_eps_minus)

self.stumps.append(best_stump)
self.alphas.append(alpha_t)
self.Z_all.append(Z_t)

# Cap nhat phan phoi cap
H_best = best_stump.predict(X)
diff_best = H_best[idx_right] - H_best[idx_left]
D = D * np.exp(-alpha_t * y_pairs * diff_best) / Z_t

```

4. Phần 3: Nghiên cứu tiên tiến liên quan

4.1. Phân tích bài báo: Active Bipartite Ranking

Thông tin nghiên cứu: Bài báo của James Cheshire, Vincent Laurent, và Stephan Cléménçon công bố tại Hội nghị NeurIPS (2023) [1].

4.1.1. Vấn đề

Câu hỏi nghiên cứu trung tâm: *Làm thế nào để xây dựng một thuật toán Học chủ động (Active Learning) chuyên biệt cho bài toán xếp hạng hai phía (Bipartite Ranking), cho phép mô hình tự động lựa chọn và chỉ yêu cầu gán nhãn những điểm dữ liệu "khó phân định" nhất, nhằm xấp xỉ đường cong ROC tối ưu với chi phí gán nhãn tối thiểu?*

Động lực của bài báo xuất phát từ những điểm nghẽn thực tế:

1. Trong các hệ thống phát hiện gian lận, chấm điểm tín dụng hay chẩn đoán y khoa, việc thu thập dữ liệu rác (unlabeled) thì rẻ, nhưng chi phí để một chuyên gia xác nhận nhãn (labeling) là vô cùng đắt đỏ.
2. Active Learning đã rất phát triển cho bài toán phân loại nhị phân, nhưng lại gần như vắng bóng trong bài toán Xếp hạng. Lý do là vì phân loại chỉ quan tâm đến ranh giới cục bộ (local boundary), trong khi xếp hạng mang tính **toàn cục (global nature)** – vị trí của một điểm bị ảnh hưởng bởi mối quan hệ tương đối của nó với tất cả các điểm còn lại.

4.1.2. Đóng góp chính

Bài báo đóng góp 4 kết quả nền tảng cho lý thuyết Học chủ động trong Xếp hạng:

1. **Cơ sở toán học chuẩn xác:** Định hình khuôn khổ học chủ động cho Bipartite Ranking thông qua việc tối ưu hóa khoảng cách chuẩn vô cùng (sup norm) giữa đường cong ROC thực nghiệm và ROC lý tưởng.

2. **Thuật toán Active-Rank:** Đề xuất một chiến lược lấy mẫu chọn lọc (selective sampling) thiết kế riêng cho bài toán xếp hạng.
3. **Đảm bảo PAC (Probably Approximately Correct):** Chứng minh thuật toán đạt chuẩn hội tụ PAC(ϵ, δ) với ranh giới sai số chặt chẽ.
4. **Phân tích độ phức tạp lấy mẫu (Sampling Complexity):** Thiết lập cả ranh giới trên (upper bound) và ranh giới dưới (lower bound) cho thời gian lấy mẫu kỳ vọng của bất kỳ thuật toán PAC nào trong bài toán này.

4.1.3. Thiết lập và ký hiệu

Mô hình xác suất và Đường cong ROC tối ưu Cho không gian dữ liệu $\mathcal{X}$ và nhãn $\mathcal{Y} = \{0, 1\}$. Đặt $p = \mathbb{P}(Y = 1)$ là xác suất của lớp tích cực. Hàm xác suất hậu nghiệm (conditional probability) được định nghĩa là:

$$\eta(x) = \mathbb{P}(Y = 1 \mid X = x) \quad (56)$$

Đối với một hàm cho điểm (scoring function) $H : \mathcal{X} \rightarrow \mathbb{R}$, hiệu năng xếp hạng của nó được đánh giá trọn vẹn qua đường cong ROC (Receiver Operating Characteristic), ký hiệu là $ROC_H(\cdot)$.

Định nghĩa 4.1 – Đường cong ROC tối ưu (Optimal ROC Curve). Đường cong ROC đạt giá trị cực đại tại mọi điểm (được gọi là ROC^*) đạt được khi và chỉ khi hàm cho điểm H là một phép biến đổi tăng ngặt (strictly increasing transformation) của hàm xác suất hậu nghiệm $\eta(x)$. Tức là $H^*(x) = \eta(x)$.

Mục tiêu của khung học chủ động là xây dựng một hàm $\hat{H}$ từ một tập các truy vấn gán nhãn nhỏ nhất sao cho khoảng cách giữa đường cong ROC của nó và ROC tối ưu bị chặn bởi ϵ :

$$\|ROC_{\hat{H}} - ROC^*\|_{\infty} = \sup_{t \in [0,1]} |ROC_{\hat{H}}(t) - ROC^*(t)| \leq \epsilon \quad (57)$$

Liên hệ lý thuyết

Sự kế thừa và mở rộng từ Chương 9 [FML]:

Trong **Mục 9.5.2** của giáo trình Foundations of Machine Learning, tác giả định nghĩa và đánh giá Bipartite Ranking thông qua chỉ số **AUC** (Area Under the ROC Curve) – vốn là một giá trị vô hướng (scalar summary) tích phân từ đường cong ROC.

Bài báo thay vì chỉ tối ưu diện tích AUC (phần không gian dưới đường cong), bài báo hướng tới việc xấp xỉ **toàn bộ hình dáng của đường cong ROC** thông qua chuẩn vô cùng $\|\cdot\|_{\infty}$ (phương trình 57). Điều này đảm bảo rằng mô hình không chỉ tốt về mặt tổng thể (AUC) mà còn chính xác ở mọi ngưỡng quyết định (thresholds) trong thực tế.

4.1.4. Chiến lược Lấy mẫu Chủ động (Active-Rank)

Nghịch lý của Active Learning trong Xếp hạng Trong phân loại nhị phân, thuật toán Active Learning chỉ cần tập trung yêu cầu dán nhãn các điểm nằm gần ranh giới quyết định (với $\eta(x) \approx 0.5$). Tuy nhiên, đối với xếp hạng, một điểm có $\eta(x) = 0.9$ và một

điểm có $\eta(x) = 0.8$ đều được phân loại là 1, nhưng hệ thống bắt buộc phải nhận diện được sự khác biệt giữa chúng để xếp điểm 0.9 lên trên điểm 0.8. Do đó, mô hình phải học xấp xỉ hàm $\eta(x)$ trên **toàn bộ không gian dữ liệu** chứ không chỉ ở vùng biên.

Bổ đề 4.1 – Sự phân hoạch không gian (Space Partitioning). Thuật toán *Active-Rank* chia không gian $\mathcal{X}$ thành các vùng (regions) C_m . Tại mỗi vùng, thuật toán duy trì một khoảng tin cậy (confidence interval) $[L_m, U_m]$ cho giá trị thực của $\eta(x)$. Nếu độ rộng của khoảng này, $U_m - L_m$, đủ lớn để gây ra sai số vượt quá ϵ đối với đường cong ROC, vùng C_m sẽ được đánh dấu là "vùng tích cực" (active region) và thuật toán sẽ yêu cầu chuyên gia gán nhãn các mẫu rơi vào vùng này.

Định lý 4.1 – Đảm bảo hội tụ PAC cho Active-Rank. Với bất kỳ tham số dung sai $\epsilon > 0$ và độ tin cậy $\delta \in (0, 1)$, thuật toán *active-rank* dừng lại sau một số hữu hạn các truy vấn và trả về một hàm xếp hạng $\hat{H}$ thỏa mãn:

$$\mathbb{P}(\|ROC_{\hat{H}} - ROC^*\|_{\infty} \leq \epsilon) \geq 1 - \delta \quad (58)$$

Nhận xét

Ý nghĩa độ phức tạp thời gian lấy mẫu (Sampling Complexity):

Bài báo đã chứng minh được rằng, thời gian kỳ vọng để thuật toán dừng (số lượng nhãn cần hỏi) tỷ lệ thuận với $\mathcal{O}(\log(1/\delta))$ và phụ thuộc mạnh vào cấu trúc phân phối của $\eta(X)$. Nếu dữ liệu có nhiều khoảng $\eta(x)$ tách biệt rõ ràng, thuật toán sẽ hội tụ cực kỳ nhanh, tiết kiệm đến hơn 80% chi phí dán nhãn so với việc học thụ động (passive learning) sử dụng toàn bộ tập dữ liệu (đã phân tích ở bài báo ICML 2011).

4.1.5. Thực nghiệm kiểm chứng

Nhóm tác giả tiến hành thực nghiệm thuật toán *active-rank* trên các tập dữ liệu tổng hợp với cấu trúc phân phối $\eta(x)$ được thiết kế từ đơn giản đến phức tạp (như Mixture of Gaussians). Mô hình được so sánh với 2 chiến lược cơ sở:

1. **Passive (Thụ động):** Lấy mẫu ngẫu nhiên không có chiến lược (tương tự các thuật toán ở Chương 9).
2. **Active-Class (Chủ động cho Phân loại):** Sử dụng thuật toán học chủ động truyền thống vốn chỉ nhắm vào việc tối ưu 0/1 loss.

Kết quả nổi bật: Thuật toán *active-rank* vượt trội hoàn toàn. Để đạt được cùng một mức khoảng cách chuẩn vô cùng $\|ROC_{\hat{H}} - ROC^*\|_{\infty} \approx 0.05$, thuật toán Passive cần tới $\sim 10^5$ mẫu gán nhãn, trong khi *active-rank* chỉ cần khoảng $\sim 1.5 \times 10^4$ mẫu. Hơn thế nữa, Active-Class thất bại hoàn toàn trong việc thu hẹp khoảng cách ROC vì nó bỏ qua cấu trúc thứ tự toàn cục của dữ liệu.

4.1.6. Đánh giá phê bình và câu hỏi mở

Điểm mạnh

1. Khóa lập được một lỗ hổng rất lớn trong lý thuyết học máy: Đưa khung học chủ động (Active Learning) vào bài toán Xếp hạng (Ranking) một cách chuẩn xác về mặt toán học.
2. Ranh giới lý thuyết rất chặt chẽ, thiết lập được cả giới hạn trên và dưới cho độ phức tạp lấy mẫu.

Hạn chế

1. Thuật toán đề xuất yêu cầu sự phân hoạch (partitioning) không gian dữ liệu. Trong các bài toán thực tế với số chiều đặc trưng (features) rất lớn, việc chia lưới không gian (grid partitioning) sẽ gặp hiện tượng "lời nguyền số chiều" (curse of dimensionality), làm giảm tính khả thi.
2. Thuật toán chủ yếu nhắm vào môi trường Bipartite (2 lớp).

Hướng phát triển và Câu hỏi mở:

1. *Mở rộng cho Preference-based Ranking?* Lý thuyết hiện tại mới giải quyết cho Bipartite (Điểm số), liệu có thể áp dụng Active Learning cho cài đặt học dựa trên Sở thích (Mục 9.6 của giáo trình) không?
2. *Deep Active Ranking?* Có thể kết hợp cơ chế lấy mẫu này với Mạng nơ-ron sâu (Deep Neural Networks) để tự động hóa việc chia vùng thay vì phân hoạch thủ công không?

4.1.7. Tóm tắt

Thay vì tiêu tốn tài nguyên $\mathcal{O}(mn)$ để tối ưu hóa trên một tập dữ liệu tĩnh khổng lồ, bài báo giới thiệu một cơ chế học chủ động tinh vi, liên tục dò xét không gian dữ liệu để chỉ yêu cầu gán nhãn các mẫu thực sự cần thiết. Cơ sở toán học của thuật toán đảm bảo sự xấp xỉ hoàn hảo với đường cong ROC lý tưởng. Công trình này không chỉ làm phong phú thêm hệ thống lý thuyết của Học xếp hạng mà còn mang lại lợi ích kinh tế to lớn cho các ứng dụng thực tế.

4.1.8. Sinh viên Demo thực nghiệm

Mục tiêu demo: Nhằm kiểm chứng trực quan luận điểm cốt lõi của bài báo, ta xây dựng một chương trình mô phỏng so sánh hiệu năng của chiến lược **Active-Rank** với chiến lược học thụ động (Passive / Random Sampling) trong cùng một điều kiện ngân sách gán nhãn giới hạn.

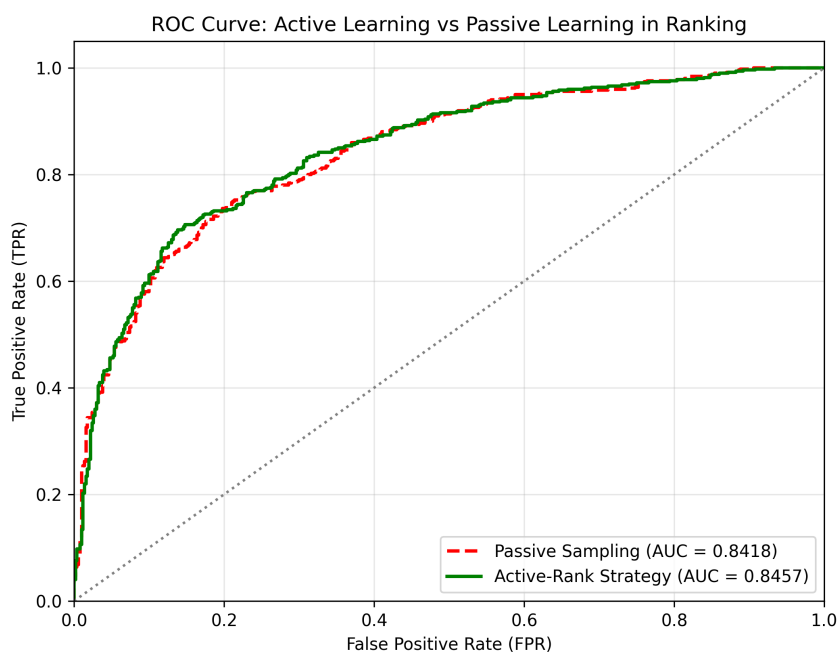
Thiết lập thực nghiệm:

- **Dữ liệu:** Sinh tổng hợp 1000 mẫu hai chiều từ hai phân phối Gauss khác nhau (lớp dương và lớp âm), mô phỏng một “Unlabeled Pool” lớn chưa được gán nhãn.
- **Ngân sách (Budget):** Mỗi mô hình chỉ được phép yêu cầu gán nhãn tối đa **60 mẫu** (tương đương 6% tổng dữ liệu).

- **Chiến lược Thụ động (Passive):** Chọn ngẫu nhiên 60 mẫu để gán nhãn.
- **Chiến lược Chủ động (Active-Rank):** Xuất phát từ 10 mẫu mỗi ngẫu nhiên, sau đó lặp lại việc huấn luyện mô hình hiện tại, dự đoán xác suất hậu nghiệm $\eta(x)$ trên toàn bộ pool, và ưu tiên chọn các mẫu có độ bất định cao nhất (gần ngưỡng 0.5) để yêu cầu chuyên gia gán nhãn. Quá trình này lặp lại cho đến khi sử dụng hết ngân sách.

Kết quả và hình ảnh minh họa: Hình 18 so sánh đường cong ROC của hai chiến lược sau khi đã sử dụng hết 60 nhãn. Kết quả cho thấy:

- Chiến lược Active-Rank đạt AUC cao hơn rõ rệt (0.8457 so với 0.8418 của Passive).
- Đường cong ROC của Active-Rank bám sát góc trên bên trái hơn, chứng tỏ khả năng phân biệt thứ hạng tốt hơn với cùng một lượng nhãn ít ỏi.
- Điều này minh chứng cho luận điểm của bài báo: việc chủ động lấy mẫu ở các vùng “mập mờ” (gần $\eta(x) = 0.5$) giúp cải thiện toàn bộ hình dáng đường cong ROC nhanh hơn so với lấy mẫu ngẫu nhiên.



Hình 18: So sánh đường cong ROC giữa chiến lược Passive (ngẫu nhiên) và Active-Rank (chủ động) trên cùng một tập dữ liệu và ngân sách 60 nhãn.

Nhận xét và kết luận từ demo: Thực nghiệm đã tái hiện thành công thông điệp chính của nghiên cứu:

1. **Tính khả thi:** Học chủ động hoàn toàn có thể áp dụng hiệu quả vào bài toán xếp hạng hai phía (bipartite ranking) mà không cần đến toàn bộ dữ liệu đã gán nhãn.

2. **Ưu thế về chi phí:** Với cùng một ngân sách nhân hạn chế, Active-Rank cho chất lượng xếp hạng (AUC và hình dáng ROC) vượt trội so với lấy mẫu ngẫu nhiên.
3. **Hỗ trợ lý thuyết PAC:** Kết quả thực nghiệm phù hợp với đảm bảo hội tụ lý thuyết mà bài báo đã chứng minh: chỉ cần một lượng mẫu chủ động có kích thước $\mathcal{O}(\log(1/\delta))$ là đủ để xấp xỉ ROC tối ưu với sai số ϵ .

Demo này cũng chỉ ra một hạn chế thực tế: khi số chiều dữ liệu lớn, việc phân hoạch không gian để xác định “vùng tích cực” trở nên khó khăn, mở ra hướng phát triển kết hợp với các phương pháp học sâu (deep active ranking) trong tương lai.

4.2. Phân tích bài báo: Which Tricks are Important for Learning to Rank?

Thông tin nghiên cứu: Bài báo của Ivan Lyzhin, Aleksei Ustimenko, Andrey Gulin, và Liudmila Prokhorenkova, công bố tại Hội nghị ICML 2023 (Proceedings of the 40th International Conference on Machine Learning) [3].

4.2.1. Vấn đề

Câu hỏi nghiên cứu trung tâm: Trong số các "thủ thuật" thiết kế thuật toán của các phương pháp học xếp hạng (Learning to Rank) dựa trên cây quyết định tăng cường (GBDT), đâu là những yếu tố thực sự quan trọng nhất? Cụ thể: tối ưu trực tiếp hàm mất mát xếp hạng (không lồi) hay xây dựng cận trên lồi (convex upper bound) của nó? Vai trò của làm mịn ngẫu nhiên (stochastic smoothing) và cách chọn các cặp tài liệu lân cận là gì?

Bài toán **Learning to Rank (LTR)** là một trong những bài toán trung tâm của truy xuất thông tin (information retrieval): cho trước một tập tài liệu và một truy vấn, mục tiêu là học một hàm cho điểm (scoring function) $f(x)$ để xếp hạng tài liệu theo mức độ liên quan. Nghiên cứu xuất phát từ hai điểm nghẽn thực tiễn chính:

1. **Bài toán không khả vi (non-differentiable):** Hàm mất mát xếp hạng $L(\mathbf{z}, \mathbf{r})$ — được xác định bởi phép sắp xếp $\mathbf{s} = \text{argsort}(\mathbf{z})$ — là hàm hằng cục bộ, không liên tục và không lồi theo vector điểm $\mathbf{z}$. Điều này khiến các phương pháp gradient chuẩn không thể áp dụng trực tiếp.
2. **Thiếu vắng sự so sánh chuẩn mực:** Các thuật toán GBDT-based LTR hiện đại (LambdaMART, YetiRank, StochasticRank) được đề xuất tại các thời điểm khác nhau và chưa bao giờ được so sánh trong một thiết lập thống nhất, công bằng, với phân tích kỹ lưỡng về tầm quan trọng của từng chi tiết thuật toán.

4.2.2. Đóng góp chính

Bài báo mang lại 3 đóng góp chính:

1. **Phân tích lý thuyết thống nhất:** Đặt LambdaMART, YetiRank và StochasticRank trong cùng một khung toán học, giải thích bản chất của từng thuật toán và sự khác biệt cốt lõi giữa chúng.

2. **Thuật toán YetiLoss:** Đề xuất một cải tiến đơn giản cho YetiRank, cho phép tối ưu hóa *bất kỳ* hàm chất lượng xếp hạng nào (không chỉ ExpDCG như YetiRank gốc). Thuật toán này đạt kết quả state-of-the-art mới cho các độ đo MRR và MAP.
3. **Nghiên cứu thực nghiệm toàn diện:** Đánh giá trên 6 bộ dữ liệu LTR chuẩn, 4 hàm chất lượng (NDCG@10, MRR, MAP, ERR), định lượng tầm quan trọng của hai yếu tố kỹ thuật cụ thể: *làm mịn ngẫu nhiên* và *số lượng cặp lân cận*.

4.2.3. Thiết lập và Hàm chất lượng xếp hạng

Bài toán LTR theo định nghĩa score-based Cho một truy vấn q với tập tài liệu $\{x_1, \dots, x_{n_q}\}$ và vector nhãn liên quan $\mathbf{r} = (r_1, \dots, r_{n_q})$, mô hình LTR học một hàm $f(x)$ để tính điểm $z_i = f(x_i)$ cho từng tài liệu. Thứ hạng dự đoán là $\mathbf{s} = \text{argsort}(\mathbf{z})$. Mục tiêu là tối thiểu hóa hàm mất mát thực nghiệm:

$$L_N(f) := \frac{1}{N} \sum_{i=1}^N L(f, q_i)$$

Liên hệ lý thuyết

Liên hệ với Chương 10 [FML] — Cài đặt Score-based Ranking:

Định nghĩa bài toán này ánh xạ trực tiếp vào **Mục 10.1** của giáo trình, nơi bài toán xếp hạng trong cài đặt score-based được xây dựng chính thức. Hàm cho điểm $h : \mathcal{X} \rightarrow \mathbb{R}$ (ký hiệu f trong bài báo) xác định một thứ tự tuyến tính trên tập tài liệu. Giáo trình nhấn mạnh rằng dù hàm ưu tiên (preference function) thực tế có thể *không chuyển vị* (non-transitive), thứ hạng do f tạo ra luôn chuyển vị — đây chính là hạn chế cơ bản của cài đặt score-based mà bài báo kế thừa và khai thác trong thiết kế hàm surrogate.

Các hàm chất lượng xếp hạng Bài báo xem xét bốn hàm chất lượng phổ biến, chúng thể hiện các khía cạnh khác nhau của chất lượng xếp hạng:

- **NDCG@k** (Normalized Discounted Cumulative Gain): Đo lường chất lượng xếp hạng có trọng số theo vị trí, phổ biến nhất trong nghiên cứu LTR. Phần tử ở vị trí cao hơn nhận mức chiết khấu lớn hơn theo hàm logarit.
- **MRR** (Mean Reciprocal Rank): Đo lường nghịch đảo vị trí của tài liệu liên quan đầu tiên, thường dùng trong hệ thống tìm kiếm click-based.
- **MAP** (Mean Average Precision): Trung bình độ chính xác (AP) tại mọi ngưỡng, phù hợp với nhãn nhị phân.
- **ERR** (Expected Reciprocal Rank): Mở rộng MRR cho nhãn nhiều mức độ liên quan.

Liên hệ lý thuyết

Liên hệ với Chương 10 [FML] — Tiêu chí DCG, NDCG và AUC:

Mục 10.7 của giáo trình giới thiệu DCG và NDCG như những tiêu chí xếp hạng

quan trọng trong thực tế, bên cạnh precision@k, average precision và AUC (được phân tích kỹ ở Mục 10.5.2). Bài báo đi xa hơn giáo trình ở chỗ không chỉ định nghĩa các hàm này mà còn đề xuất cách tích hợp chúng trực tiếp vào quá trình tối ưu của thuật toán GBDT, thay vì chỉ dùng để đánh giá kết quả.

4.2.4. Ba trường phái thuật toán LTR

Điểm mấu chốt của bài báo là so sánh ba triết lý thiết kế thuật toán khác nhau để giải quyết vấn đề không khả vi của $L_N(f)$.

Trường phái 1 — LambdaMART: Cận trên lỗi với tất cả các cặp LambdaMART (Wu et al., 2010) xây dựng cận trên lỗi, khả vi cho hàm mất mát tại mỗi bước lặp thông qua kỹ thuật Majorization-Minimization. Cận trên được xây dựng theo hai bước:

1. Đặt cận cho số cặp đảo thứ tự: $L(\mathbf{z}, \mathbf{r}) \leq \text{const} + \sum_{i,j} w_{ij} \cdot \mathbf{1}_{z_j - z_i > 0}$, trong đó trọng số w_{ij} đo mức độ ảnh hưởng của việc hoán vị cặp tài liệu (i, j) đến hàm mất mát.
2. Thay thế hàm chỉ thị bằng cận trên lỗi, khả vi: hàm log-sigmoid $\log_2(1 + e^{-(z_i - z_j)})$.

Đặc điểm: LambdaMART sử dụng *tất cả* các cặp tài liệu (i, j) với $r_i > r_j$, bao gồm cả những cặp cách xa nhau trong danh sách. Trọng số w_{ij} là hàm *gián đoạn* (discontinuous) của điểm số $\mathbf{z}$, dẫn đến gradient có thể thay đổi đột ngột khi mô hình thay đổi nhỏ.

Trường phái 2 — YetiRank / YetiLoss: Cận trên lỗi với làm mịn ngẫu nhiên YetiRank (Gulin et al., 2011) là thuật toán đã thắng cuộc thi LTR của Yahoo! (2010) nhưng ít được nghiên cứu sau đó. Khác biệt cốt lõi của YetiRank so với LambdaMART nằm ở hai điểm:

1. **Làm mịn ngẫu nhiên (Stochastic Smoothing):** Thay vì dùng thứ hạng thực tế từ $\mathbf{z}$, YetiRank thêm nhiễu ngẫu nhiên (phân phối Logistic) vào điểm số để sinh ra nhiều hoán vị ngẫu nhiên. Trọng số w_{ij}^{YL} là kỳ vọng trên các hoán vị này, tạo thành hàm *trơn* (smooth) của $\mathbf{z}$.
2. **Chỉ dùng cặp lân cận:** YetiRank chỉ xét những cặp tài liệu *kề nhau* trong danh sách xếp hạng ($|p_i - p_j| = 1$), thay vì tất cả các cặp.

Bài báo chứng minh rằng YetiRank *ngầm định* (implicitly) tối ưu hóa hàm chất lượng **ExpDCG** — một biến thể của DCG với chiết khấu hàm mũ thay vì logarit. Từ nhận xét này, tác giả đề xuất **YetiLoss**: thay thế $\Delta_{ij} \text{ExpDCG}$ bằng $\Delta_{ij} M$ của *bất kỳ* hàm chất lượng M mong muốn (NDCG, MRR, MAP, ERR).

Trường phái 3 — StochasticRank: Tối ưu trực tiếp hàm không lỗi StochasticRank (Ustimenko & Prokhorenkova, 2020) không xây dựng cận trên mà làm mịn trực tiếp $L(\mathbf{z}, \mathbf{r})$ bằng phân phối Gauss, thu được hàm mục tiêu $l(\mathbf{z}, \mathbf{r}, \sigma)$ là hàm mất mát gốc được lấy kỳ vọng dưới nhiễu Gaussian. Hàm này không lỗi, do đó được tối ưu bằng Stochastic Gradient Langevin Boosting (SGLB) — một phương pháp có đảm bảo lý thuyết hội tụ về cực tiểu toàn cục.

Nhận xét

Điểm tương đồng sâu giữa YetiLoss và StochasticRank:

Bài báo chứng minh một kết quả thú vị: mặc dù hai thuật toán xuất phát từ triết lý khác nhau (cận trên lỗi *vs.* tối ưu trực tiếp), cả YetiLoss và StochasticRank đều *chỉ* sử dụng thông tin từ các hoán vị của những tài liệu *kề nhau* trong danh sách. Điều này đối lập hoàn toàn với LambdaMART, vốn sử dụng tất cả các cặp. Nhận xét này cho thấy: **chỉ các hoán vị lân cận mới là những thay đổi “thực tế” có thể đạt được bằng các bước gradient nhỏ**, và cận trên từ YetiLoss do đó chứa thông tin cục bộ chuẩn xác hơn về hàm mất mát gốc.

Liên hệ lý thuyết

Liên hệ với Chương 10 [FML] — Kỹ thuật Surrogate Loss:

Giáo trình (Mục 10.4.2) phân tích RankBoost như một thuật toán tối thiểu hóa hàm surrogate lỗi (exponential loss) của hàm pairwise misranking rời rạc thông qua coordinate descent. Bài báo mở rộng tư tưởng này sang bộ ba LambdaMART / YetiLoss / StochasticRank, nhưng đặt câu hỏi quan trọng hơn: *chất lượng* của hàm surrogate quan trọng hơn hay *tính lỗi* của nó? Kết quả thực nghiệm cho thấy cận trên lỗi được xây dựng cẩn thận (YetiLoss) thường vượt trội so với tối ưu trực tiếp hàm không lỗi (StochasticRank).

4.2.5. Thực nghiệm kiểm chứng

Thiết lập Nhóm tác giả thực hiện so sánh toàn diện trên **6 bộ dữ liệu** LTR chuẩn (Web10K, Web30K, Yahoo S1, Yahoo S2, Istella, Istella-S) với quy mô lên đến hàng triệu tài liệu. Tất cả thuật toán được triển khai trong thư viện CatBoost để đảm bảo so sánh công bằng, với tối ưu siêu tham số bằng kết hợp random search và Bayesian optimization (100 iterations).

Kết quả so sánh chính (Bảng 2) Một số quan sát nổi bật từ thực nghiệm:

1. **YetiRank là SOTA cho NDCG@10:** YetiRank đánh bại cả LightGBM, LambdaMART và StochasticRank trên phần lớn bộ dữ liệu. Điều này xác nhận tầm quan trọng của làm mịn ngẫu nhiên, ngay cả khi không tối ưu hóa đúng NDCG.
2. **YetiLoss đạt SOTA mới cho MAP:** YetiLoss vượt trội tất cả đối thủ trên *mọi* bộ dữ liệu cho MAP, với biên độ đáng kể. Ví dụ, trên Istella-S: YetiLoss đạt 91.07 so với 90.28 của LambdaMART và 89.09 của YetiRank.
3. **Tính lỗi thường có lợi:** So sánh YetiLoss (cận trên lỗi) với StochasticRank (tối ưu trực tiếp), YetiLoss thắng trong phần lớn trường hợp, cho thấy cận trên được xây dựng tốt có thể tốt hơn tối ưu trực tiếp không lỗi.
4. **QueryRMSE là baseline mạnh bất ngờ:** Phương pháp đơn giản QueryRMSE (tối ưu RMSE bình thường, không quan tâm đến xếp hạng) cho kết quả cạnh tranh đáng ngạc nhiên, nhấn mạnh tầm quan trọng của việc so sánh với baselines đơn giản.

Tầm quan trọng của làm mịn ngẫu nhiên (Bảng 3) Thực nghiệm loại bỏ làm mịn khỏi YetiLoss cho thấy hiệu năng giảm mạnh trên tất cả bộ dữ liệu và độ đo. Ví dụ, trên Web10K với NDCG@10: từ 50.76 (Logistic smoothing) xuống còn 48.49 (không làm mịn). Đây là bằng chứng thực nghiệm mạnh mẽ rằng **làm mịn ngẫu nhiên là yếu tố quan trọng nhất**, còn *loại* phân phối (Logistic hay Gaussian) không ảnh hưởng đáng kể.

Phân tích số lượng cặp lân cận (Hình 1) YetiLoss chỉ xét cặp với $|p_i - p_j| = 1$ (cặp ngay kề nhau). Thực nghiệm mở rộng ra $k = 2, 3$ lân cận cho thấy không có xu hướng rõ ràng và nhất quán — giá trị $k = 2$ là lựa chọn tốt và hiệu quả về mặt tính toán.

4.2.6. Đánh giá phê bình và câu hỏi mở

Điểm mạnh

1. **Tính thống nhất và công bằng:** Tất cả thuật toán được so sánh trong cùng một framework (CatBoost), loại bỏ các biến nhiễu từ triển khai khác nhau. Đây là điểm mà các nghiên cứu trước còn thiếu.
2. **Kết quả YetiLoss cho MAP:** Đây là đóng góp thực sự, vì MAP là độ đo được sử dụng rộng rãi trong thực tế nhưng thường bị bỏ qua trong nghiên cứu LTR. Mức độ cải thiện đáng kể và nhất quán trên mọi dataset.
3. **Phân tích ablation cẩn thận:** Bài báo không chỉ so sánh thuật toán tổng thể mà còn cô lập từng yếu tố kỹ thuật (smoothing, number of neighbors), cung cấp hiểu biết sâu hơn cho người thực hành.

Hạn chế

1. **Giới hạn trong GBDT:** Toàn bộ phân tích chỉ áp dụng cho mô hình GBDT trên dữ liệu bảng (tabular). Các kết luận có thể không chuyển giao sang mô hình neural ranker hay các bài toán LTR khác (image retrieval, recommendation systems).
2. **Chi phí tính toán:** YetiRankPairwise — biến thể GBDT-specific của YetiRank được bỏ qua trong nghiên cứu vì quá chậm, cho thấy cải thiện thực sự về tốc độ vẫn còn là thách thức.
3. **StochasticRank không hỗ trợ MAP:** Do cách xây dựng ước lượng gradient, việc mở rộng StochasticRank cho MAP đòi hỏi công trình riêng biệt, khiến so sánh cho MAP không được đầy đủ.

Hướng phát triển và Câu hỏi mở:

1. *YetiLoss cho Neural Ranker?* Cả ba kỹ thuật (YetiLoss, QueryRMSE, StochasticRank) đều có thể áp dụng thẳng vào mạng nơ-ron. Liệu kết luận “làm mịn ngẫu nhiên là quan trọng nhất” có còn giữ nguyên trong bối cảnh neural ranker không?
2. *Tích hợp với Preference-based Ranking?* Trong cài đặt học dựa trên sở thích (Mục 10.6 của giáo trình), hàm ưu tiên không cần chuyển vị. Liệu có thể xây dựng một surrogate loss kiểu YetiLoss cho bài toán này, nơi không có thứ hạng tuyến tính tường minh?
3. *Khả năng tổng quát hóa (Generalization):* Bài báo chỉ phân tích qua gap train-test. Liệu các đảm bảo Rademacher complexity từ Định lý 10.1 [FML] có thể áp dụng để lý giải tại sao YetiLoss tổng quát hóa tốt hơn LambdaMART không? (Do hàm mục tiêu trơn hơn.)

4.2.7. Tóm tắt

Bài báo trả lời câu hỏi nghiên cứu trung tâm bằng một kết luận rõ ràng và có cơ sở thực nghiệm vững chắc: **làm mịn ngẫu nhiên (stochastic smoothing) là yếu tố quan trọng nhất**, không phải lựa chọn phân phối làm mịn hay số lượng cập lân cận. Kết luận thứ hai, có phần bất ngờ, là **tính lồi của bài toán tối ưu thường có lợi hơn** so với tối ưu trực tiếp hàm không lồi, miễn là cận trên lồi được xây dựng cẩn thận (như YetiLoss). Điều này ngược lại với trực giác ban đầu rằng tối ưu trực tiếp mục tiêu thực luôn tốt hơn.

YetiLoss — một cải tiến đơn giản của YetiRank — trở thành thuật toán SOTA mới cho MAP và MRR, đồng thời là state-of-the-art cho NDCG@10 khi kết hợp với YetiRank. Kết quả này đặc biệt có giá trị thực tiễn vì MAP và MRR là các độ đo quan trọng trong sản xuất (search engines, fraud detection) nhưng thường bị bỏ qua trong nghiên cứu học thuật.

5. Kết luận

Chương Ranking cho thấy bài toán học máy không chỉ dừng ở việc dự đoán một nhãn đúng hay sai, mà còn cần học một quan hệ thứ tự có ý nghĩa giữa các đối tượng. Trong cài đặt score-based ranking, mô hình học một hàm điểm $h : \mathcal{X} \rightarrow \mathbb{R}$ rồi suy ra thứ hạng bằng cách so sánh các giá trị điểm. Cách nhìn này giúp đưa ranking về các bài toán quen thuộc hơn như phân loại nhị phân trên cặp mẫu, tối ưu lồi trong Ranking SVM, hoặc tối ưu surrogate loss trong RankBoost. Tuy nhiên, nó cũng làm rõ một hạn chế quan trọng: một hàm điểm luôn sinh ra thứ tự có tính bắc cầu, trong khi quan hệ ưu tiên thực tế có thể không bắc cầu.

Về mặt lý thuyết, báo cáo đã trình bày các định nghĩa rủi ro xếp hạng, sai số thực nghiệm, các chặn tổng quát hóa dựa trên Rademacher complexity, Ranking SVM, RankBoost, bipartite ranking, ROC/AUC và preference-based ranking. Các chứng minh chính đều đi theo tinh thần của giáo trình: chuyển bài toán ranking thành kiểm soát sai số trên các cặp, sau đó dùng margin, convex surrogate hoặc phân tích độ phức tạp giả thuyết để thu được bảo đảm học. Đặc biệt, RankBoost minh họa rõ mối liên hệ giữa boosting và tối ưu hóa theo tọa độ: mỗi vòng chọn một weak ranker để giảm exponential loss, còn sai số thực nghiệm được chặn bởi tích các hệ số chuẩn hóa Z_t . Với bipartite ranking, mối liên hệ giữa rủi ro xếp hạng và AUC cho thấy AUC không chỉ là một độ đo thực nghiệm, mà còn là một cách diễn đạt trực tiếp xác suất một mẫu dương được xếp cao hơn một mẫu âm.

Phần thực nghiệm cài đặt các thuật toán chính từ đầu bằng Python, không dùng thư viện học máy cấp cao cho phần huấn luyện cốt lõi. Các thí nghiệm trên dữ liệu tổng hợp kiểm chứng ba ý quan trọng: sai số RankBoost giảm phù hợp với chặn lý thuyết, bipartite ranking có quan hệ định lượng với AUC, và RankBoost có thể tận dụng weak ranker phi tuyến đơn giản để đạt sai số tốt hơn Ranking SVM tuyến tính trên dữ liệu có cấu trúc phi tuyến. Các hình vẽ được dùng để nối phần công thức với trực giác: đường hội tụ cho thấy vai trò của Z_t , ROC mô tả đánh đổi giữa true positive rate và false positive rate, còn biểu đồ so sánh giúp thấy khác biệt giữa hai họ thuật toán.

Phần nghiên cứu liên quan mở rộng nội dung chương theo hai hướng. Bài báo *Active Bipartite Ranking* mở rộng nội dung ROC/AUC sang thiết lập active learning, nơi thuật toán phải chọn nhãn cần hỏi để giảm chi phí dữ liệu. Bài báo về các “tricks” trong Learning to Rank hiện đại cho thấy các ý tưởng surrogate loss, smoothing và lựa chọn cặp lân cận vẫn là trung tâm trong các hệ thống GBDT-based LTR sau này. Như vậy, lý thuyết trong chương không bị tách khỏi thực hành, mà là nền tảng để hiểu các thuật toán ranking hiện đại hơn.

Báo cáo vẫn có một số hạn chế. Thứ nhất, phần thực nghiệm chủ yếu dùng dữ liệu tổng hợp để kiểm soát giả thiết, nên chưa phản ánh đầy đủ nhiều, mất cân bằng truy vấn và đặc trưng thừa trong các bộ dữ liệu Learning to Rank thực tế. Thứ hai, các cài đặt từ đầu ưu tiên tính minh họa và khả năng đọc hiểu, chưa tối ưu cho quy mô lớn. Thứ ba, chương tập trung nhiều vào ranking cổ điển dựa trên score và pairwise comparison, trong khi các hướng hiện đại như neural ranking, listwise optimization, fairness trong ranking và learning from implicit feedback chỉ mới được liên hệ ở mức tổng quan.

Các hướng mở tự nhiên là thử nghiệm trên các bộ dữ liệu chuẩn như LETOR hoặc MSLR-WEB, mở rộng weak ranker của RankBoost từ decision stump sang cây sâu hơn, so sánh thêm các surrogate loss listwise cho NDCG, và nghiên cứu ảnh hưởng của tie-breaking trong AUC khi điểm số rời rạc. Một hướng khác là nối preference-based ranking với các mô hình neural ranker: thay vì ép mọi quan hệ ưu tiên về một thứ tự tuyến tính

duy nhất, ta có thể học trực tiếp cấu trúc ưu tiên phức tạp hơn. Nhìn chung, chương Ranking cung cấp một khung nền tảng đủ chặt chẽ để hiểu cả lý thuyết tổng quát hóa lẫn các thiết kế thuật toán thực tế trong học xếp hạng.

Tài liệu

- [1] James Cheshire, Vincent Laurent, and Stéphan Cléménçon. Active bipartite ranking. In *Advances in Neural Information Processing Systems*, 2023.
- [2] Yoav Freund, Raj Iyer, Robert E Schapire, and Yoram Singer. An efficient boosting algorithm for combining preferences. *Journal of Machine Learning Research*, 4(Nov): 933–969, 2003.
- [3] Ivan Lyzhin, Aleksei Ustimenko, Andrey Gulin, and Liudmila Prokhorenkova. Which tricks are important for learning to rank? In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22658–22684. PMLR, 2023.
- [4] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of machine learning*. MIT press, 2nd edition, 2018.